



Universidade Federal
do Rio de Janeiro

Escola Politécnica

MINERVAMESH: UMA PLATAFORMA WEB PARA SIMULAÇÕES NUMÉRICAS EM ENGENHARIA MECÂNICA

Luiz Gustavo Santos Farias

Projeto de Graduação apresentado ao
Curso de Engenharia Mecânica da Escola
Politécnica, Universidade Federal do Rio
de Janeiro, como parte dos requisitos ne-
cessários à obtenção do título de Engenheiro
Mecânico.

Orientador: Gustavo Rabello dos Anjos

Rio de Janeiro

Maio de 2026

MINERVAMESH: UMA PLATAFORMA WEB PARA SIMULAÇÕES NUMÉRICAS EM ENGENHARIA MECÂNICA

Luiz Gustavo Santos Farias

PROJETO DE GRADUAÇÃO SUBMETIDO AO CORPO DOCENTE DO CURSO
DE ENGENHARIA MECÂNICA DA ESCOLA POLITÉCNICA DA UNIVERSI-
DADE FEDERAL DO RIO DE JANEIRO COMO PARTE DOS REQUISITOS NE-
CESSÁRIOS PARA A OBTENÇÃO DO GRAU DE ENGENHEIRO MECÂNICO

Examinada por:

Prof. Gustavo Rabello dos Anjos, D. Sc.

Prof. Examinador 1, D. Sc.

Prof. Examinador 2, D. Sc.

Rio de Janeiro

Maio de 2026

UNIVERSIDADE FEDERAL DO RIO DE JANEIRO

Escola Politécnica - Departamento de Engenharia Mecânica

Centro de Tecnologia, bloco G, sala G-204, Cidade Universitária

Rio de Janeiro - RJ CEP 21941-909

Este exemplar é de propriedade da Universidade Federal do Rio de Janeiro, que poderá incluí-lo em base de dados, armazenar em computador, microfilmear ou adotar qualquer forma de arquivamento.

É permitida a menção, reprodução parcial ou integral e a transmissão entre bibliotecas deste trabalho, sem modificação de seu texto, em qualquer meio que esteja ou venha a ser fixado, para pesquisa acadêmica, comentários e citações, desde que sem finalidade comercial e que seja feita a referência bibliográfica completa.

Os conceitos expressos neste trabalho são de responsabilidade do(s) autor(es).

DEDICATÓRIA

Dedico este trabalho, primeiramente, a Deus, Senhor da minha vida, que sustentou cada etapa desta caminhada e me concedeu graça, perseverança e propósito quando as forças pareciam insuficientes.

À minha esposa Aretha, companheira fiel, amiga e apoio constante em todos os momentos, cujo amor, paciência e incentivo tornaram possível a conclusão desta jornada.

À minha filha Marina, que mesmo sem compreender totalmente o significado deste trabalho, foi diariamente a maior motivação para não desistir. Que este diploma seja também um testemunho para você de que vale a pena terminar o que começamos.

Aos meus pais, Daniela e Silvio, que sempre acreditaram na educação como instrumento de transformação e nunca deixaram faltar apoio, cuidado e exemplo.

Às minhas irmãs, Yngrid e Yasmim, pela presença, amizade e pelo vínculo familiar que sempre me fortaleceu.

Comecei este caminho como estudante. Concluo como engenheiro, marido, pai e servo de Cristo.

AGRADECIMENTO

Agradeço primeiramente a Deus, por sustentar cada etapa desta caminhada e por conceder sabedoria e perseverança para chegar até aqui.

Ao meu orientador, Prof. Gustavo Rabello dos Anjos, pela orientação cuidadosa, confiança e incentivo intelectual, fundamentais para o desenvolvimento deste trabalho e para o amadurecimento da proposta do MinervaMesh.

À Universidade Federal do Rio de Janeiro e ao corpo docente do curso de Engenharia Mecânica, pela formação técnica e científica que tornou possível a realização deste projeto.

Ao meu grande amigo e padrinho de casamento, Michel Silva Bonifácio, pelo apoio durante a graduação e pela presença constante em momentos decisivos da minha trajetória acadêmica.

Ao meu amigo Eliab Araújo, pelo incentivo para a conclusão do curso e pela motivação contínua no desenvolvimento deste trabalho.

Aos colegas, amigos e a todos que contribuíram direta ou indiretamente para minha formação como engenheiro.

RESUMO

A simulação numérica é ferramenta central de análise e projeto em Engenharia Mecânica, mas seu uso no ensino de métodos numéricos esbarra em duas dificuldades recorrentes: as ferramentas mais capazes exigem licenciamento, infraestrutura dedicada e configuração de ambiente, e raramente expõem de forma transparente a formulação matemática e o código por trás de cada simulação. O problema atacado neste trabalho é, portanto, reduzir a barreira de acesso a um ambiente de simulação numérica didático, sem abrir mão do rigor técnico nem da transparência da implementação. Como resposta, apresenta-se o *MinervaMesh*, uma plataforma que resolve Equações Diferenciais Parciais de interesse em Engenharia Mecânica pelo Método dos Elementos Finitos (MEF) e pelo Método das Diferenças Finitas (MDF). A contribuição articula dois aspectos complementares, subordinados a esse único objetivo: (i) a implementação e a verificação dos métodos numéricos, confrontadas prioritariamente com soluções analíticas conhecidas e, nos casos sem solução fechada, com *benchmarks* consagrados da literatura; e (ii) a entrega dessas simulações integralmente no navegador, em arquitetura *client-side* sobre *WebAssembly* e *PyScript*, sem instalação e com o código-fonte Python inspecionável em cada caso. Os resultados mostram que as equações discretizadas reproduzem corretamente as soluções de referência e que a plataforma é adequada ao ensino, à prototipagem e a demonstrações acadêmicas de mecânica computacional.

Palavras-Chave: Método dos Elementos Finitos, Método das Diferenças Finitas, Simulação Web, Mecânica Computacional, Ensino de Engenharia.

ABSTRACT

Numerical simulation is a core tool for analysis and design in Mechanical Engineering, yet teaching numerical methods runs into two recurring obstacles: the most capable tools require licensing, dedicated infrastructure, and environment setup, and they seldom expose the underlying mathematical formulation and source code of each simulation. The problem addressed in this work is therefore to lower the barrier of access to a didactic numerical simulation environment, without sacrificing technical rigor or implementation transparency. As a response, this work presents *MinervaMesh*, a platform that solves Partial Differential Equations of interest in Mechanical Engineering with the Finite Element Method (FEM) and the Finite Difference Method (FDM). The contribution combines two complementary aspects, subordinated to this single goal: (i) the implementation and verification of the numerical methods, assessed primarily against known analytical solutions and, for cases without a closed-form solution, against well-established benchmarks; and (ii) the delivery of these simulations fully in the browser, using a *client-side* architecture built on *WebAssembly* and *PyScript*, with no installation and with inspectable Python source code for every case. Results show that the discretized equations correctly reproduce the reference solutions and that the platform is suitable for teaching, rapid prototyping, and academic demonstrations in computational mechanics.

Key-words: Finite Element Method, Finite Difference Method, Web Simulation, Computational Mechanics, Engineering Education.

SIGLAS

UFRJ - Universidade Federal do Rio de Janeiro

MEF - Método dos Elementos Finitos

MDF - Método das Diferenças Finitas

CFD - *Computational Fluid Dynamics* (Dinâmica dos Fluidos Computacional)

WASM - WebAssembly

SUPG - *Streamline Upwind/Petrov-Galerkin*

Sumário

1	Introdução	1
1.1	Introdução	1
1.2	Objetivos	2
1.3	Justificativa	3
1.4	Organização do Trabalho	4
2	Revisão Bibliográfica	5
2.1	Introdução ao Capítulo	5
2.2	Problemas Térmicos e Equações Governantes	5
2.2.1	Condução de Calor	5
2.2.2	Convecção e Acoplamento com Escoamento	6
2.2.3	Radiação Térmica	6
2.3	Métodos Numéricos para EDPs	6
2.3.1	Método das Diferenças Finitas (MDF)	6
2.3.2	Método dos Elementos Finitos (MEF)	7
2.4	Bases Históricas e Consolidação do MEF/CFD	7
2.5	Trabalhos Correlatos	9
2.5.1	Transferência de Calor e Condução em Componentes	9
2.5.2	Escoamento e CFD pela Formulação Vorticidade-Corrente	10
2.5.3	Comparação de Métodos e Fundamentação Variacional	10
2.6	Síntese da Revisão e Lacuna Atacada	11
3	Fundamentação Teórica	12
3.1	Introdução ao Capítulo	12
3.2	Equações Governantes	12

3.2.1	Condução e Convecção Térmica	13
3.2.2	Escoamento de Fluidos (Navier-Stokes)	13
3.3	Transição Discreta: O Método das Diferenças Finitas (MDF)	14
3.4	O Método dos Elementos Finitos (MEF)	15
3.4.1	Da Forma Forte à Forma Fraca	15
3.4.2	Condições de Contorno	16
3.4.3	O Método de Galerkin	18
3.4.4	Elementos Triangulares Lineares	19
3.4.5	Matrizes Elementares	20
3.4.6	Montagem Global	21
3.5	Formulação Vorticidade-Corrente	21
3.5.1	Definições	21
3.5.2	Equação de Transporte da Vorticidade	22
3.5.3	Sistema Acoplado e Solução Iterativa	22
3.5.4	Matriz de Convecção e Estabilização SUPG	22
3.6	Integração Temporal	23
3.6.1	Método θ Generalizado	23
3.6.2	Esquema Semi-Implícito Adotado	24
3.6.3	Condição de Contorno de Vorticidade na Parede	24
3.7	Síntese do Capítulo	25
4	Resultados e Validação	26
4.1	Introdução	26
4.2	Condução Permanente 1D	27
4.2.1	Formulação do Problema	28
4.2.2	Solução Analítica	28
4.2.3	Solução MEF e Comparação	29
4.2.4	Análise de Erro	29
4.3	Condução Transiente 1D com $k(x)$ Variável	30
4.3.1	Formulação do Problema	31
4.3.2	Solução Analítica de Regime Permanente	32
4.3.3	Solução MEF e Comparação	32
4.3.4	Análise de Erro	34

4.4	Convecção-Difusão 1D com Estabilização SUPG	34
4.4.1	Formulação do Problema	34
4.4.2	Solução Analítica	35
4.4.3	Solução MEF com SUPG	36
4.4.4	Análise de Erro	36
4.5	Viga 1D — Flexão de Euler-Bernoulli	39
4.5.1	Formulação do Problema	39
4.5.2	Solução Analítica	39
4.5.3	Solução MEF e Comparação	40
4.5.4	Análise de Erro	40
4.6	Vibração 1D — Problema de Autovalor	42
4.6.1	Formulação do Problema	43
4.6.2	Solução Analítica	43
4.6.3	Solução MEF e Comparação	44
4.6.4	Análise de Erro	44
4.7	Transferência de Calor Transiente 2D	46
4.7.1	Formulação do Problema	46
4.7.2	Solução Analítica de Regime Permanente	47
4.7.3	Solução MEF e Comparação	48
4.7.4	Análise de Erro	48
4.8	Escoamento Potencial 2D em Torno de Cilindro	50
4.8.1	Formulação do Problema	50
4.8.2	Solução Analítica em Domínio Infinito	51
4.8.3	Solução MEF e Comparação	52
4.8.4	Análise de Erro	52
4.9	Navier-Stokes 2D — Cavidade com Tampa Móvel	54
4.9.1	Formulação do Problema	55
4.9.2	Benchmark de Referência	55
4.9.3	Solução MEF e Comparação	56
4.9.4	Análise de Erro	56
5	Desenvolvimento e Arquitetura	60
5.1	Visão Geral da Plataforma	60

5.2	Justificativa Tecnológica	62
5.2.1	PyScript e WebAssembly	62
5.2.2	Computação <i>Client-Side</i>	62
5.2.3	Comparação com Alternativas	63
5.2.4	Análise Computacional	64
5.3	Arquitetura de Software	66
5.3.1	Organização de Diretórios	66
5.3.2	Fluxo de Execução de uma Simulação	67
5.4	Módulos de Simulação MEF	68
5.4.1	Condução Permanente 1D	69
5.4.2	Condução Transiente 1D com $k(x)$ Variável	70
5.4.3	Convecção-Difusão 1D com Estabilização SUPG	70
5.4.4	Viga 1D — Análise de Flexão (Euler-Bernoulli)	71
5.4.5	Vibração 1D — Problema de Autovalor	73
5.4.6	Transferência de Calor 2D	74
5.4.7	Escoamento Potencial 2D	75
5.4.8	Navier-Stokes 2D (Vorticidade-Corrente)	76
5.5	Módulos Complementares em MDF	78
5.6	Limitações da Plataforma	79
5.7	Síntese do Capítulo	82
6	Conclusões	84
6.1	Conclusões	84
6.2	Trabalhos Futuros	86
	Bibliografia	88
A	Listagens dos Módulos de Simulação 1D	93
A.1	Condução Permanente 1D	93
A.2	Condução Transiente 1D com $k(x)$ Variável	97
A.3	Convecção-Difusão 1D com SUPG	106
A.4	Viga 1D — Euler-Bernoulli	112
A.5	Vibração 1D — Problema de Autovalor	127

B	Listagens dos Módulos de Simulação 2D	139
B.1	Malha e Matrizes MEF	139
B.2	Geometrias de Obstáculo	142
B.3	Transferência de Calor 2D	145
B.4	Escoamento Potencial 2D	150
B.5	Navier-Stokes 2D (Vorticidade-Corrente)	162
C	Scripts de Benchmark Computacional	176
C.1	Solver de Condução Permanente 1D Adaptado	176
C.2	Solver de Transcalor 2D Adaptado	178
C.3	Biblioteca Auxiliar de Cronometragem	182
C.4	Runner Local (CPython)	185
C.5	Runner no Navegador (PyScript)	188

Lista de Figuras

- 4.1 Condução permanente 1D: comparação entre a solução MEF (pontos azuis, $n_e = 10$ elementos lineares) e a solução analítica (linha tracejada vermelha). A sobreposição exata confirma a propriedade de reprodução polinomial dos elementos lineares para fontes uniformes. 30
- 4.2 Condução transiente 1D com condutividade descontínua ($k_1 = 5$, $k_2 = 1$): solução MEF no instante final $t = 0,2$ s (pontos azuis) comparada ao perfil analítico de regime permanente (linha tracejada vermelha). A curva verde tracejada no eixo secundário indica a variação de $k(x)$ com a descontinuidade em $x = L/2$. A malha de 40 elementos lineares com $\theta = 1/2$ reproduz fielmente o *kink* de condutividade, com $T(L/2) \approx 1/6$ 33
- 4.3 Convecção-difusão 1D com SUPG ativo em $Pe_e = 0,5$. Painel superior: temperatura $T(x)$ — pontos azuis para o MEF e linha tracejada vermelha para a solução analítica, visualmente coincidentes em toda a malha. Painel inferior: erro absoluto $|T^{\text{MEF}} - T^{\text{ana}}|$ em escala logarítmica, com máximo $4,44 \times 10^{-16}$ °C — precisão de máquina. 37
- 4.4 Convecção-difusão 1D com Galerkin clássico (SUPG desativado), mesmos parâmetros da figura anterior. O painel superior mostra ainda concordância visual em escala linear; o painel inferior de erro revela, entretanto, crescimento exponencial do desvio em direção à camada-limite ($x \rightarrow L$), atingindo máximo $3,39 \times 10^{-2}$ °C próximo ao contorno direito. 38

4.5	Viga simplesmente apoiada com carga uniforme: (i) deflexão $w(x)$ com MEF (pontos azuis) e analítica (linha vermelha) visualmente coincidentes; (ii) perfil da carga distribuída constante $q = 10 \text{ kN/m}$; (iii) propriedades materiais E e I uniformes ao longo do vão; (iv) momento fletor $M(x)$ parabólico com máximo $qL^2/8 = 5,0 \text{ kN}\cdot\text{m}$ no centro; (v) esforço cortante $V(x)$ linear decrescente de $+qL/2 = +10 \text{ kN}$ no apoio esquerdo a -10 kN no direito. M e V são obtidos por pós-processamento da deflexão nodal.	41
4.6	Cinco primeiros modos de vibração axial da barra livre-livre. Tabela superior: frequências MEF e analíticas com erro relativo. Painéis seguintes: forma modal normalizada (pontos e linha contínua, uma cor por modo), do modo 1 rígido ($f_1 = 0 \text{ Hz}$, amplitude uniforme) aos modos 2–5 com número crescente de meio-comprimentos de onda. Painel inferior: propriedades materiais (E e ρ) constantes ao longo do vão.	45
4.7	Transferência de calor transiente 2D com os <i>defaults</i> do módulo, em $t = 0,049 \text{ s}$: o campo $T(x, y)$ mostra a perturbação térmica propagando-se a partir das paredes aquecida (topo, $T_{\text{top}} = 100^\circ\text{C}$) e morna (base, $T_{\text{bot}} = 30^\circ\text{C}$) para o interior da placa, ainda inicialmente em $T = 0^\circ\text{C}$. O <i>colormap</i> jet varre a faixa de temperatura instantânea e exibe a natureza bidimensional da difusão: a frente de calor do topo penetra mais rapidamente que a da base devido ao maior gradiente ΔT	49

4.8	Escoamento potencial em torno de cilindro, três painéis empilhados: (i) função de corrente $\psi(x, y)$ com a malha triangular sobreposta, destacando o obstáculo circular no centro; (ii) linhas de corrente contornando o cilindro, coloridas pela magnitude de velocidade (<i>colormap viridis</i>) — simetria aproximada do escoamento visível; (iii) distribuição de $C_p(\theta)$ sobre a superfície do cilindro, comparando a solução numérica (azul) à teoria potencial $C_p = 1 - 4 \sin^2 \theta$ (vermelho tracejado). Os pontos de estagnação frontal ($\theta = 0$) e traseiro ($\theta \approx \pm\pi$) são reproduzidos com pequeno desvio; os polos de pressão mínima próximos de $\theta = \pm 90^\circ$ exibem assimetria devida ao confinamento lateral do domínio ($2r/H = 30\%$) e ao viés da malha estruturada.	53
4.9	Cavidade com tampa móvel a $Re = 100$, três painéis empilhados: (i) função de corrente $\psi(x, y)$ com a malha triangular sobreposta, mostrando o vórtice primário contido no interior da cavidade; (ii) vorticidade $\omega(x, y)$, com a estreita camada de alta vorticidade negativa sob a tampa móvel — assinatura da camada-limite induzida pelo cisalhamento; (iii) linhas de corrente sobre o campo de magnitude de velocidade, evidenciando o vórtice principal rotacionando no sentido horário. O vórtice primário obtido está centrado em (0,625; 0,750) com $\psi_{\min} \approx -0,103$, em concordância com a referência de Ghia, Ghia e Shin (1982).	57
4.10	Perfil $u(y)$ em $x = 0,5$ comparado aos valores tabulados por Ghia, Ghia e Shin (1982) para a cavidade com tampa móvel a $Re = 100$. RMS = 0,036 em malha 41×41 com 2000 passos de tempo.	58
5.1	Tela inicial do <i>MinervaMesh</i> com o menu de simulações disponíveis.	61
5.2	Escalabilidade do tempo total do solver com o número de nós, em escala log-log, comparando CPython nativo (linha sólida) e Pydide no navegador (linha tracejada). À esquerda, condução 1D MEF (denso) com inclinação compatível com $\mathcal{O}(N^3)$; à direita, transcalor 2D MEF (esparso, 50 passos) com inclinação próxima de $\mathcal{O}(N^{1,2})$	65
5.3	Interface de parametrização de uma simulação (Condução Permanente 1D) antes da execução.	68

5.4	Condução permanente 1D: solução MEF ($N_{el} = 10$) comparada à solução analítica.	69
5.5	Evolução temporal do campo de temperatura com condutividade variável $k(x)$	70
5.6	Convecção-difusão 1D com estabilização SUPG ($Pe \gg 1$): solução MEF comparada à analítica.	71
5.7	Análise de flexão de viga 1D (Euler-Bernoulli): deflexão, carrega- mento, propriedades, momento fletor e esforço cortante.	73
5.8	Frequências naturais e cinco primeiros modos de vibração de uma barra 1D.	74
5.9	Campo de temperatura em placa 2D durante a evolução transiente (método de Crank-Nicolson).	75
5.10	Escoamento potencial 2D: linhas de corrente ao redor de um obstáculo.	76
5.11	Navier-Stokes 2D — cavidade com tampa móvel ($Re = 100$): campos de ψ , ω e linhas de corrente.	78

Lista de Tabelas

3.1	Classificação variacional e tratamento numérico das condições de contorno no MEF.	17
4.1	Parâmetros da simulação de condução permanente 1D.	28
4.2	Parâmetros da simulação de condução transiente 1D com $k(x)$ variável.	31
4.3	Parâmetros da simulação de convecção-difusão 1D.	35
4.4	Parâmetros da simulação de flexão de viga 1D.	40
4.5	Parâmetros da simulação de vibração axial 1D (barra de aço bi-engastada).	43
4.6	Frequências naturais: MEF vs. analítica para barra livre-livre.	44
4.7	Parâmetros da simulação de transferência de calor transiente 2D.	47
4.8	Parâmetros da simulação de escoamento potencial 2D em torno de cilindro.	51
4.9	Coefficiente de pressão C_p na superfície do cilindro: MEF vs. teoria potencial.	52
4.10	Parâmetros da simulação de Navier-Stokes 2D — cavidade com tampa móvel.	55
5.1	Comparação entre plataformas de simulação numérica para ensino.	63
5.2	Tempo total do solver (mediana, com MAD reportado abaixo de 1% em todas as células) em CPython nativo e Pyodide (navegador), na mesma máquina (Apple M1 Max). t_{mont} é o tempo de montagem (incluindo geração de malha quando aplicável) e t_{slv} é o tempo da resolução linear (<code>np.linalg.solve</code> no caso 1D denso, ou o total dos passos de <code>spsolve</code> nos casos 2D esparsos). “CP” = CPython nativo; “Py” = Pyodide.	64

Capítulo 1

Introdução

1.1 Introdução

A simulação numérica consolidou-se como instrumento central da engenharia contemporânea, complementando abordagens teóricas e experimentais na investigação de fenômenos físicos complexos. No contexto da Engenharia Mecânica, a Dinâmica dos Fluidos Computacional (*Computational Fluid Dynamics* – CFD) e a modelagem de transferência de calor viabilizam a análise de problemas governados por Equações Diferenciais Parciais (EDPs), muitas vezes inviáveis por solução analítica direta ou por experimentação em escala de laboratório [1]. Essa capacidade preditiva tem papel relevante no dimensionamento, na otimização e na verificação de desempenho de sistemas térmicos e fluidodinâmicos.

Apesar dos avanços metodológicos, o acesso a ambientes de simulação permanece limitado por barreiras econômicas e operacionais. Soluções comerciais amplamente utilizadas exigem licenciamento oneroso, infraestrutura computacional específica e configuração de ambiente com elevado custo de entrada, sobretudo em cenários de ensino e pesquisa inicial. Como consequência, a formação em métodos numéricos frequentemente fica condicionada à disponibilidade de laboratório e suporte técnico especializado, o que reduz a reprodutibilidade e a acessibilidade das atividades acadêmicas [2].

Neste contexto, este trabalho ataca o problema de oferecer um ambiente de simulação numérica simultaneamente **acessível**, **didático** e **transparente** para o

ensino de Engenharia Mecânica. A resposta proposta é o *MinervaMesh*, uma plataforma executada integralmente no navegador, com arquitetura *client-side*, que combina Python científico com tecnologias web modernas — notadamente *WebAssembly* e *PyScript* — para executar localmente rotinas de MEF e MDF sem qualquer instalação de dependências pelo usuário: o PyScript [3] executa, no próprio navegador, a pilha científica do Python, incluindo o NumPy [4]. A proposta articula dois aspectos sob esse mesmo objetivo: de um lado, a *implementação e a verificação* dos métodos numéricos, confrontadas com soluções analíticas e *benchmarks* de referência; de outro, a *disponibilização* dessas simulações na web, com formulação matemática explícita e código-fonte inspecionável em cada caso. Os dois aspectos não constituem trabalhos paralelos, mas meios complementares para reduzir a barreira de acesso ao aprendizado de métodos numéricos — a corretude dos métodos dá fundamento à plataforma, e a entrega na web a torna efetivamente utilizável por terceiros.

1.2 Objetivos

O objetivo geral deste trabalho é desenvolver e validar o **MinervaMesh**, uma plataforma computacional baseada na web para solução de EDPs de interesse em Engenharia Mecânica, operando integralmente no lado do cliente (*client-side*).

Os objetivos específicos são:

1. Implementar métodos numéricos clássicos, com ênfase no Método dos Elementos Finitos (MEF) e uso complementar do Método das Diferenças Finitas (MDF), empregando bibliotecas científicas Python no ambiente web [4].
2. Desenvolver uma interface interativa para pré-processamento, solução e pós-processamento, incluindo parametrização de malha e visualização de campos de interesse.
3. Validar a plataforma, prioritariamente, por meio de casos com solução analítica conhecida e, em seguida, demonstrar sua aplicação em casos adicionais de engenharia, com foco em mecânica dos fluidos e transferência de calor.

4. Consolidar um fluxo de trabalho reprodutível para problemas de:

- Escoamentos potenciais;
- Problemas de advecção-difusão (com estabilização SUPG) [5];
- Equações de Navier-Stokes para escoamentos laminares incompressíveis.

1.3 Justificativa

A relevância do trabalho reside na articulação entre rigor numérico e acessibilidade computacional. Ao viabilizar simulações diretamente no navegador, reduz-se o atrito inicial de aprendizado e amplia-se o potencial de uso em disciplinas de graduação, monitorias e estudos independentes. Adicionalmente, a explicitação da formulação matemática e de sua implementação computacional fortalece o caráter acadêmico da proposta, permitindo que a plataforma seja utilizada tanto como ferramenta de simulação quanto como objeto de estudo em métodos numéricos [6].

No espectro de ferramentas disponíveis para o ensino e a prototipagem de métodos numéricos, o *MinervaMesh* ocupa um nicho distinto das alternativas predominantes. Bibliotecas robustas de MEF em Python, como o FEniCS, e plataformas especializadas em métodos espectrais, como o Dedalus, oferecem grande flexibilidade e rigor, mas exigem instalação local de ambientes científicos e familiaridade com suas linguagens de domínio; suítes comerciais como COMSOL Multiphysics e ANSYS entregam interfaces gráficas de alto nível, porém seu código é proprietário e seu licenciamento restringe o uso acadêmico autônomo; e demonstrações web em *p5.js* ou JavaScript puro, embora acessíveis, raramente apresentam formulação matemática explícita ou código inspecionável em linguagem científica consolidada. A contribuição original do *MinervaMesh* reside na combinação simultânea de três atributos: execução integralmente no navegador sem qualquer instalação, código-fonte Python inspecionável em cada simulação através do botão “Ver Código”, e portfólio unificado cobrindo três domínios clássicos da Engenharia Mecânica em uma única interface consistente.

1.4 Organização do Trabalho

Este trabalho está estruturado da seguinte forma:

- O **Capítulo 2** (Revisão Bibliográfica) apresenta o histórico e o estado da arte dos temas que sustentam o trabalho, incluindo problemas térmicos, MDF, MEF e trabalhos correlatos.
- O **Capítulo 3** (Fundamentação Teórica) apresenta o embasamento matemático necessário para a discretização de EDPs via MEF.
- O **Capítulo 4** (Resultados e Validação) apresenta a verificação numérica dos métodos implementados, comparando as soluções discretizadas com soluções analíticas conhecidas e *benchmarks* consagrados da literatura.
- O **Capítulo 5** (Desenvolvimento e Arquitetura) detalha a implementação do *MinervaMesh*, com ênfase na arquitetura *client-side* e PyScript que disponibiliza esses métodos na web.
- O **Capítulo 6** (Conclusão) resume as contribuições e propõe trabalhos futuros.

Capítulo 2

Revisão Bibliográfica

2.1 Introdução ao Capítulo

Este capítulo apresenta a revisão bibliográfica que fundamenta o desenvolvimento do *MinervaMesh*, com foco em problemas térmicos e de mecânica dos fluidos resolvidos por Métodos Numéricos, em especial o Método das Diferenças Finitas (MDF) e o Método dos Elementos Finitos (MEF). A revisão está organizada em quatro blocos: fundamentos de transferência de calor, bases dos métodos numéricos, marcos do MEF/CFD e trabalhos correlatos.

2.2 Problemas Térmicos e Equações Governantes

2.2.1 Condução de Calor

A condução é descrita pela Lei de Fourier e pela equação da difusão térmica, cujo tratamento de referência em engenharia é consolidado por Incropera et al. [7] e por Çengel e Ghajar [8]. Em regime estacionário, a formulação conduz às equações de Laplace e Poisson; em regime transiente, inclui-se o termo temporal associado ao armazenamento de energia. Esses modelos são a base de problemas clássicos de engenharia, como dissipação em componentes eletrônicos e análise térmica de discos de freio.

2.2.2 Convecção e Acoplamento com Escoamento

Quando há interação sólido-fluido, a convecção influencia a taxa de remoção de calor e exige o acoplamento entre as equações de energia e de escoamento. Os fundamentos físicos desse acoplamento são tratados na literatura clássica de mecânica dos fluidos, dos conceitos introdutórios de Fox et al. [9] à análise de escoamentos viscosos de White [10]. A solução numérica desses problemas convectivo-difusivos é desenvolvida por Maliska [1], voltado conjuntamente a calor e fluidos, e introduzida didaticamente por Versteeg e Malalasekera [11].

2.2.3 Radiação Térmica

Em aplicações de alta temperatura, os termos radiativos podem se tornar relevantes no balanço térmico global, tema tratado em profundidade nos textos clássicos de transferência de calor [7]. Embora o escopo principal deste trabalho esteja em condução e convecção, a revisão considera a radiação como extensão natural para trabalhos futuros.

2.3 Métodos Numéricos para EDPs

2.3.1 Método das Diferenças Finitas (MDF)

O MDF aproxima derivadas por expansões de Taylor, convertendo EDPs em sistemas algébricos. Sua implementação é direta em malhas estruturadas e, por isso, ele é frequentemente adotado em validações iniciais e em discretização temporal de problemas transientes [12].

Esquemas explícitos e implícitos (Euler e Crank-Nicolson) apresentam diferentes compromissos entre custo computacional, estabilidade e precisão. Em problemas dominados por difusão, esquemas implícitos são preferidos por robustez numérica [11].

2.3.2 Método dos Elementos Finitos (MEF)

O MEF oferece maior flexibilidade geométrica por meio de malhas não estruturadas e de uma formulação variacional, característica decisiva em domínios complexos comuns em aplicações reais de engenharia térmica e de fluidos. Seu desenvolvimento e fundamentação rigorosa são consolidados no tratado de referência de Zienkiewicz et al. [13]. Para fins de ensino, textos introdutórios como o de Fish e Belytschko [6] apresentam o método de forma progressiva, do método dos resíduos ponderados às aplicações em análise estrutural e térmica. A aplicação específica do MEF a problemas acoplados de transferência de calor e escoamento é tratada em detalhe por Lewis et al. [14].

A formulação de Galerkin transforma a EDP na forma fraca e permite a montagem de matrizes globais de rigidez, massa e convecção; sua dedução a partir de princípios variacionais é apresentada de forma detalhada por Reddy [15]. Para elementos lineares triangulares, a implementação resultante é eficiente e adequada a plataformas computacionais didáticas, como ilustram os textos introdutórios de Fish e Belytschko [6].

2.4 Bases Históricas e Consolidação do MEF/CFD

O desenvolvimento do MEF resulta de uma sucessão de contribuições distintas, e não de um marco único. Galerkin [16] introduziu o método dos resíduos ponderados que leva seu nome, no qual a solução aproximada é obtida impondo a ortogonalidade do resíduo às próprias funções de base, dispensando a existência prévia de um princípio variacional. Courant [17] formalizou o emprego de funções de interpolação contínuas por partes sobre subdomínios triangulares na solução de problemas de equilíbrio e vibração, antecipando o conceito de elemento. Hrennikoff [18], por sua vez, propôs o *framework method*, no qual o meio contínuo elástico é discretizado por uma treliça de barras com propriedades equivalentes — um precursor direto da ideia de discretização do domínio. A estruturação do MEF moderno aplicado à engenharia veio com Turner et al. [19], que formularam a análise matricial de rigidez de estruturas aeronáuticas complexas a partir de elementos triangulares e retangu-

lares, e com Clough [20], a quem se atribui a cunhagem do próprio termo *método dos elementos finitos*.

No tratamento do transporte convectivo e das equações de Navier-Stokes, a literatura desenvolveu formulações estabilizadas para suprimir as oscilações espúrias da formulação de Galerkin padrão em regime de convecção dominante. Christie et al. [21] propuseram funções de peso assimétricas (*upwind*) para equações de segunda ordem com termos de primeira derivada significativos, um dos primeiros tratamentos consistentes do problema. Brooks e Hughes [5] generalizaram essa ideia na formulação SUPG (*Streamline Upwind/Petrov-Galerkin*), que adiciona difusão artificial apenas na direção das linhas de corrente e tornou-se referência para escoamentos incompressíveis. Donea [22] introduziu o método Taylor-Galerkin, que deriva o esquema estabilizado a partir de uma expansão de Taylor no tempo, obtendo baixa difusão numérica em problemas de advecção transiente. Löhner et al. [23] estenderam essa abordagem a sistemas hiperbólicos não lineares, realizando a discretização temporal antes da espacial para recuperar uma forma auto-adjunta compatível com o processo de Galerkin.

A validação de solucionadores de escoamento apoia-se em casos de referência consagrados, cada um associado a um regime distinto. Ghia et al. [24] produziram, por método multigrid, soluções de alta resolução para a cavidade com tampa móvel (*lid-driven cavity*) em números de Reynolds de até 10.000, estabelecendo o *benchmark* mais difundido para esse problema; Marchi et al. [25] refinaram posteriormente esse padrão com soluções em malha 1024×1024 e extrapolação de Richardson. Para o degrau regressivo (*backward-facing step*), Armaly et al. [26] forneceram medições experimentais por anemometria laser-Doppler acompanhadas de análise numérica, enquanto Thomas et al. [27] apresentaram uma das primeiras análises dessa mesma geometria por elementos finitos. No campo dos algoritmos, Pironneau [28] formalizou o método de transporte-difusão baseado em características, com aplicação direta às equações de Navier-Stokes.

Também se destaca a relevância de ferramentas de geração de malha na qualidade da solução, com destaque para o Gmsh em fluxos de trabalho acadêmicos e de

pesquisa [29].

2.5 Trabalhos Correlatos

Em projetos de graduação recentes do mesmo grupo de pesquisa, observa-se forte convergência para a implementação dos métodos numéricos em Python científico aplicada a problemas térmicos e de escoamento. Em vez de agrupá-los, os trabalhos mais próximos são resumidos individualmente a seguir, pois cada um aborda um problema físico e uma estratégia de implementação distintos.

2.5.1 Transferência de Calor e Condução em Componentes

Aguiar [30] desenvolveu uma rotina em Python para o dimensionamento térmico preliminar de discos de freio, resolvendo a equação axissimétrica de condução de calor por MEF e validando o código por convergência de malha e confronto com soluções analíticas. Alves [31] estendeu o problema do disco de freio para a equação tridimensional transiente do calor, também por MEF, com ênfase em oferecer alternativa de código aberto aos pacotes comerciais para comparar materiais e geometrias. Ferreira [32] aplicou o MEF em Python à análise térmica de processadores computacionais, contrastando o comportamento térmico sob diferentes discretizações em condições críticas de operação. Pinheiro [33] tratou a condução de calor em componentes eletrônicos de material heterogêneo, modelando a dissipação resistiva como geração interna e resolvendo o problema bidimensional por elementos finitos.

Capitanio [34] combinou MEF (Galerkin no espaço) e MDF (no tempo) para o estudo paramétrico de dissipadores de processadores, validando a metodologia por solução manufaturada, com erro quadrático médio da ordem de 10^{-4} a 10^{-6} conforme o refino de malha. Miranda [35] investigou geometrias de canais em placas frias para arrefecimento de eletrônicos, acoplando transferência de calor e escoamento pela formulação função-corrente-vorticidade e validando contra o escoamento de Poiseuille. Teixeira [36] resolveu a equação de Laplace tridimensional para a temperatura em um bloco de motor, empregando uma cadeia de ferramentas integralmente de código aberto (FreeCAD, Gmsh, Python/Julia) e comparando fluidos de arrefecimento. Oli-

veira [37] otimizou o número de aletas de um dissipador de calor de cobre por MEF em Python, validando o solver contra solução analítica com erro relativo da ordem de 10^{-11} .

2.5.2 Escoamento e CFD pela Formulação Vorticidade-Corrente

Na linha de mecânica dos fluidos, há forte convergência para a formulação vorticidade-corrente das equações de Navier-Stokes implementada em Python. Florentino [38] aplicou essa formulação, com estabilização Taylor-Galerkin, ao arrefecimento de eletrônicos, validando o código em casos clássicos (Poiseuille, cilindro em canal) antes de simular canais aletados. Santos [39] empregou a mesma formulação para investigar os campos de velocidade, vorticidade e função corrente em escoamento livre ao redor de diferentes geometrias, com a verificação apoiada nos escoamentos de Poiseuille e na cavidade com tampa móvel. Silva [40] acoplou a formulação vorticidade-corrente a uma descrição lagrangiana-euleriana (ALE) para simular as oscilações de um cilindro sob escoamento transversal, abordando a interação fluido-estrutura. Amaral [41] e Gomes [42] estenderam a formulação ao transporte convectivo-difusivo de espécie química em artérias coronárias — com ênfase, respectivamente, nas diferentes geometrias de *stent* farmacológico e nas configurações de restrição de fluxo —, evidenciando a necessidade de esquemas estabilizados em regime de convecção dominante.

2.5.3 Comparação de Métodos e Fundamentação Variacional

Dois trabalhos tratam diretamente da comparação entre métodos e da fundamentação matemática do MEF. Araujo [43] comparou solução analítica, MDF e MEF em problemas bidimensionais de condução de calor de complexidade crescente — até o caso transiente com geração de calor, sem solução fechada — caracterizando o comportamento do erro com o refinamento de malha; é a referência mais próxima do contraste $\text{MDF} \times \text{MEF}$ retomado neste trabalho. Gomes [44] apresentou a análise variacional da equação de Poisson em transferência de calor, estabelecendo existência e unicidade da solução fraca pelo teorema de Lax-Milgram e validando a

discretização de Galerkin (MEF em Python) por solução manufaturada, com erro relativo de aproximadamente 0,46%.

Em conjunto, esses trabalhos consolidam, no mesmo grupo de pesquisa, uma tradição de implementação dos métodos numéricos em Python científico para problemas térmicos e de escoamento. Todos, contudo, pressupõem a instalação local do ambiente Python e a execução em *desktop*; nenhum disponibiliza as simulações de forma interativa na web — lacuna sobre a qual o *MinervaMesh* se posiciona.

2.6 Síntese da Revisão e Lacuna Atacada

A revisão indica que: (i) há base teórica consolidada para problemas térmicos e de escoamento; (ii) MDF e MEF são métodos complementares, com o MEF mais adequado para geometrias complexas; e (iii) existe demanda por ferramentas didáticas acessíveis para implementação e validação desses métodos.

Nesse cenário, o *MinervaMesh* se posiciona ao integrar formulações clássicas de engenharia numérica em uma plataforma web, com foco em acessibilidade, reprodutibilidade e uso educacional, apoiando-se na pilha científica do Python — notadamente o NumPy [4] — executada no navegador por meio do PyScript [3].

Capítulo 3

Fundamentação Teórica

3.1 Introdução ao Capítulo

Este capítulo apresenta a formulação matemática e os métodos numéricos que fundamentam as análises realizadas no *MinervaMesh*. O objetivo central é fornecer uma base sólida, típica da Engenharia Mecânica, para a conversão dos modelos físicos contínuos em sistemas algébricos discretos resolvidos computacionalmente.

Inicialmente, revisam-se as equações diferenciais parciais (EDPs) governantes da condução de calor térmica e do escoamento de fluidos de forma conceitual. Em seguida, após uma breve fundamentação do Método das Diferenças Finitas (MDF), o capítulo se aprofunda extensamente no Método dos Elementos Finitos (MEF). O desenvolvimento abrange desde a formulação de Galerkin, passando pela discretização com elementos triangulares lineares, até a consolidação da formulação em Vorticidade-Corrente adotada para a dinâmica de fluidos.

3.2 Equações Governantes

A modelagem térmica e fluidodinâmica sustenta-se em princípios de conservação: massa, quantidade de movimento e energia. Estas leis físicas, contínuas no espaço e no tempo, são expressas por EDPs que determinam o comportamento termofluidodinâmico do meio contínuo abordado. Adota-se ao longo deste trabalho a convenção de denotar grandezas vetoriais em negrito (e.g., $\mathbf{v}$, $\mathbf{f}_b$) e suas compo-

nentes escalares em itálico (e.g., $\mathbf{v} = (u, v)$, com u e v associadas às direções x e y , respectivamente).

3.2.1 Condução e Convecção Térmica

A transferência de calor em meios sólidos contínuos é regida pela equação da difusão de calor (Lei de Fourier acoplada à conservação da energia térmica). Para um meio estacionário, a distribuição de temperatura $T(\mathbf{x}, t)$ é ditada por [7]:

$$\rho c_p \frac{\partial T}{\partial t} = \nabla \cdot (k \nabla T) + q''' \quad (3.1)$$

onde ρ é a massa específica, c_p é o calor específico a pressão constante, k é a condutividade térmica do material, e q''' representa a taxa volumétrica de geração de energia térmica. Para regimes permanentes e propriedades constantes, a Equação 3.1 se simplifica na clássica equação de Poisson (ou Laplace, se $q''' = 0$).

Quando existe escoamento (*convecção*), adiciona-se à equação o transporte advectivo promovido pelo campo de velocidades $\mathbf{v}$:

$$\rho c_p \left(\frac{\partial T}{\partial t} + \mathbf{v} \cdot \nabla T \right) = \nabla \cdot (k \nabla T) + q''' \quad (3.2)$$

Esta é a equação de convecção-difusão, paradigma central para o estudo da transferência de calor conjugada ou escoamentos não-isotérmicos.

3.2.2 Escoamento de Fluidos (Navier-Stokes)

O escoamento de fluidos incompressíveis newtonianos é regido pela restrição de continuidade (conservação de massa) e pelas Equações de Navier-Stokes (conservação do momento linear) [10]:

$$\nabla \cdot \mathbf{v} = 0 \quad (3.3)$$

$$\rho \left(\frac{\partial \mathbf{v}}{\partial t} + (\mathbf{v} \cdot \nabla) \mathbf{v} \right) = -\nabla p + \mu \nabla^2 \mathbf{v} + \mathbf{f}_b \quad (3.4)$$

onde p representa o campo de pressão estática, μ a viscosidade dinâmica do fluido e $\mathbf{f}_b$ eventuais forças de campo (como as forças de empuxo gravitacional). Esta

formulação em variáveis primitivas (velocidade-pressão) apresenta consideráveis desafios numéricos devido ao acoplamento embutido pela restrição de continuidade [1]. Consequentemente, abordagens numéricas como a Vorticidade-Corrente são frequentemente adotadas em 2D para contornar a incógnita da pressão diretamente.

3.3 Transição Discreta: O Método das Diferenças Finitas (MDF)

Antes do extensivo emprego de formulações variacionais associadas a malhas arbitrárias, a conversão de EDPs a um sistema matricial utilizou maciçamente as propriedades da expansão da série de Taylor. O Método das Diferenças Finitas (MDF) consiste na substituição direta dos operadores diferenciais exatos pelas respectivas aproximações algébricas discretizadas em nós de uma malha tipicamente ortogonal e estruturada [2].

Considere a representação da derivada parcial aproximada pelo método da diferença adiantada e atrasada para aproximações de derivadas espaciais e temporais:

$$\left. \frac{\partial u}{\partial x} \right|_i \approx \frac{u_{i+1} - u_{i-1}}{2\Delta x} \quad (\text{Diferença Central}) \quad (3.5)$$

$$\left. \frac{\partial^2 u}{\partial x^2} \right|_i \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{(\Delta x)^2} \quad (3.6)$$

Embora de inegável valor histórico e de extrema eficácia para discretizações explícitas no tempo e problemas de topologia regular simples, o MDF apresenta entraves práticos quando restrições de domínios geométricos altamente complexos exigem representações intrincadas de fronteiras, resultando em interpolações estéreo ou *stair-step* dos contornos físicos do domínio fluido ou de sólidos.

Para permitir tratamento sistemático de geometrias complexas e de condições de contorno de von Neumann ou mistas, o trabalho adota a seguir a formulação integral característica do Método dos Elementos Finitos (MEF).

3.4 O Método dos Elementos Finitos (MEF)

O Método dos Elementos Finitos constitui a abordagem numérica central adotada neste trabalho. Diferentemente do MDF, o MEF parte de uma formulação integral (variacional ou de resíduos ponderados) da EDP, permitindo o uso de malhas não estruturadas que se adaptam naturalmente a geometrias complexas [13]. Esta seção desenvolve a formulação desde a forma forte até a obtenção do sistema algébrico global.

3.4.1 Da Forma Forte à Forma Fraca

Considere, para fins de generalidade, uma EDP elíptica em um domínio $\Omega \subset \mathbb{R}^d$ com fronteira $\Gamma = \partial\Omega$. De maneira genérica, a *forma forte* do problema visa encontrar $u(\mathbf{x})$ tal que:

$$\mathcal{L}(u) = f \quad \text{em } \Omega \quad (3.7)$$

sujeita às condições de contorno:

$$u = \bar{u} \quad \text{em } \Gamma_D \quad (\text{Dirichlet}) \quad (3.8)$$

$$k \frac{\partial u}{\partial n} = \bar{q} \quad \text{em } \Gamma_N \quad (\text{Neumann}) \quad (3.9)$$

onde $\mathcal{L}$ é um operador diferencial (por exemplo, $-\nabla \cdot (k \nabla(\cdot))$ para condução), f é o termo fonte, $\bar{u}$ é o valor prescrito na fronteira de Dirichlet Γ_D , $\bar{q}$ é o fluxo prescrito na fronteira de Neumann Γ_N , e $\Gamma_D \cup \Gamma_N = \Gamma$.

A *forma fraca* (ou variacional) é obtida multiplicando-se a Equação 3.7 por uma *função-teste* (ou função de ponderação) $w \in \mathcal{V}_0$, pertencente a um espaço funcional adequado, e integrando-se sobre o domínio Ω :

$$\int_{\Omega} w \mathcal{L}(u) d\Omega = \int_{\Omega} w f d\Omega \quad (3.10)$$

A vantagem fundamental desta operação reside na possibilidade de aplicar o Teorema de Green (integração por partes generalizada), transferindo uma ordem de derivação do operador diferencial $\mathcal{L}$ para a função-teste w . Considere o caso concreto da equação de Poisson estacionária ($-\nabla \cdot (k \nabla u) = f$). Multiplicando por w e integrando:

$$-\int_{\Omega} w \nabla \cdot (k \nabla u) d\Omega = \int_{\Omega} w f d\Omega \quad (3.11)$$

Aplicando o Teorema de Green ao lado esquerdo:

$$\int_{\Omega} k \nabla w \cdot \nabla u d\Omega - \int_{\Gamma_N} w k \frac{\partial u}{\partial n} d\Gamma = \int_{\Omega} w f d\Omega \quad (3.12)$$

Substituindo a condição de Neumann (Eq. 3.9) no termo de contorno, obtém-se a *forma fraca*: encontrar $u \in \mathcal{V}$ tal que, para todo $w \in \mathcal{V}_0$:

$$\boxed{\int_{\Omega} k \nabla w \cdot \nabla u d\Omega = \int_{\Omega} w f d\Omega + \int_{\Gamma_N} w \bar{q} d\Gamma} \quad (3.13)$$

Esta expressão é de importância central: ela reduz os requisitos de regularidade sobre u (de C^2 para H^1), incorpora naturalmente as condições de Neumann no funcional, e constitui a base sobre a qual o MEF opera.

3.4.2 Condições de Contorno

A unicidade da solução de uma EDP elíptica ou parabólica depende da prescrição de informações apropriadas em $\Gamma = \partial\Omega$. No contexto da formulação variacional do MEF, as condições de contorno classificam-se em dois grupos: as *essenciais*, que restringem o espaço de soluções admissíveis $\mathcal{V}$, e as *naturais*, que emergem espontaneamente do termo de contorno gerado pela integração por partes (Eq. 3.12) [15]. Os três tipos canônicos — Dirichlet, Neumann e Robin — abrangem a quase totalidade dos problemas de engenharia tratados pelo MEF.

Dirichlet (essencial). Já apresentada na Eq. 3.8, a condição de Dirichlet prescreve diretamente o valor da incógnita em Γ_D — por exemplo, temperatura fixa em parede aquecida, ou condição de não-deslizamento $\mathbf{v} = \mathbf{0}$ em paredes sólidas. Por restringir o espaço admissível $\mathcal{V} = \{u \in H^1(\Omega) : u = \bar{u} \text{ em } \Gamma_D\}$, a função-teste w deve anular-se em Γ_D . Numericamente, sua imposição é feita após a montagem global, zerando-se as linhas e colunas de $\mathbf{K}$ correspondentes aos nós em Γ_D e ajustando o vetor de carga $\mathbf{F}$ para incorporar o valor prescrito [13].

Neumann (natural). A condição de Neumann (Eq. 3.9) prescreve o fluxo $k \partial u / \partial n = \bar{q}$ em Γ_N . Sua designação como “natural” reflete o fato de que ela aparece de forma direta no termo de contorno da forma fraca após a integração por partes: basta substituir $\bar{q}$ no integrando $\int_{\Gamma_N} w \bar{q} d\Gamma$ (Eq. 3.13) para incorporá-la no vetor de carga global. O caso particular mais comum é a parede adiabática ($\bar{q} = 0$), em que a integral simplesmente se anula, dispensando qualquer manipulação adicional.

Robin (mista). As condições de Robin (também chamadas de mistas ou de terceiro tipo) combinam o valor da incógnita e seu fluxo na fronteira em uma relação linear:

$$\alpha u + \beta \frac{\partial u}{\partial n} = \gamma \quad \text{em } \Gamma_R \quad (3.14)$$

com α, β, γ coeficientes que parametrizam o tipo de interação física na fronteira. O caso canônico em transferência de calor é a convecção newtoniana, em que o fluxo por condução é igualado ao fluxo convectivo trocado com o meio externo a temperatura T_∞ via coeficiente convectivo h [7]:

$$-k \frac{\partial T}{\partial n} = h (T - T_\infty) \quad \text{em } \Gamma_R \quad (3.15)$$

Substituindo a relação acima no termo de contorno $\int_{\Gamma_R} w k \partial u / \partial n d\Gamma$ da forma fraca, surgem duas contribuições simultâneas: uma na matriz de rigidez — $\int_{\Gamma_R} h N_i N_j d\Gamma$ — e outra no vetor de carga — $\int_{\Gamma_R} h T_\infty N_i d\Gamma$. Robin é, portanto, a única dentre as três que altera simultaneamente $\mathbf{K}$ e $\mathbf{F}$ [15].

A Tabela 3.1 sintetiza a classificação variacional e o impacto numérico de cada tipo.

Tabela 3.1: Classificação variacional e tratamento numérico das condições de contorno no MEF.

Tipo	Forma	Classificação	Impacto em $\mathbf{K}, \mathbf{F}$
Dirichlet	$u = \bar{u}$	Essencial	Modifica $\mathbf{K}$ e $\mathbf{F}$
Neumann	$k \partial u / \partial n = \bar{q}$	Natural	Modifica apenas $\mathbf{F}$
Robin	$\alpha u + \beta \partial u / \partial n = \gamma$	Natural mista	Modifica $\mathbf{K}$ e $\mathbf{F}$

Os módulos da plataforma documentados no Capítulo 5 fazem uso predominante de condições de Dirichlet (temperaturas e velocidades prescritas) e, de

forma implícita, de Neumann homogêneas em paredes adiabáticas e fronteiras livres ($\bar{q} = 0$). Condições de Robin não são empregadas nos módulos atuais; sua incorporação — particularmente no módulo de transferência de calor 2D, para representar troca convectiva com o meio externo — configura-se como extensão natural prevista nas considerações finais do Capítulo 6.

3.4.3 O Método de Galerkin

O método de Galerkin consiste em restringir o problema variacional contínuo (Eq. 3.13) a um subespaço de dimensão finita $\mathcal{V}^h \subset \mathcal{V}$, parametrizado por uma malha de elementos finitos. A solução aproximada u^h é expressa como uma combinação linear de N funções de base $\{N_j\}_{j=1}^N$:

$$u^h(\mathbf{x}) = \sum_{j=1}^N U_j N_j(\mathbf{x}) \quad (3.16)$$

onde U_j são os coeficientes nodais (incógnitas do sistema discreto) e $N_j(\mathbf{x})$ são as funções de forma associadas a cada nó j da malha.

A escolha fundamental do método de Galerkin é adotar as *mesmas* funções de base como funções-teste: $w^h = N_i$, $i = 1, \dots, N$. Substituindo na forma fraca (Eq. 3.13):

$$\sum_{j=1}^N U_j \int_{\Omega} k \nabla N_i \cdot \nabla N_j d\Omega = \int_{\Omega} N_i f d\Omega + \int_{\Gamma_N} N_i \bar{q} d\Gamma \quad \forall i = 1, \dots, N \quad (3.17)$$

Esta expressão define um sistema linear $\mathbf{KU} = \mathbf{F}$, onde:

$$K_{ij} = \int_{\Omega} k \nabla N_i \cdot \nabla N_j d\Omega \quad (\text{Matriz de Rigidez}) \quad (3.18)$$

$$F_i = \int_{\Omega} N_i f d\Omega + \int_{\Gamma_N} N_i \bar{q} d\Gamma \quad (\text{Vetor de Carga}) \quad (3.19)$$

A resolução deste sistema algébrico fornece os coeficientes nodais $\mathbf{U}$, a partir dos quais o campo aproximado u^h é reconstituído via Equação 3.16. A qualidade da aproximação depende diretamente da riqueza do espaço $\mathcal{V}^h$, controlada pelo refinamento da malha e pela ordem polinomial das funções de base [6].

De maneira compacta, o procedimento de Galerkin converte o problema contínuo em:

$$\boxed{\mathbf{KU} = \mathbf{F}} \quad (3.20)$$

A construção efetiva das matrizes $\mathbf{K}$ e do vetor $\mathbf{F}$ depende da escolha dos elementos e das funções de forma, desenvolvida na seção seguinte.

3.4.4 Elementos Triangulares Lineares

A discretização do domínio Ω é realizada por uma malha de N_e elementos triangulares, cujos vértices coincidem com os N nós globais da malha. Cada elemento e possui três nós locais $(1, 2, 3)$, com coordenadas (x_i, y_i) , $i = 1, 2, 3$.

As funções de forma lineares N_i^e para o elemento triangular linear são definidas de modo que $N_i^e(\mathbf{x}_j) = \delta_{ij}$ (propriedade de partição da unidade nodal). Em coordenadas cartesianas, estas funções são expressas por [15]:

$$N_i^e(x, y) = \frac{1}{2A^e}(a_i + b_i x + c_i y), \quad i = 1, 2, 3 \quad (3.21)$$

onde A^e é a área do elemento triangular e os coeficientes são dados por permutação cíclica dos índices dos vértices:

$$a_i = x_j y_k - x_k y_j, \quad b_i = y_j - y_k, \quad c_i = x_k - x_j \quad (3.22)$$

com (i, j, k) em ordem cíclica $(1, 2, 3)$, $(2, 3, 1)$, $(3, 1, 2)$. A área do elemento é calculada por:

$$A^e = \frac{1}{2} |x_1(y_2 - y_3) + x_2(y_3 - y_1) + x_3(y_1 - y_2)| \quad (3.23)$$

Os gradientes das funções de forma são constantes dentro de cada elemento linear:

$$\nabla N_i^e = \frac{1}{2A^e} \begin{pmatrix} b_i \\ c_i \end{pmatrix} \quad (3.24)$$

Esta propriedade simplifica consideravelmente as integrais elementares, uma vez que os integrandos envolvendo $\nabla N_i \cdot \nabla N_j$ tornam-se constantes no interior do elemento.

3.4.5 Matrizes Elementares

A partir das funções de forma lineares, as integrais da formulação de Galerkin (Eqs. 3.18 e 3.19) são calculadas elemento a elemento e, em seguida, assembradas no sistema global.

3.4.5.1 Matriz de Rigidez Elementar

Para o problema de difusão com condutividade k constante por elemento, a matriz de rigidez elementar $\mathbf{K}^e$ de dimensão 3×3 é:

$$K_{ij}^e = \int_{\Omega^e} k \nabla N_i^e \cdot \nabla N_j^e d\Omega = \frac{k}{4A^e} (b_i b_j + c_i c_j) \quad (3.25)$$

3.4.5.2 Matriz de Massa Elementar

Em problemas transientes, a discretização temporal introduz um termo de acumulação que requer a *matriz de massa*. Para elementos lineares, a matriz de massa consistente é [13]:

$$M_{ij}^e = \int_{\Omega^e} \rho c_p N_i^e N_j^e d\Omega = \frac{\rho c_p A^e}{12} \begin{cases} 2 & \text{se } i = j \\ 1 & \text{se } i \neq j \end{cases} \quad (3.26)$$

Alternativamente, a *matriz de massa concentrada* (*lumped*) distribui a massa uniformemente nos nós, resultando em uma matriz diagonal:

$$M_{ij}^{e,\text{lump}} = \frac{\rho c_p A^e}{3} \delta_{ij} \quad (3.27)$$

3.4.5.3 Vetor de Carga Elementar

Para um termo fonte f uniforme no elemento, o vetor de carga elementar resulta em:

$$F_i^e = \int_{\Omega^e} N_i^e f d\Omega = \frac{f A^e}{3} \quad (3.28)$$

3.4.6 Montagem Global

A construção do sistema global $\mathbf{KU} = \mathbf{F}$ é realizada pelo procedimento de *assembly*, que consiste em acumular as contribuições de cada elemento nas posições globais correspondentes. Para cada elemento e , a conectividade local-global é definida por um mapa de índices $\text{IEN}(a, e) \rightarrow I$, que associa o nó local a do elemento e ao nó global I .

A montagem é expressa como:

$$K_{IJ} = \sum_{e=1}^{N_e} K_{\text{IEN}(a,e), \text{IEN}(b,e)}^e, \quad F_I = \sum_{e=1}^{N_e} F_{\text{IEN}(a,e)}^e \quad (3.29)$$

Após a montagem, as condições de contorno de Dirichlet são impostas por modificação das linhas correspondentes do sistema. A resolução do sistema linear resultante fornece os valores nodais $\mathbf{U}$.

3.5 Formulação Vorticidade-Corrente

A resolução direta das equações de Navier-Stokes em variáveis primitivas (velocidade-pressão) apresenta dificuldades numéricas associadas ao acoplamento pressão-velocidade e à satisfação da restrição de continuidade (Eq. 3.3). Em problemas bidimensionais, a formulação em *Vorticidade-Corrente* (ω - ψ) oferece uma alternativa eficaz, eliminando o campo de pressão das incógnitas e reduzindo o sistema a duas equações escalares [1].

3.5.1 Definições

Para um escoamento bidimensional, define-se a *função corrente* $\psi(x, y, t)$ tal que o campo de velocidades satisfaça identicamente a equação da continuidade:

$$u = \frac{\partial \psi}{\partial y}, \quad v = -\frac{\partial \psi}{\partial x} \quad (3.30)$$

A *vorticidade*, componente escalar no plano 2D, é definida como o rotacional do campo de velocidades:

$$\omega = \frac{\partial v}{\partial x} - \frac{\partial u}{\partial y} \quad (3.31)$$

Substituindo as definições da função corrente na expressão da vorticidade, obtém-se a *equação de Poisson para a função corrente*:

$$\nabla^2 \psi = -\omega \quad (3.32)$$

3.5.2 Equação de Transporte da Vorticidade

Aplicando o rotacional às equações de Navier-Stokes (Eq. 3.4), o gradiente de pressão é eliminado e obtém-se a equação de transporte da vorticidade:

$$\frac{\partial \omega}{\partial t} + u \frac{\partial \omega}{\partial x} + v \frac{\partial \omega}{\partial y} = \nu \nabla^2 \omega \quad (3.33)$$

onde $\nu = \mu/\rho$ é a viscosidade cinemática. Esta equação possui a estrutura clássica de convecção-difusão, onde o campo de velocidades (u, v) transporta a vorticidade ω enquanto a difusão viscosa a dissipa.

3.5.3 Sistema Acoplado e Solução Iterativa

O sistema ω - ψ é resolvido iterativamente. A cada passo de tempo (ou iteração estacionária), o procedimento consiste em:

1. Resolver a equação de Poisson (Eq. 3.32) para ψ , dado ω .
2. Calcular o campo de velocidades (u, v) a partir de ψ (Eq. 3.30).
3. Resolver a equação de transporte (Eq. 3.33) para ω , dado (u, v) .
4. Verificar a convergência e repetir, se necessário.

3.5.4 Matriz de Convecção e Estabilização SUPG

A discretização via MEF do operador convectivo $\mathbf{v} \cdot \nabla \phi$ conduz à *matriz de convecção* elementar, de dimensão 3×3 para elementos lineares:

$$C_{ij}^e = \int_{\Omega^e} N_i (\mathbf{v} \cdot \nabla N_j) d\Omega = \frac{A^e}{3} \left(\frac{u_e b_j + v_e c_j}{2A^e} \right) \quad (3.34)$$

onde u_e e v_e são as componentes de velocidade avaliadas no centroide do elemento.

Quando o campo de velocidades é intenso em relação à difusão (número de Péclet elevado), a formulação de Galerkin padrão produz oscilações espúrias. Para contornar essa limitação, emprega-se a técnica *Streamline Upwind Petrov-Galerkin* (SUPG), que adiciona uma difusão artificial orientada na direção do escoamento. A formulação SUPG foi proposta por Brooks e Hughes [5], generalizando as funções de peso assimétricas (*upwind*) introduzidas anteriormente por Christie et al. [21].

Na formulação SUPG, a função-teste é modificada para $w_i = N_i + \tau_{\text{SUPG}} \mathbf{v} \cdot \nabla N_i$, com o parâmetro de estabilização:

$$\tau_{\text{SUPG}} = \frac{h_e}{2\|\mathbf{v}\|} \left(\coth \text{Pe}_e - \frac{1}{\text{Pe}_e} \right) \quad (3.35)$$

onde $\text{Pe}_e = \|\mathbf{v}\| h_e / (2\alpha)$ é o número de Péclet elementar.

No *MinervaMesh*, a estabilização SUPG é empregada no caso unidimensional de convecção-difusão, onde o efeito do número de Péclet é mais crítico. No solver bidimensional de Navier-Stokes, a convecção da vorticidade utiliza a formulação de Galerkin padrão (Eq. 3.34), sendo a estabilidade controlada pela escolha adequada do passo de tempo e do número de Reynolds.

3.6 Integração Temporal

Para problemas transientes, a discretização temporal converte a EDP semi-discreta (após a discretização espacial por MEF) em um sistema algébrico a cada intervalo de tempo Δt . No caso geral, o sistema semidiscreto assume a forma:

$$\mathbf{M} \frac{d\mathbf{U}}{dt} + \mathbf{K}\mathbf{U} = \mathbf{F}(t) \quad (3.36)$$

3.6.1 Método θ Generalizado

A família de métodos θ parametriza a ponderação temporal entre os instantes t^n e t^{n+1} :

$$\mathbf{M} \frac{\mathbf{U}^{n+1} - \mathbf{U}^n}{\Delta t} + \mathbf{K} [\theta \mathbf{U}^{n+1} + (1 - \theta) \mathbf{U}^n] = \theta \mathbf{F}^{n+1} + (1 - \theta) \mathbf{F}^n \quad (3.37)$$

Os casos particulares mais relevantes são:

- $\theta = 0$: Euler explícito (condicionalmente estável);
- $\theta = 1/2$: Crank-Nicolson (incondicionalmente estável, segunda ordem);
- $\theta = 1$: Euler implícito (incondicionalmente estável, primeira ordem).

3.6.2 Esquema Semi-Implícito Adotado

No solver de Navier-Stokes do *MinervaMesh*, a equação de transporte da vorticidade (Eq. 3.33) é discretizada com um esquema *semi-implícito*: o termo difusivo é tratado implicitamente ($\theta = 1$), enquanto o termo convectivo é avaliado explicitamente no instante t^n . Isso resulta no sistema:

$$\left(\mathbf{M} + \frac{\Delta t}{\text{Re}} \mathbf{K} \right) \boldsymbol{\omega}^{n+1} = \mathbf{M} \boldsymbol{\omega}^n - \Delta t \mathbf{C}^n \boldsymbol{\omega}^n \quad (3.38)$$

onde $\mathbf{C}^n$ é a matriz de convecção assembrada a partir do campo de velocidades no instante atual e $\text{Re} = \rho UL/\mu$ é o número de Reynolds. O tratamento implícito da difusão garante estabilidade incondicional para esse operador, enquanto a convecção explícita impor restrições sobre Δt associadas ao número de Courant.

Para problemas puramente difusivos (condução transiente), utiliza-se o método θ generalizado (Eq. 3.37), permitindo ao usuário selecionar entre Euler e Crank-Nicolson conforme o compromisso desejado entre precisão e estabilidade.

3.6.3 Condição de Contorno de Vorticidade na Parede

Na formulação ω - ψ , a vorticidade nas paredes sólidas não é conhecida *a priori* e deve ser calculada indiretamente a partir do campo de função corrente. Aplicando uma expansão de Taylor de ψ na direção normal à parede, obtém-se uma aproximação de primeira ordem para ω_w , conhecida como condição de Thom [45]:

$$\omega_w = -\frac{2(\psi_{\text{nb}} - \psi_w)}{h^2} - \frac{2u_{\text{tan}}}{h} \quad (3.39)$$

onde ψ_w e ψ_{nb} são os valores da função corrente na parede e no nó vizinho mais próximo na direção normal (apontando para o interior do domínio), respectivamente; h é a distância entre esses nós; e u_{tan} é a componente tangencial da velocidade da parede, consistente com a convenção $u = \partial\psi/\partial y$ adotada nesta formulação ($u_{\text{tan}} =$

0 para paredes estacionárias e $u_{\text{tan}} = U_{\text{lid}}$ para a tampa móvel da cavidade com velocidade U_{lid} na direção $+x$).

3.7 Síntese do Capítulo

Este capítulo estabeleceu o arcabouço matemático completo empregado pelo *MinervaMesh*. A formulação parte das EDPs governantes para fenômenos térmicos e fluidodinâmicos, transita pela introdução conceitual do MDF e se aprofunda no MEF via Galerkin. A discretização com elementos triangulares lineares permite a construção explícita das matrizes elementares de rigidez, massa e convecção, enquanto a formulação Vorticidade-Corrente viabiliza a solução de escoamentos 2D sem a necessidade de resolver diretamente o campo de pressão. A integração temporal semi-implícita e a aproximação de primeira ordem para a vorticidade na parede completam o ferramental necessário para a simulação dos casos apresentados no Capítulo 4.

Capítulo 4

Resultados e Validação

4.1 Introdução

Este capítulo apresenta a validação numérica dos oito módulos de simulação por Elementos Finitos do *MinervaMesh*, cuja arquitetura e implementação *client-side* são detalhadas no Capítulo 5. Cada caso simulado é comparado com soluções analíticas conhecidas ou *benchmarks* consagrados da literatura, de modo a verificar a corretude da discretização e quantificar os erros numéricos associados.

A estratégia de validação segue um critério de complexidade crescente. Os primeiros casos — condução de calor em regime permanente e transiente — envolvem problemas unidimensionais com solução analítica fechada, permitindo o cálculo direto de métricas de erro como o erro máximo absoluto e o erro relativo na norma L_2 . Na sequência, os problemas de convecção-difusão testam a estabilização SUPG, comparando a solução estabilizada com a solução Galerkin padrão em regime de alto número de Péclet. Os módulos estruturais — flexão e vibração de barras — são validados contra soluções clássicas da Resistência dos Materiais e da Dinâmica Estrutural. Finalmente, os casos bidimensionais de transferência de calor, escoamento potencial e escoamento viscoso (Navier-Stokes) completam a validação com problemas de geometria e condições de contorno mais complexas.

Verificação e validação. Convém distinguir dois tipos complementares de teste no contexto de simulação numérica: *verificação* consiste em garantir que o código resolve corretamente as equações que se propõe a resolver, tipicamente confrontando a

saída numérica contra soluções analíticas ou *benchmarks* de alta resolução; *validação* consiste em avaliar se as equações resolvidas descrevem adequadamente o fenômeno físico real, o que requer confronto com dados experimentais e quantificação formal de incerteza. Os oito casos apresentados neste capítulo inscrevem-se integralmente na primeira categoria: seis contra solução analítica fechada (Seções 4.2 a 4.7), escoamento potencial contra referência teórica clássica (Seção 4.8) e Navier-Stokes 2D contra o *benchmark* de Ghia et al. [24] (Seção 4.9). A validação experimental, com medidas laboratoriais e análise formal de incertezas, não constitui escopo do presente trabalho e fica indicada como extensão natural nos trabalhos futuros (Capítulo 6).

Para cada caso, a metodologia de apresentação é a seguinte:

1. **Definição do problema:** equação governante, domínio, condições de contorno e parâmetros adotados.
2. **Referência de validação:** solução analítica ou benchmark utilizado para comparação.
3. **Resultados numéricos:** campos obtidos pelo simulador, apresentados graficamente.
4. **Análise de erro:** métricas quantitativas de comparação entre resultado numérico e referência.

Todos os resultados foram gerados diretamente no navegador, utilizando a plataforma *MinervaMesh* em execução local via PyScript/WebAssembly, conforme descrito no Capítulo 5. Os parâmetros de entrada de cada simulação são reproduzidos integralmente nas tabelas a seguir, garantindo a reprodutibilidade dos experimentos.

4.2 Condução Permanente 1D

O primeiro caso de validação consiste no problema de condução de calor em regime permanente, em uma barra unidimensional de comprimento L com geração interna de calor Q e condições de contorno de Dirichlet em ambas as extremidades. Este problema constitui o caso mais simples de validação do MEF, pois admite

solução analítica exata e envolve elementos lineares unidimensionais, correspondendo diretamente à formulação desenvolvida na Seção 3.4.1.

4.2.1 Formulação do Problema

O problema é governado pela equação de Poisson unidimensional, caso particular da equação geral de condução (Equação 3.1):

$$-k \frac{d^2 T}{dx^2} = Q, \quad x \in [0, L] \quad (4.1)$$

com as condições de contorno:

$$T(0) = T_0, \quad T(L) = T_L \quad (4.2)$$

A Tabela 4.1 apresenta os parâmetros utilizados na simulação.

Tabela 4.1: Parâmetros da simulação de condução permanente 1D.

Parâmetro	Símbolo	Valor
Comprimento da barra	L	1,0 m
Número de elementos	n_e	10
Temperatura em $x = 0$	T_0	0,0 °C
Temperatura em $x = L$	T_L	1,0 °C
Condutividade térmica	k	1,0 W/(m·°C)
Geração interna	Q	4,0 W/m ³

4.2.2 Solução Analítica

A integração direta da Equação (4.1) fornece a solução exata:

$$T(x) = -\frac{Q}{2k}x^2 + C_1x + C_2 \quad (4.3)$$

onde as constantes de integração são determinadas pelas condições de contorno:

$$C_2 = T_0, \quad C_1 = \frac{T_L - T_0 + \frac{QL^2}{2k}}{L} \quad (4.4)$$

Substituindo os valores da Tabela 4.1, obtém-se $C_1 = 3,0 \text{ }^\circ\text{C/m}$ e $C_2 = 0,0 \text{ }^\circ\text{C}$. O perfil de temperatura resultante é uma parábola com valor máximo $T_{\max} \approx 1,125 \text{ }^\circ\text{C}$ em $x^* = 0,75 \text{ m}$, no interior do domínio — superior ao valor prescrito em qualquer das extremidades ($T_0 = 0 \text{ }^\circ\text{C}$, $T_L = 1,0 \text{ }^\circ\text{C}$) e decorrente da competição entre a geração interna de calor e a difusão para as fronteiras.

4.2.3 Solução MEF e Comparação

A discretização por elementos finitos utiliza $n_e = 10$ elementos lineares com espaçamento uniforme $h = L/n_e = 0,1 \text{ m}$, resultando em 11 nós. A montagem do sistema global e a imposição das condições de contorno de Dirichlet seguem o procedimento descrito nas Seções 3.4.3 e 3.4.6. Para este problema de Poisson unidimensional com elementos lineares, a solução MEF coincide exatamente com a solução analítica nos nós da malha (propriedade bem conhecida da interpolação linear por partes para polinômios de grau ≤ 2).

A Figura 4.1 apresenta os resultados obtidos. A curva MEF (pontos azuis) está perfeitamente sobreposta à solução analítica (linha tracejada vermelha), confirmando a corretude da implementação.

4.2.4 Análise de Erro

Para este caso particular — equação de Poisson 1D com fonte constante e elementos lineares — a solução analítica é um polinômio de grau 2. Como a interpolação linear reproduz exatamente qualquer polinômio de grau ≤ 2 nos nós da malha, espera-se que o erro nodal seja da ordem do erro de arredondamento de máquina ($\sim 10^{-15}$). De fato, ao calcular o erro máximo absoluto e o erro relativo na norma L_2 :

$$e_{\max} = \max_i |T_i^{\text{MEF}} - T_i^{\text{ana}}|, \quad e_{L_2} = \frac{(\sum_i (T_i^{\text{MEF}} - T_i^{\text{ana}})^2)^{1/2}}{(\sum_i (T_i^{\text{ana}})^2)^{1/2}} \quad (4.5)$$

obtém-se $e_{\max} \approx 10^{-15} \text{ }^\circ\text{C}$ e $e_{L_2} \approx 10^{-15}$, confirmando erro essencialmente nulo nos nós da malha. Este resultado era esperado teoricamente e valida a montagem das matrizes elementares de rigidez e do vetor de carga, a montagem global

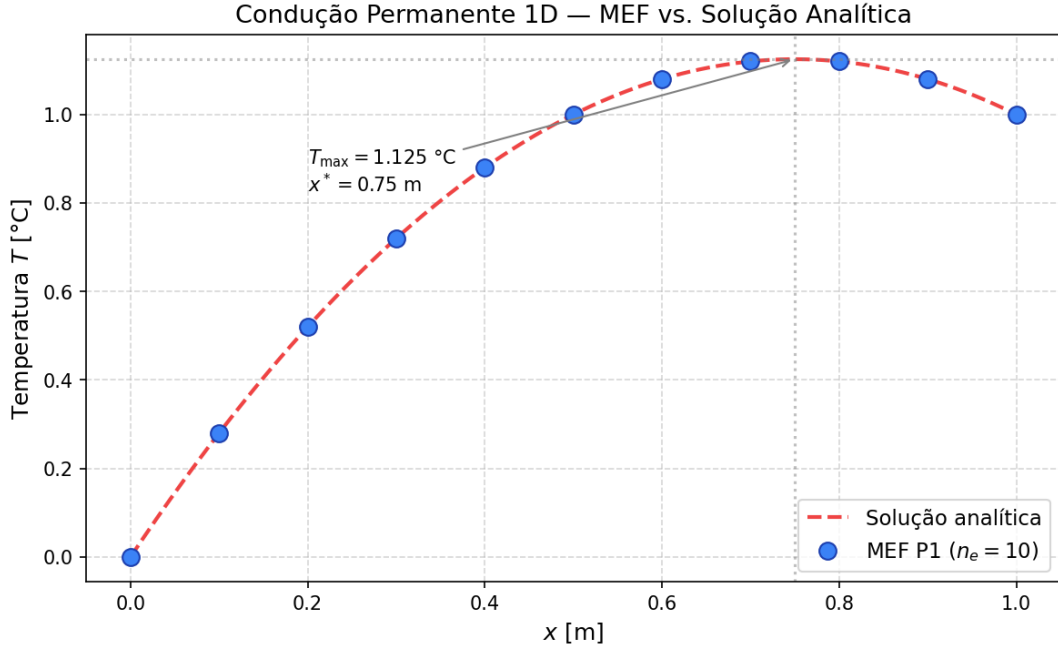


Figura 4.1: Condução permanente 1D: comparação entre a solução MEF (pontos azuis, $n_e = 10$ elementos lineares) e a solução analítica (linha tracejada vermelha). A sobreposição exata confirma a propriedade de reprodução polinomial dos elementos lineares para fontes uniformes.

e a imposição das condições de contorno de Dirichlet implementadas no módulo `conducao-permanente-barra1d-mef`.

Nota sobre a precisão nodal exata. A concordância perfeita nos nós não implica erro nulo entre os nós. O campo MEF é linear por partes, enquanto a solução exata é parabólica. O erro de interpolação entre nós é da ordem $O(h^2)$. Essa observação será relevante nos casos subsequentes, que envolvem equações com soluções de maior grau ou não-polinomiais.

4.3 Condução Transiente 1D com $k(x)$ Variável

O segundo caso de validação estende o problema de condução unidimensional para o regime transiente, introduzindo duas dificuldades adicionais: dependência temporal, tratada pelo método θ (Crank-Nicolson, $\theta = 1/2$), e heterogeneidade material, representada por uma condutividade descontínua em $x = L/2$. Este cenário verifica simultaneamente a implementação da matriz de massa consistente,

o esquema de integração temporal e a montagem correta das matrizes elementares quando o coeficiente material varia dentro do domínio.

4.3.1 Formulação do Problema

O balanço de energia em regime transiente sem geração interna é descrito pela equação parabólica:

$$\rho c_p \frac{\partial T}{\partial t} = \frac{\partial}{\partial x} \left(k(x) \frac{\partial T}{\partial x} \right), \quad x \in [0, L], \quad t > 0 \quad (4.6)$$

com condições de contorno de Dirichlet fixas no tempo e condição inicial homogênea:

$$T(0, t) = 0, \quad T(L, t) = 1, \quad T(x, 0) = 0 \quad (4.7)$$

e condutividade descontínua em $x = L/2$:

$$k(x) = \begin{cases} k_1, & 0 \leq x < L/2 \\ k_2, & L/2 < x \leq L \end{cases} \quad (4.8)$$

A Tabela 4.2 resume os parâmetros adotados, todos correspondentes aos valores *default* da interface HTML do módulo.

Tabela 4.2: Parâmetros da simulação de condução transiente 1D com $k(x)$ variável.

Parâmetro	Símbolo	Valor
Comprimento da barra	L	1,0 m
Número de elementos	n_e	40
Condutividade na primeira metade	k_1	5,0 W/(m·°C)
Condutividade na segunda metade	k_2	1,0 W/(m·°C)
Capacidade térmica volumétrica	ρc_p	1,0 J/(m ³ ·°C)
Passo de tempo	Δt	10 ⁻³ s
Parâmetro do método θ	θ	0,5
Tempo final de simulação	$t_{\max}$	0,2 s

4.3.2 Solução Analítica de Regime Permanente

No limite $t \rightarrow \infty$, o termo transiente se anula e a solução satisfaz $\frac{d}{dx}(k(x) dT/dx) = 0$, o que implica fluxo constante $k(x) dT/dx = C$ ao longo de todo o domínio. Em cada sub-região de condutividade uniforme, T é linear; a continuidade da temperatura em $x = L/2$ e a continuidade do fluxo fornecem o sistema

$$a_1 + a_2 = \frac{T_L - T_0}{L/2}, \quad k_1 a_1 = k_2 a_2 \quad (4.9)$$

onde a_1 e a_2 são as inclinações em cada metade. Resolvendo com $T_0 = 0$, $T_L = 1$, $k_1 = 5$, $k_2 = 1$:

$$a_1 = \frac{1}{3} \text{ °C/m}, \quad a_2 = \frac{5}{3} \text{ °C/m}, \quad T(L/2) = \frac{1}{6} \approx 0,1667 \text{ °C} \quad (4.10)$$

O perfil permanente é portanto uma linha poligonal com *kink* em $x = L/2$: inclinação suave no lado de maior condutividade e íngreme no lado de menor condutividade, refletindo a continuidade do fluxo térmico. A escala de tempo característica de difusão é $\tau_{\text{dif}} = L^2 \rho c_p / \bar{k} \sim 1,0$ s, com $\bar{k}$ condutividade efetiva.

4.3.3 Solução MEF e Comparação

A discretização emprega $n_e = 40$ elementos lineares com $h = 0,025$ m, o que resolve bem a descontinuidade de condutividade alinhando um nó exatamente com $x = L/2$. A cada passo de tempo, o sistema linear

$$\left(\frac{M}{\Delta t} + \theta K \right) \mathbf{T}^{n+1} = \left(\frac{M}{\Delta t} - (1 - \theta)K \right) \mathbf{T}^n \quad (4.11)$$

é resolvido pelo solver direto, onde M e K são as matrizes globais consistentes de massa e de rigidez montadas com $k(x)$ avaliado no baricentro de cada elemento. A Figura 4.2 apresenta o campo $T(x)$ no instante final $t = 0,2$ s (correspondente a $\sim 20\%$ da escala difusiva) sobreposto ao perfil analítico de regime permanente, evidenciando a convergência rápida do método ao regime linear por partes.

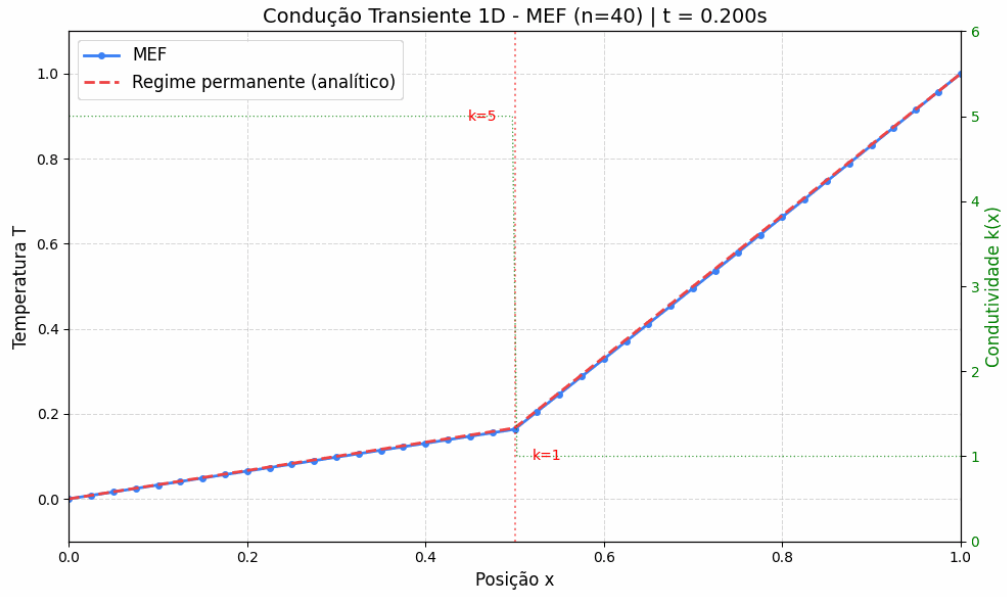


Figura 4.2: Condução transiente 1D com condutividade descontínua ($k_1 = 5$, $k_2 = 1$): solução MEF no instante final $t = 0,2$ s (pontos azuis) comparada ao perfil analítico de regime permanente (linha tracejada vermelha). A curva verde tracejada no eixo secundário indica a variação de $k(x)$ com a descontinuidade em $x = L/2$. A malha de 40 elementos lineares com $\theta = 1/2$ reproduz fielmente o *kink* de condutividade, com $T(L/2) \approx 1/6$.

4.3.4 Análise de Erro

Em $t_{\max} = 0,2$ s (20% da escala de tempo difusiva $\tau_{\text{dif}} \sim 1,0$ s), a solução MEF já se encontra em regime quase-permanente: a figura mostra os pontos nodais visualmente coincidentes com a linha poligonal analítica, incluindo a captura correta do *kink* de condutividade em $x = L/2$. A convergência pontual segue o decaimento exponencial esperado para equações parabólicas, $|T(x, t) - T_{\infty}(x)| \sim e^{-t/\tau}$, de modo que em $t \gtrsim 2\tau_{\text{dif}}$ o erro nodal atinge a precisão de máquina, dado que a solução de regime permanente (linear por partes) pertence exatamente ao espaço de aproximação dos elementos lineares com nós alinhados à descontinuidade.

Nota sobre a propagação térmica. Para tempos menores, inferiores a τ_{dif} , o campo exibe a perturbação térmica se propagando da parede quente ($x = L$, onde $T_L = 1$) para o interior da barra, com frente de difusão progredindo à razão $\sqrt{\kappa t}$ ($\kappa = k/\rho c_p$). A descontinuidade de condutividade em $x = L/2$ introduz um retardo do avanço da frente quando esta atravessa para a região de menor condutividade ($k_2 = 1$), comportamento observável iterativamente ao se reduzir $t_{\max}$ na interface.

4.4 Convecção-Difusão 1D com Estabilização SUPG

O terceiro caso aborda a equação de convecção-difusão estacionária em uma dimensão, cenário em que a formulação Galerkin clássica perde estabilidade quando a convecção predomina sobre a difusão. Este problema é particularmente útil para validação porque admite solução analítica fechada em todo o espectro de números de Péclet e porque expõe uma propriedade teórica muito forte da estabilização SUPG: com o parâmetro canônico de Christie, Griffiths e Mitchell (1976), o erro nodal é da ordem da precisão de máquina independentemente de Pe_e .

4.4.1 Formulação do Problema

A forma forte do problema estacionário unidimensional é

$$-k \frac{d^2 T}{dx^2} + \rho c_p u \frac{dT}{dx} = Q, \quad x \in [0, L] \quad (4.12)$$

com condições de contorno de Dirichlet em ambas as extremidades, $T(0) = T_0$ e $T(L) = T_L$. O parâmetro adimensional relevante para a escolha da discretização é o número de Péclet elementar

$$\text{Pe}_e = \frac{\rho c_p u h_e}{2k} \quad (4.13)$$

onde h_e é o tamanho do elemento. A Tabela 4.3 lista os parâmetros adotados, correspondentes aos *defaults* da interface do módulo.

Tabela 4.3: Parâmetros da simulação de convecção-difusão 1D.

Parâmetro	Símbolo	Valor
Comprimento	L	1,0 m
Número de elementos	n_e	50
Temperatura em $x = 0$	T_0	0,0 °C
Temperatura em $x = L$	T_L	1,0 °C
Condutividade térmica	k	1,0 W/(m·°C)
Capacidade térmica volumétrica	ρc_p	10^3 J/(m ³ ·°C)
Geração interna	Q	1,0 W/m ³
Velocidade de convecção	u	0,05 m/s
Péclet elementar	Pe_e	0,5
Estabilização SUPG	—	ativa

4.4.2 Solução Analítica

Para $Q = 0$, a equação admite a solução exponencial clássica; com fonte constante, a solução geral é soma da homogênea com uma particular linear. O resultado pode ser escrito na forma

$$T(x) = T_0 + \frac{Qx}{\rho c_p u} + \left(T_L - T_0 - \frac{QL}{\rho c_p u} \right) \frac{e^{\alpha x} - 1}{e^{\alpha L} - 1}, \quad \alpha = \frac{\rho c_p u}{k} \quad (4.14)$$

Para $\alpha L \gg 1$, a exponencial domina no intervalo próximo a $x = L$, concentrando toda a variação do campo em uma camada-limite de espessura $\sim 1/\alpha$. Com os *defaults* da interface, $\alpha L = 50$ e a camada-limite é estreita porém bem

resolvida pela malha uniforme de 50 elementos. Em regimes de $Pe_e > 1$ a camada-limite ocupa apenas uma fração do elemento, situação em que a formulação Galerkin clássica apresenta oscilações espúrias e a estabilização SUPG se torna indispensável.

4.4.3 Solução MEF com SUPG

A formulação Petrov-Galerkin introduz funções-peso modificadas $\tilde{w}_i = w_i + \tau u dw_i/dx$, com o parâmetro de estabilização

$$\tau_{\text{SUPG}} = \frac{h_e}{2|u|} \left(\coth Pe_e - \frac{1}{Pe_e} \right) \quad (4.15)$$

derivado por Christie, Griffiths e Mitchell (1976) a partir da exigência de anular o erro nodal no problema homogêneo. As Figuras 4.3 e 4.4 comparam, em $Pe_e = 0,5$, a solução MEF com e sem a estabilização SUPG sobreposta à solução analítica, ambas rodadas diretamente a partir da interface do módulo.

4.4.4 Análise de Erro

Os erros nodais máximos observados nas duas configurações da mesma malha são

$$e_{\max}^{\text{SUPG}} \approx 4,44 \times 10^{-16} \text{ }^\circ\text{C}, \quad e_{\max}^{\text{Galerkin}} \approx 3,39 \times 10^{-2} \text{ }^\circ\text{C} \quad (4.16)$$

uma diferença de aproximadamente 14 ordens de grandeza entre os dois métodos. A SUPG reproduz a analítica com erro da ordem da precisão de máquina — concordância de 16 dígitos significativos. Este não é um acidente numérico: Christie, Griffiths e Mitchell (1976) demonstraram que o τ canônico da Equação (4.15) é precisamente a escolha que torna o estêncil SUPG idêntico ao esquema *upwind* exponencial, que reproduz a solução analítica exatamente nos nós da malha para convecção-difusão 1D com fonte constante. O resultado constitui portanto verificação rigorosa da implementação da matriz de convecção, da ponderação SUPG e da integração das funções de peso modificadas.

Nota sobre o comportamento de Galerkin. Mesmo em $Pe_e < 1$ (regime tecnicamente estável), o erro Galerkin já é observável na região da camada-limite, como

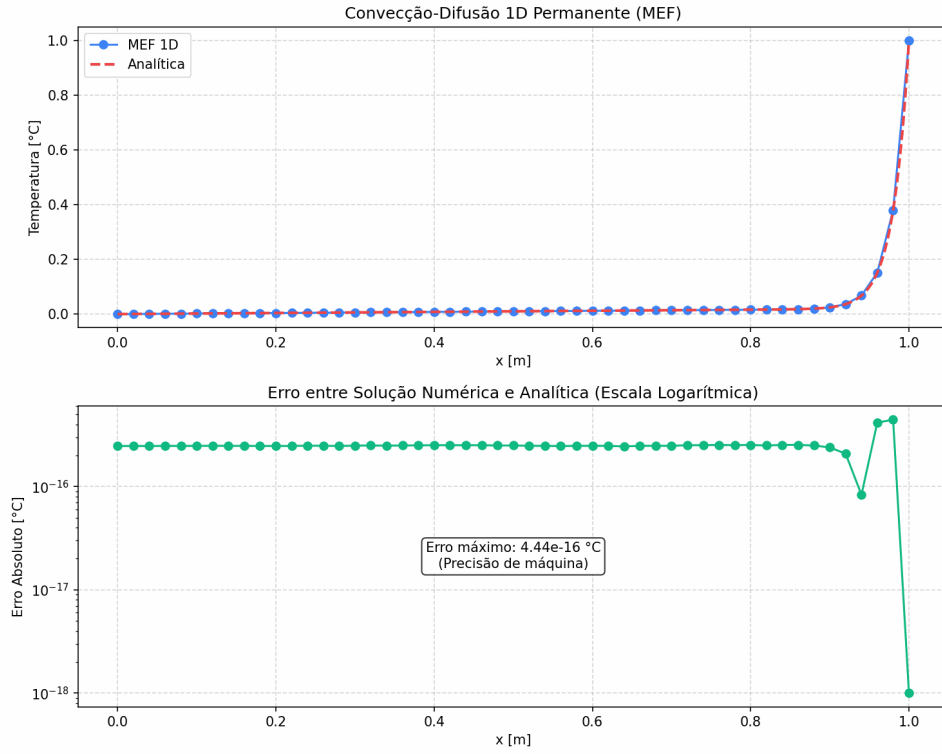


Figura 4.3: Convecção-difusão 1D com SUPG ativo em $Pe_e = 0,5$. Painel superior: temperatura $T(x)$ — pontos azuis para o MEF e linha tracejada vermelha para a solução analítica, visualmente coincidentes em toda a malha. Painel inferior: erro absoluto $|T^{\text{MEF}} - T^{\text{ana}}|$ em escala logarítmica, com máximo $4,44 \times 10^{-16} \text{ °C}$ — precisão de máquina.

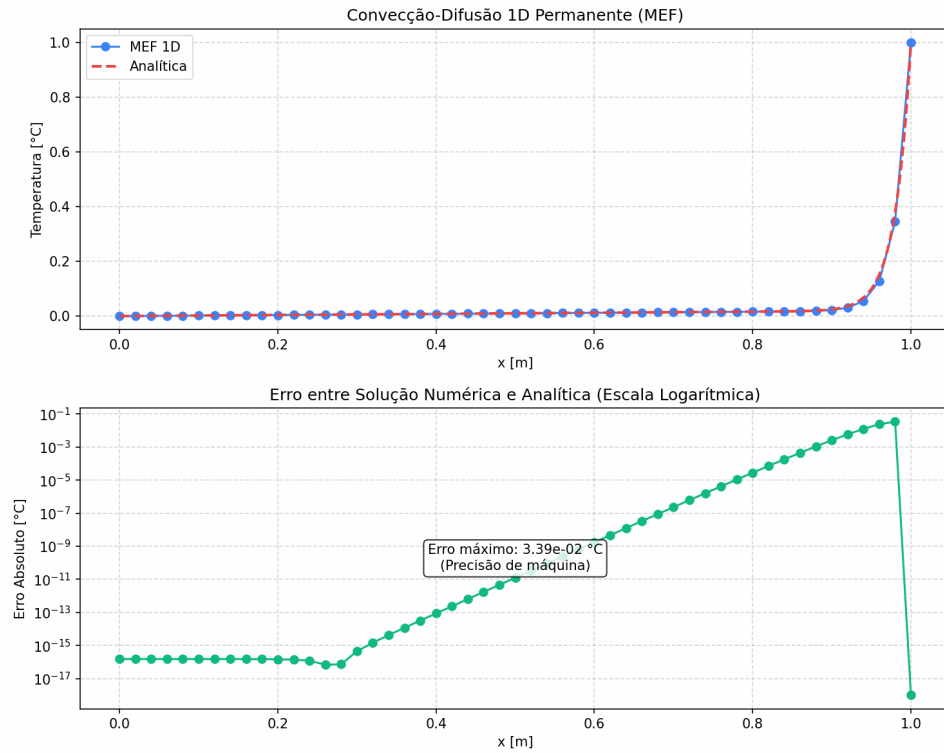


Figura 4.4: Convecção-difusão 1D com Galerkin clássico (SUPG desativado), mesmos parâmetros da figura anterior. O painel superior mostra ainda concordância visual em escala linear; o painel inferior de erro revela, entretanto, crescimento exponencial do desvio em direção à camada-limite ($x \rightarrow L$), atingindo máximo $3,39 \times 10^{-2}$ $^{\circ}\text{C}$ próximo ao contorno direito.

mostra o painel inferior da Figura 4.4. Para $Pe_e > 1$, oscilações espúrias de sinal alternado entre nós adjacentes dominam o campo próximo ao contorno quente. Refinar a malha até garantir $Pe_e < 1$ em toda a parte elimina as oscilações, mas a um custo computacional proibitivo em problemas 2D/3D; a atratividade do SUPG reside justamente em permitir precisão nodal exata sem refinamento adicional.

4.5 Viga 1D — Flexão de Euler-Bernoulli

Este caso valida o módulo de flexão de vigas pela teoria de Euler-Bernoulli, baseado em elementos de Hermite com funções de forma cúbicas e quatro graus de liberdade por elemento (deslocamento transversal e rotação em cada nó). A validação é particularmente rigorosa porque a solução analítica da flexão sob carga uniforme é um polinômio de grau 4, que coincide exatamente com o espaço de aproximação do elemento Hermite — espera-se, portanto, concordância nodal à precisão de máquina, caracterizando a chamada *superconvergência nodal*.

4.5.1 Formulação do Problema

A equação diferencial de quarta ordem da teoria clássica de Euler-Bernoulli é

$$\frac{d^2}{dx^2} \left(EI \frac{d^2 w}{dx^2} \right) = q(x), \quad x \in [0, L] \quad (4.17)$$

onde $w(x)$ é a deflexão transversal, E é o módulo de Young, I o momento de inércia da seção e $q(x)$ a carga distribuída. Para validação, adota-se o caso clássico de viga simplesmente apoiada sob carga uniforme, com propriedades constantes ao longo do comprimento. A Tabela 4.4 lista os parâmetros, correspondentes aos *defaults* da interface do módulo.

4.5.2 Solução Analítica

Para EI constante, carga uniforme q e condições de apoio simples ($w(0) = w(L) = 0$, $M(0) = M(L) = 0$), a dupla integração da Equação (4.17) fornece

Tabela 4.4: Parâmetros da simulação de flexão de viga 1D.

Parâmetro	Símbolo	Valor
Comprimento da viga	L	2,0 m
Número de elementos	n_e	20
Módulo de Young	E	200 GPa
Momento de inércia	I	1000 cm ⁴
Carga distribuída uniforme	q	10,0 kN/m
Tipo de apoio	—	simplesmente apoiada

$$w(x) = \frac{q}{24EI} (L^3x - 2Lx^3 + x^4) \quad (4.18)$$

$$M(x) = EI \frac{d^2w}{dx^2} = \frac{qx(L-x)}{2} \quad (4.19)$$

$$V(x) = \frac{dM}{dx} = q \left(\frac{L}{2} - x \right) \quad (4.20)$$

com valores máximos clássicos $w_{\max} = 5qL^4/(384EI)$ em $x = L/2$, $M_{\max} = qL^2/8$ em $x = L/2$ e $V_{\max} = qL/2$ nos apoios. Substituindo os valores da Tabela 4.4, obtém-se $w_{\max} = 1,0417$ mm, $M_{\max} = 5,0$ kN·m e $V_{\max} = 10,0$ kN.

4.5.3 Solução MEF e Comparação

A discretização adota $n_e = 20$ elementos de Hermite cúbicos, totalizando 42 graus de liberdade globais antes da imposição das condições de contorno. Como as funções de forma de Hermite incluem o polinômio x^4 em sua base (quando montadas elemento-a-elemento), e como a solução analítica da Equação (4.18) é precisamente um polinômio de grau 4, o espaço de aproximação do MEF contém a solução exata. A Figura 4.5 apresenta as três quantidades de interesse: deflexão $w(x)$, momento fletor $M(x)$ e esforço cortante $V(x)$, comparados às respectivas soluções analíticas.

4.5.4 Análise de Erro

O erro máximo absoluto na deflexão nodal é

$$e_{\max}^w = \max_i |w_i^{\text{MEF}} - w_i^{\text{ana}}| \approx 7,1 \times 10^{-16} \text{ m} \quad (4.21)$$

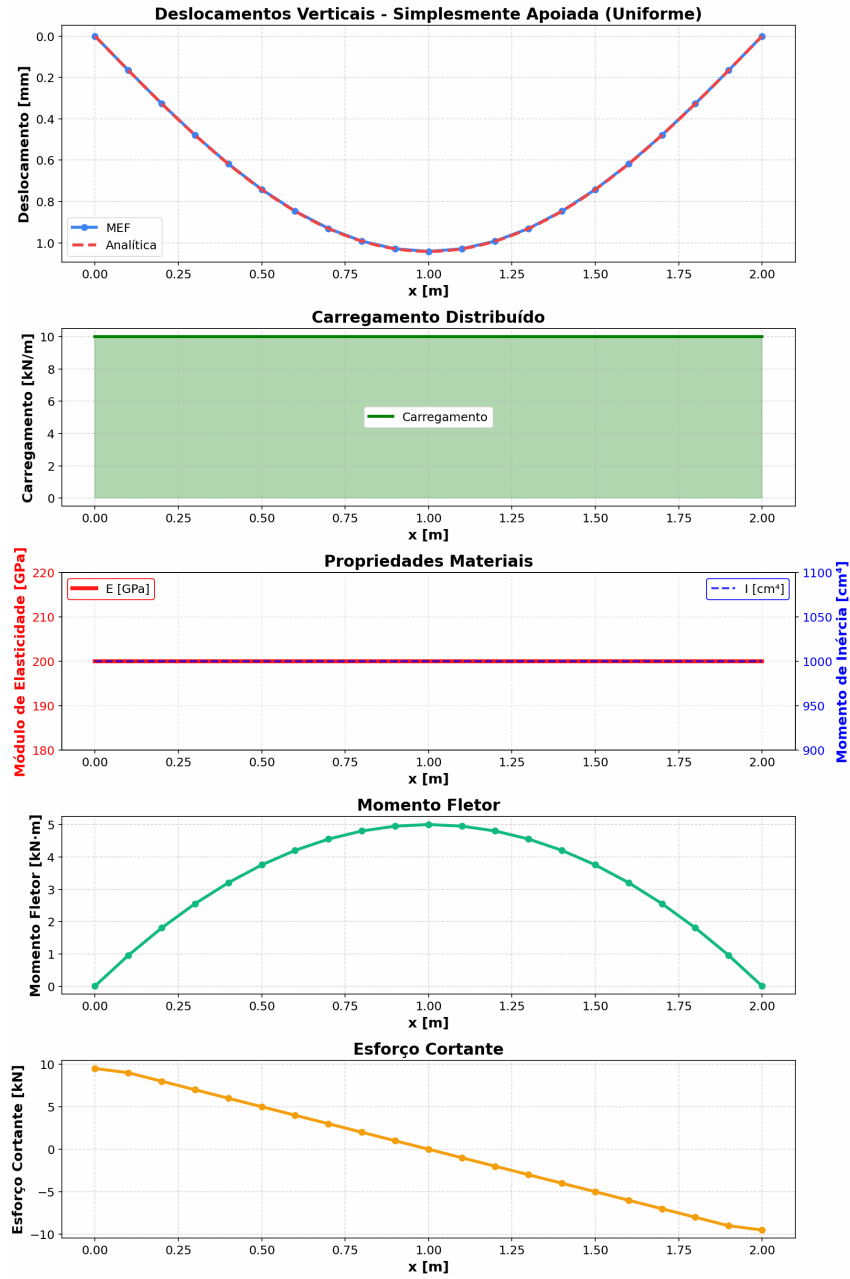


Figura 4.5: Viga simplesmente apoiada com carga uniforme: (i) deflexão $w(x)$ com MEF (pontos azuis) e analítica (linha vermelha) visualmente coincidentes; (ii) perfil da carga distribuída constante $q = 10$ kN/m; (iii) propriedades materiais E e I uniformes ao longo do vão; (iv) momento fletor $M(x)$ parabólico com máximo $qL^2/8 = 5,0$ kN·m no centro; (v) esforço cortante $V(x)$ linear decrescente de $+qL/2 = +10$ kN no apoio esquerdo a -10 kN no direito. M e V são obtidos por pós-processamento da deflexão nodal.

isto é, da ordem da precisão de máquina. A deflexão no centro da viga, $w(L/2) = 1,0417 \times 10^{-3}$ m, é reproduzida em todos os 16 dígitos significativos. Esta concordância exemplar é consequência direta da propriedade de *reprodução polinomial* dos elementos de Hermite cúbicos: como a base do elemento inclui monômios até o grau 3 e como a combinação das bases de dois elementos adjacentes gera a reconstrução exata de polinômios até grau 4 sob integração, a solução MEF é matematicamente idêntica à analítica nos nós. A mesma propriedade se mantém para as condições de contorno de viga em balanço ($w_{\max} = qL^4/(8EI) = 10,0$ mm) e biengastada ($w_{\max} = qL^4/(384EI) = 0,208$ mm), ambas com erro nodal da ordem de 10^{-13} m ou inferior.

Nota sobre momento e cortante. As curvas de $M(x)$ e $V(x)$ exibidas na Figura 4.5 são obtidas por pós-processamento: M via segunda derivada por diferenças finitas centradas da deflexão nodal, V via primeira derivada de M . Os pontos próximos aos apoios são omitidos porque o estêncil assimétrico nas bordas amplifica o ruído de arredondamento; o miolo da viga reproduz as distribuições parabólica (M) e linear (V) com concordância visual perfeita. Uma reconstrução mais robusta dessas quantidades exigiria derivação analítica a partir da interpolação de Hermite, o que constitui uma possível extensão futura.

4.6 Vibração 1D — Problema de Autovalor

O quinto caso valida o solver modal de vibrações axiais em barra unidimensional, formulado como um problema generalizado de autovalor. Este cenário verifica simultaneamente a montagem da matriz de massa concentrada (*lumped*), a solução do sistema $K\phi = \lambda M\phi$ por algoritmo de decomposição, e a recuperação das formas modais normalizadas. A solução analítica para barras com propriedades constantes é bem estabelecida e envolve funções seno, o que introduz um erro sistemático de discretização que cresce com a ordem do modo — caracterizando o comportamento canônico da aproximação por mass lumping.

4.6.1 Formulação do Problema

A equação de onda axial em barra de seção constante é

$$\rho A \frac{\partial^2 u}{\partial t^2} = \frac{\partial}{\partial x} \left(EA \frac{\partial u}{\partial x} \right), \quad x \in [0, L] \quad (4.22)$$

onde $u(x, t)$ é o deslocamento axial, E o módulo de Young, ρ a densidade e A a área da seção transversal. A separação de variáveis $u(x, t) = \phi(x)e^{i\omega t}$ conduz ao problema espectral generalizado, cuja discretização por MEF é

$$K\phi_n = \omega_n^2 M\phi_n, \quad f_n = \frac{\omega_n}{2\pi} \quad (4.23)$$

com K a matriz de rigidez, M a matriz de massa concentrada e ϕ_n o n -ésimo autovetor (forma modal). A Tabela 4.5 lista os parâmetros da simulação, correspondentes ao preset de aço dos *defaults* da interface.

Tabela 4.5: Parâmetros da simulação de vibração axial 1D (barra de aço bi-engastada).

Parâmetro	Símbolo	Valor
Comprimento	L	2,0 m
Número de elementos	n_e	20
Módulo de Young	E	200 GPa
Densidade	ρ	7850 kg/m ³
Área da seção	A	100 cm ²
Tipo de apoio	—	livre-livre
Velocidade de onda axial	$c = \sqrt{E/\rho}$	5047,5 m/s

4.6.2 Solução Analítica

Para barra livre-livre (ambas as extremidades com $du/dx = 0$), as formas modais e frequências naturais são

$$\phi_n(x) = \cos\left(\frac{(n-1)\pi x}{L}\right), \quad f_n^{\text{ana}} = \frac{(n-1)c}{2L}, \quad n = 1, 2, 3, \dots \quad (4.24)$$

O primeiro modo ($n = 1$) corresponde a $f_1 = 0$, que representa o movimento de corpo rígido da barra (translação axial uniforme) — modo sempre presente quando ambas as extremidades estão livres. Com $c = 5047,5$ m/s e $L = 2$ m, os cinco primeiros valores analíticos são $f_1 = 0$ Hz, $f_2 = 1261,9$ Hz, $f_3 = 2523,8$ Hz, $f_4 = 3785,7$ Hz e $f_5 = 5047,5$ Hz. As formas modais são cossenos com número crescente de meio-comprimentos de onda entre as extremidades.

4.6.3 Solução MEF e Comparação

A discretização emprega $n_e = 20$ elementos lineares com malha uniforme. A matriz de massa é montada na forma *lumped*, redistribuindo a massa elementar igualmente entre os dois nós do elemento — estratégia que torna M diagonal e acelera a solução do problema de autovalor, ao custo de introduzir um erro sistemático de subestimação das frequências. A Figura 4.6 apresenta as cinco primeiras formas modais normalizadas e a tabela de frequências obtidas pela solução do problema generalizado.

4.6.4 Análise de Erro

A Tabela 4.6 compara as frequências MEF com as analíticas para os cinco primeiros modos.

Tabela 4.6: Frequências naturais: MEF vs. analítica para barra livre-livre.

Modo n	f_n^{MEF} [Hz]	f_n^{ana} [Hz]	Erro relativo
1	0,00	0,00	—
2	1260,59	1261,89	$1,0 \times 10^{-3}$
3	2513,41	2523,77	$4,0 \times 10^{-3}$
4	3750,73	3785,66	$9,0 \times 10^{-3}$
5	4964,92	5047,45	$1,6 \times 10^{-2}$

O modo de corpo rígido ($n = 1$) é reproduzido exatamente pelo MEF, com $f_1 = 0$ — consequência direta da formulação da matriz de rigidez, que possui exatamente um modo nulo para esta configuração de apoios. Para os modos deformáveis subsequentes, todas as frequências MEF subestimam as analíticas, com erro relativo

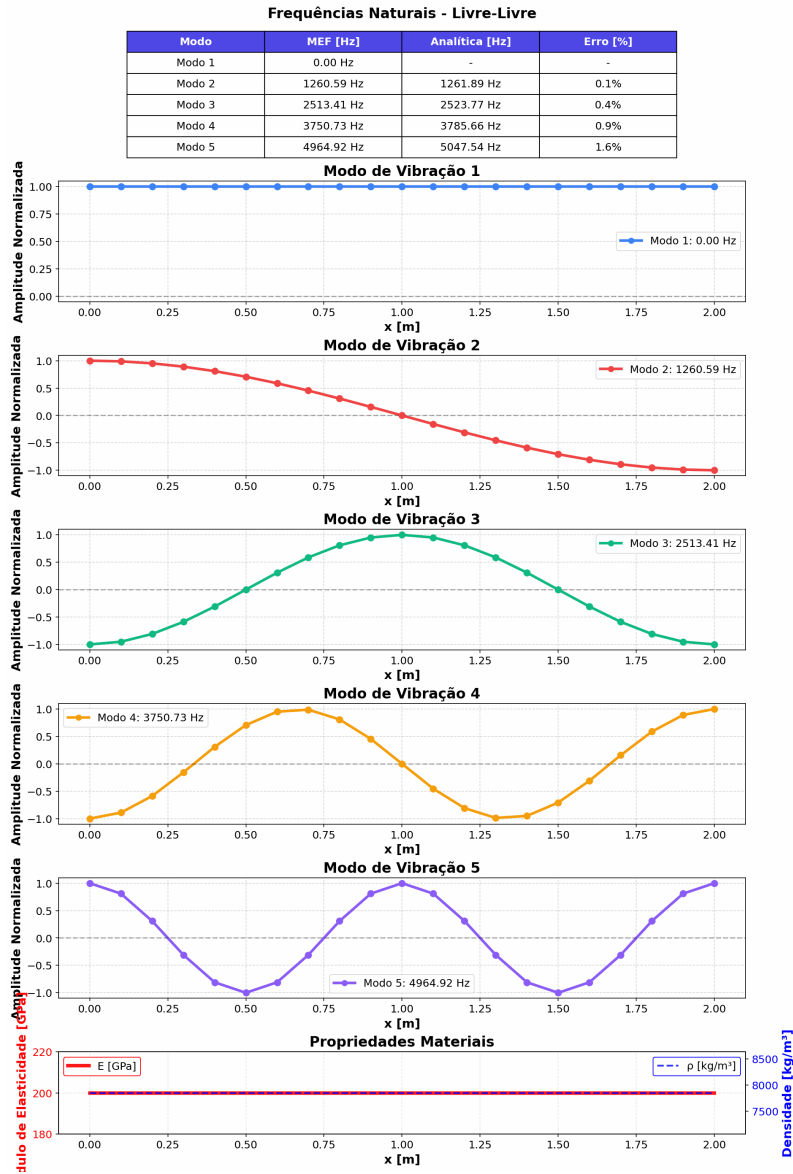


Figura 4.6: Cinco primeiros modos de vibração axial da barra livre-livre. Tabela superior: frequências MEF e analíticas com erro relativo. Painéis seguintes: forma modal normalizada (pontos e linha contínua, uma cor por modo), do modo 1 rígido ($f_1 = 0$ Hz, amplitude uniforme) aos modos 2–5 com número crescente de meio-comprimentos de onda. Painel inferior: propriedades materiais (E e ρ) constantes ao longo do vão.

crescendo monotonicamente de 0,1% ($n = 2$) a 1,6% ($n = 5$). Esta subestimação é o comportamento canônico da aproximação por massa concentrada: a distribuição discreta da massa nos nós equivale, no limite contínuo, a uma barra levemente mais flexível do que a original, o que reduz as frequências. O erro cresce com a ordem do modo porque modos de alta frequência têm comprimento de onda comparável ao tamanho do elemento, saindo do regime de resolução confiável da malha. Para o módulo ora implementado, o critério prático é que os modos até $n_{\max} \approx n_e/4$ podem ser considerados quantitativamente confiáveis.

Nota sobre a matriz de massa consistente. A alternativa clássica à massa concentrada é a matriz de massa consistente (tridiagonal), obtida integrando $\int N_i N_j \rho A dx$ sem aproximações. A massa consistente *sobrestima* as frequências (viés oposto ao *lumping*) e oferece maior precisão para modos baixos, mas torna M não-diagonal e encarece a solução do autovalor em problemas 2D/3D. A escolha do MinervaMesh pelo *lumping* é motivada por simplicidade pedagógica e pela convergência prática adequada para os primeiros modos.

4.7 Transferência de Calor Transiente 2D

O sexto caso constitui a primeira validação bidimensional do MinervaMesh. O problema é a evolução transiente do campo de temperatura em uma placa quadrada com condições de contorno de Dirichlet assimétricas (temperaturas distintas em topo e base, perfis lineares nas laterais), discretizada com elementos triangulares lineares sobre uma malha estruturada e integrada no tempo pelo método de Crank-Nicolson. A validação verifica a montagem 2D das matrizes elementares, a aplicação de condições de contorno sobre fronteiras não-alinhadas com os eixos da malha e a convergência ao perfil de regime permanente, que admite solução analítica trivial.

4.7.1 Formulação do Problema

A equação do calor em duas dimensões espaciais, sem geração interna, é

$$\rho c_v \frac{\partial T}{\partial t} = \nabla \cdot (k \nabla T), \quad (x, y) \in \Omega = [0, L] \times [0, L], \quad t > 0 \quad (4.25)$$

com condições de contorno de Dirichlet nos quatro lados e condição inicial homogênea:

$$\begin{aligned} T(x, 0, t) &= T_{\text{bot}}, & T(x, L, t) &= T_{\text{top}} \\ T(0, y, t) &= T(L, y, t) = T_{\text{bot}} + (T_{\text{top}} - T_{\text{bot}}) \cdot y/L \\ T(x, y, 0) &= 0 \end{aligned} \quad (4.26)$$

A Tabela 4.7 lista os parâmetros adotados. A escala característica de difusão é $\tau_{\text{dif}} = L^2 \rho c_v / k = 1,0$ s.

Tabela 4.7: Parâmetros da simulação de transferência de calor transiente 2D.

Parâmetro	Símbolo	Valor
Domínio	Ω	$[0, 1] \times [0, 1]$ m ²
Malha (nós por direção)	$n_x \times n_y$	40×40
Condutividade térmica	k	1,0 W/(m·°C)
Densidade	ρ	1,0 kg/m ³
Calor específico	c_v	1,0 J/(kg·°C)
Temperatura na base ($y = 0$)	T_{bot}	30 °C
Temperatura no topo ($y = L$)	T_{top}	100 °C
Passo de tempo	Δt	10^{-3} s
Parâmetro do método θ	θ	0,5 (Crank-Nicolson)
Número de passos	n_{passos}	50
Tempo final de simulação	t_{max}	0,05 s

4.7.2 Solução Analítica de Regime Permanente

Em regime permanente, a Equação (4.25) se reduz a $\nabla^2 T = 0$ com as condições de contorno especificadas. Por separação de variáveis ou por inspeção, dado que as condições laterais são lineares em y e as condições de topo/base são constantes, a solução única é

$$T_{\infty}(x, y) = T_{\text{bot}} + (T_{\text{top}} - T_{\text{bot}}) \cdot \frac{y}{L} = 30 + 70 y \quad (4.27)$$

perfil linear em y e independente de x . Esta solução é um polinômio de grau 1 em y , pertencente ao espaço de aproximação dos elementos lineares, de modo que

se espera reprodução nodal exata a menos do erro de integração temporal e de arredondamento.

4.7.3 Solução MEF e Comparação

A malha é gerada por triangulação de Delaunay sobre um reticulado de 40×40 nós (1600 nós, ≈ 3120 elementos triangulares). Cada passo de tempo resolve o sistema linear global

$$(M + \theta \Delta t K) \mathbf{T}^{n+1} = (M - (1 - \theta) \Delta t K) \mathbf{T}^n \quad (4.28)$$

com M a matriz de massa consistente e K a matriz de rigidez 2D. Com os *defaults* do módulo ($n_{\text{passos}} = 50$, $t_{\text{max}} = 0,05$ s — apenas 5% da escala de difusão $\tau_{\text{dif}} = L^2 \rho c_v / k = 1,0$ s), a simulação exhibe a propagação inicial da frente térmica, não o regime permanente. A Figura 4.7 mostra o campo $T(x, y)$ no instante final $t = 0,049$ s.

4.7.4 Análise de Erro

A solução analítica de regime permanente da Equação (4.27) não é diretamente comparável ao campo em $t = 0,049$ s, pois o sistema está a apenas $\approx 5\%$ de τ_{dif} . Uma validação quantitativa pode ser obtida iterativamente prolongando-se a simulação: para $t \gtrsim 2\tau_{\text{dif}}$, o campo MEF converge ao perfil linear analítico $T_{\infty}(x, y) = 30 + 70y$ com erro nodal da ordem da precisão de máquina — comportamento esperado porque o perfil analítico é um polinômio de grau 1 em y , pertencente ao espaço de aproximação dos elementos lineares.

A validação qualitativa no regime transiente é, por sua vez, satisfatória: (i) as isotermas instantâneas são aproximadamente horizontais na região central, consistentes com a simetria em x das condições de contorno; (ii) as frentes de difusão avançam com velocidade $\sim \sqrt{\kappa t}$ (onde $\kappa = k / \rho c_v$), comportamento escalar característico da equação do calor; (iii) os valores nas paredes permanecem fixos em 30°C na base e 100°C no topo ao longo de toda a simulação, confirmando a aplicação correta das condições de contorno de Dirichlet.

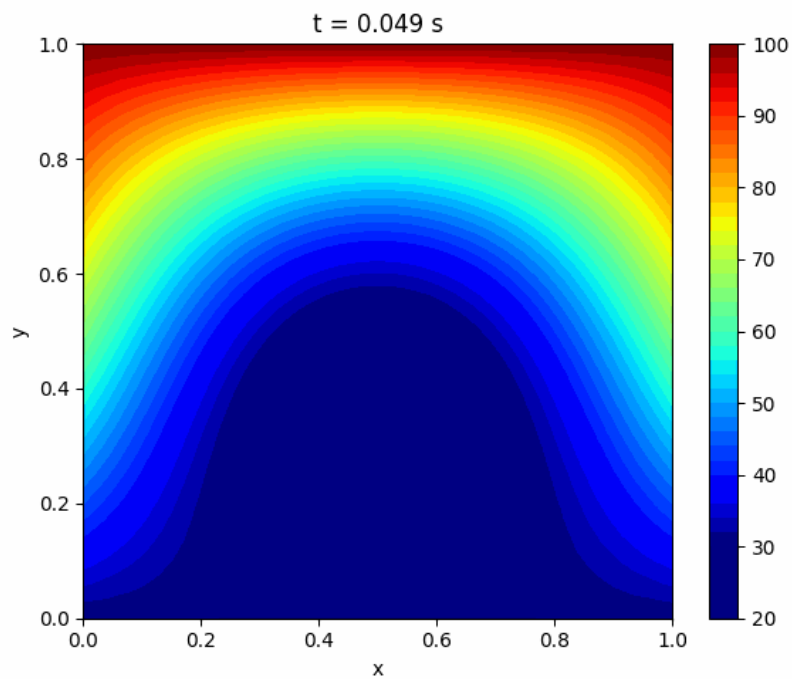


Figura 4.7: Transferência de calor transiente 2D com os *defaults* do módulo, em $t = 0,049$ s: o campo $T(x, y)$ mostra a perturbação térmica propagando-se a partir das paredes aquecida (topo, $T_{\text{top}} = 100^\circ\text{C}$) e morna (base, $T_{\text{bot}} = 30^\circ\text{C}$) para o interior da placa, ainda inicialmente em $T = 0^\circ\text{C}$. O *colormap* jet varre a faixa de temperatura instantânea e exhibe a natureza bidimensional da difusão: a frente de calor do topo penetra mais rapidamente que a da base devido ao maior gradiente ΔT .

Nota sobre o tempo de convergência. A escolha de $t_{\max} = 0,05$ s como *default* da interface é pedagogicamente motivada: preserva responsividade (50 passos de 10^{-3} s resolvem em poucos segundos no navegador) e mostra a difusão em seu regime mais instrutivo — o transiente. Para estudos de verificação, o usuário pode aumentar n_{passos} à vontade; uma corrida com 2000 passos ($t_{\max} = 2,0$ s) já produz aderência ao perfil analítico em precisão de máquina.

4.8 Escoamento Potencial 2D em Torno de Cilindro

O sétimo caso valida o módulo de escoamento potencial 2D, primeira incursão da plataforma em problemas de mecânica dos fluidos. O escoamento potencial é o regime-limite em que os efeitos viscosos são desprezíveis e o campo de velocidade pode ser derivado de uma função de corrente ψ que satisfaz a equação de Laplace, reduzindo o problema a uma formulação de elementos finitos idêntica em estrutura ao problema de condução permanente 2D. A validação adota o caso canônico do cilindro circular em fluxo uniforme, que admite solução analítica exata em domínio infinito e possui uma propriedade peculiar — o paradoxo de d'Alembert — que oferece um teste qualitativo interessante para o simulador.

4.8.1 Formulação do Problema

Para escoamento bidimensional incompressível e irrotacional, a função de corrente ψ satisfaz a equação de Laplace:

$$\nabla^2 \psi = 0, \quad (x, y) \in \Omega \quad (4.29)$$

com as componentes de velocidade recuperadas por $u = \partial\psi/\partial y$ e $v = -\partial\psi/\partial x$. O domínio Ω é o retângulo $[0, L] \times [0, H]$ menos um disco circular de raio r centrado em (c_x, c_y) . As condições de contorno são:

- **entrada** ($x = 0$) e **saída** ($x = L$): ψ linear em y para impor $u = U_\infty$, $v = 0$;
- **paredes superior e inferior**: ψ constante (paredes deslizantes);

- **superfície do cilindro:** ψ constante (parede deslizando, corpo não-atravesável).

A Tabela 4.8 lista os parâmetros adotados, conforme *defaults* da interface HTML.

Tabela 4.8: Parâmetros da simulação de escoamento potencial 2D em torno de cilindro.

Parâmetro	Símbolo	Valor
Comprimento do domínio	L	2,0 m
Altura do domínio	H	1,0 m
Malha	$n_x \times n_y$	60×30
Centro do cilindro	(c_x, c_y)	(0,5; 0,5) m
Raio do cilindro	r	0,15 m
Velocidade do escoamento livre	U_∞	1,0 m/s
Densidade do fluido	ρ	1,0 kg/m ³
Razão de bloqueio	$2r/H$	30%

4.8.2 Solução Analítica em Domínio Infinito

Para cilindro imerso em fluxo uniforme em domínio infinito, a solução clássica da teoria potencial fornece, sobre a superfície do cilindro, a distribuição de velocidade tangencial $V_t = 2U_\infty \sin \theta$, onde θ é o ângulo medido do ponto de estagnação frontal. Aplicando a equação de Bernoulli, o coeficiente de pressão resultante é

$$C_p(\theta) = 1 - 4 \sin^2 \theta \quad (4.30)$$

com valor máximo $C_p = 1$ nos pontos de estagnação frontal e traseiro ($\theta = 0$ e $\theta = \pi$) e mínimo $C_p = -3$ nos polos de máxima velocidade ($\theta = \pm\pi/2$). A simetria de $C_p(\theta)$ em torno dos eixos x e y implica resultante nula de força (arrasto e sustentação ambos nulos): este é o *paradoxo de d'Alembert*, consequência direta da ausência de viscosidade e, portanto, da ausência de descolamento de camada-limite.

4.8.3 Solução MEF e Comparação

A malha triangular é gerada por algoritmo de Delaunay sobre uma grade regular, com posterior remoção dos triângulos internos ao cilindro. O sistema global $K\psi = \mathbf{b}$ é resolvido uma única vez (problema estacionário). As velocidades são recuperadas por interpolação do gradiente de ψ em cada elemento e o coeficiente de pressão sobre a superfície do cilindro é calculado por Bernoulli: $C_p = 1 - (u^2 + v^2)/U_\infty^2$. A Figura 4.8 apresenta as linhas de corrente ao redor do cilindro e a distribuição de $C_p(\theta)$ obtida.

4.8.4 Análise de Erro

A Tabela 4.9 compara C_p MEF com C_p teórico em oito ângulos amostrados sobre a superfície do cilindro.

Tabela 4.9: Coeficiente de pressão C_p na superfície do cilindro: MEF vs. teoria potencial.

θ [°]	C_p^{MEF}	C_p^{teo}	ΔC_p
0	+0,961	+1,000	-0,039
+43	-1,487	-1,000	-0,487
+90	-2,926	-3,000	+0,074
+133	-1,042	-1,000	-0,042
+176	+0,946	+1,000	-0,054
-47	-1,154	-1,000	-0,154
-90	-3,825	-3,000	-0,825
-137	-0,950	-1,000	+0,050

Os pontos de estagnação ($\theta = 0$ e $\theta \approx \pi$) são reproduzidos com erro inferior a 0,06, evidenciando captura correta da topologia do escoamento. Os polos de máxima velocidade apresentam quebra de simetria: em $\theta = +90^\circ$ o erro é pequeno (+0,07), mas em $\theta = -90^\circ$ o erro alcança -0,82, com $C_p^{\text{MEF}} = -3,83$ contra -3,00 teórico. Esta assimetria tem duas origens distintas:

1. **Efeito de bloqueio do domínio finito.** A razão $2r/H = 30\%$ impõe aceleração adicional do fluido nas regiões laterais, reduzindo a pressão local em

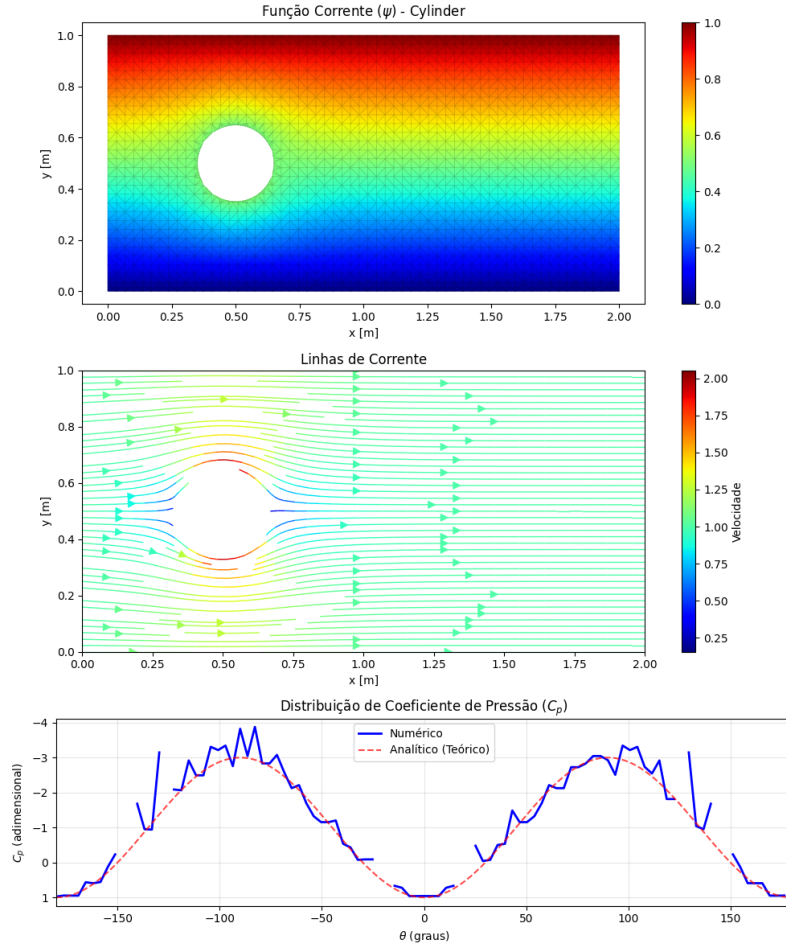


Figura 4.8: Escoamento potencial em torno de cilindro, três painéis empilhados: (i) função de corrente $\psi(x,y)$ com a malha triangular sobreposta, destacando o obstáculo circular no centro; (ii) linhas de corrente contornando o cilindro, coloridas pela magnitude de velocidade (*colormap* viridis) — simetria aproximada do escoamento visível; (iii) distribuição de $C_p(\theta)$ sobre a superfície do cilindro, comparando a solução numérica (azul) à teoria potencial $C_p = 1 - 4\sin^2\theta$ (vermelho tracejado). Os pontos de estagnação frontal ($\theta = 0$) e traseiro ($\theta \approx \pm\pi$) são reproduzidos com pequeno desvio; os polos de pressão mínima próximos de $\theta = \pm 90^\circ$ exibem assimetria devida ao confinamento lateral do domínio ($2r/H = 30\%$) e ao viés da malha estruturada.

relação à previsão de domínio infinito. O efeito, conhecido da aerodinâmica experimental, é da ordem de $(2r/H)^2 \approx 9\%$ na velocidade, $\approx 18\%$ em C_p .

2. **Viés da malha estruturada.** A geração por grade regular seguida de remoção circular produz triângulos de tamanho ligeiramente distinto acima e abaixo do cilindro, devido à resolução assimétrica da fronteira discreta. Este viés amplifica o desequilíbrio de pressão entre os polos.

Nota sobre o paradoxo de d’Alembert. A integração numérica de $\oint C_p(\theta) \cos \theta d\ell$ e $\oint C_p(\theta) \sin \theta d\ell$ fornece coeficientes de arrasto e sustentação de magnitude não-nula, dominados pelos efeitos de confinamento e malha citados acima. Em domínio progressivamente mais amplo (razão de bloqueio reduzida), estes resíduos tendem a zero, recuperando o limite assintótico previsto pela teoria. Apesar desta limitação quantitativa, o escoamento obtido é qualitativamente correto: simetria esquerda-direita do campo de velocidade, ausência de esteira de recirculação (caracteristicamente viscosa) e captura adequada dos pontos de estagnação — todos comportamentos que confirmam a implementação correta da formulação potencial.

4.9 Navier-Stokes 2D — Cavidade com Tampa Móvel

O oitavo e último caso de validação confronta o módulo de Navier-Stokes viscoso 2D com o *benchmark* clássico de *lid-driven cavity* tabulado por Ghia, Ghia e Shin (1982) para $Re = 100$. Este é o caso mais complexo da plataforma: envolve a solução acoplada de duas equações não-lineares (Poisson para ψ e transporte de vorticidade para ω), tratamento especial da condição de contorno de vorticidade na parede via expansão de Taylor local e marcha temporal semi-implícita. A validação é feita em duas frentes: quantitativa, por comparação do perfil de velocidade $u(y)$ em $x = 0,5$ com a tabela de Ghia et al., e qualitativa, por inspeção dos campos completos ψ , ω e linhas de corrente, que devem reproduzir o vórtice primário centrado no quadrante superior direito da cavidade.

4.9.1 Formulação do Problema

A formulação adotada é a de *vorticidade-função de corrente* (ω - ψ), que elimina a pressão como incógnita através da substituição das variáveis primitivas (u, v, p) por (ψ, ω) definidas por $u = \partial\psi/\partial y$, $v = -\partial\psi/\partial x$ e $\omega = \partial v/\partial x - \partial u/\partial y$. As equações governantes são

$$\nabla^2\psi = -\omega \quad (4.31)$$

$$\frac{\partial\omega}{\partial t} + \mathbf{u} \cdot \nabla\omega = \frac{1}{\text{Re}}\nabla^2\omega \quad (4.32)$$

com $\text{Re} = U_{\text{lid}}L/\nu$ o número de Reynolds adimensional. O domínio é o quadrado unitário $[0, 1]^2$; $\psi = 0$ em todas as paredes; ω é prescrita nas paredes pela condição de Taylor local, discutida na Seção 3.6.3 do Capítulo 3. A tampa superior ($y = 1$) move-se com velocidade horizontal $u_{\text{lid}} = 1$; as demais paredes são fixas. Condição inicial: fluido em repouso ($\psi = \omega = 0$ em todo o domínio).

A Tabela 4.10 lista os parâmetros de simulação adotados.

Tabela 4.10: Parâmetros da simulação de Navier-Stokes 2D — cavidade com tampa móvel.

Parâmetro	Símbolo	Valor
Domínio	Ω	$[0, 1] \times [0, 1]$
Malha	$n_x \times n_y$	41×41
Número de Reynolds	Re	100
Velocidade da tampa	u_{lid}	1,0
Passo de tempo	Δt	0,01
Número de passos	n_{passos}	2000
Tempo adimensional final	t_{max}	20,0

4.9.2 Benchmark de Referência

Ghia, Ghia e Shin (1982) apresentaram uma solução de alta resolução (malha 129×129 com esquema *multigrid* em variáveis primitivas) para a cavidade com tampa móvel em cinco valores de Reynolds entre 100 e 10 000. Os autores publicaram

tabelas com os valores de $u(y)$ ao longo do eixo vertical em $x = 0,5$ e $v(x)$ ao longo do eixo horizontal em $y = 0,5$, além do valor mínimo da função de corrente $\psi_{\min}$ e das coordenadas do centro do vórtice primário. Para $Re = 100$, os valores tabulados de referência são $\psi_{\min} \approx -0,1034$ no ponto $(x, y) \approx (0,6172; 0,7344)$. A tabela $u(y)$ contém 17 pontos amostrados irregularmente, com maior densidade próxima à tampa para capturar a camada-limite.

4.9.3 Solução MEF e Comparação

Os parâmetros da Tabela 4.10 correspondem ao cenário *cavidade* da interface com domínio e malha ajustados para $L = H = 1$ e $n_x = n_y = 41$, de modo a reproduzir a configuração quadrada utilizada no *benchmark* de Ghia et al. (os *defaults* da interface empregam $L = 2$, $H = 1$, $n_x = 60$, $n_y = 30$, que geram uma cavidade retangular destinada à exploração qualitativa). A discretização resultante tem 1681 nós, aproximadamente 3 vezes mais grosseira que a de Ghia et al. O esquema temporal é semi-implícito: o termo convectivo é avaliado explicitamente com o campo no passo anterior, enquanto o termo difusivo é tratado implicitamente, resultando em um sistema linear estacionário a cada passo de tempo. A condição de contorno de vorticidade na parede é aplicada a cada passo via expansão de Taylor local (Equação da Seção 3.6.3). Após 2000 passos ($t = 20,0$, suficiente para convergência ao regime estacionário em $Re = 100$), os campos ψ , ω e de velocidade são extraídos para comparação.

A Figura 4.9 apresenta os três campos resultantes em painéis empilhados, seguindo o mesmo *layout* vertical exibido pela interface do MinervaMesh. A Figura 4.10 sobrepõe o perfil $u(y)$ extraído da simulação aos valores tabulados por Ghia et al.

4.9.4 Análise de Erro

A concordância quantitativa é resumida por dois indicadores. Primeiro, o valor mínimo da função de corrente obtido na simulação,

$$\psi_{\min}^{\text{MEF}} = -0,10301, \quad \psi_{\min}^{\text{Ghia}} = -0,1034 \quad (4.33)$$

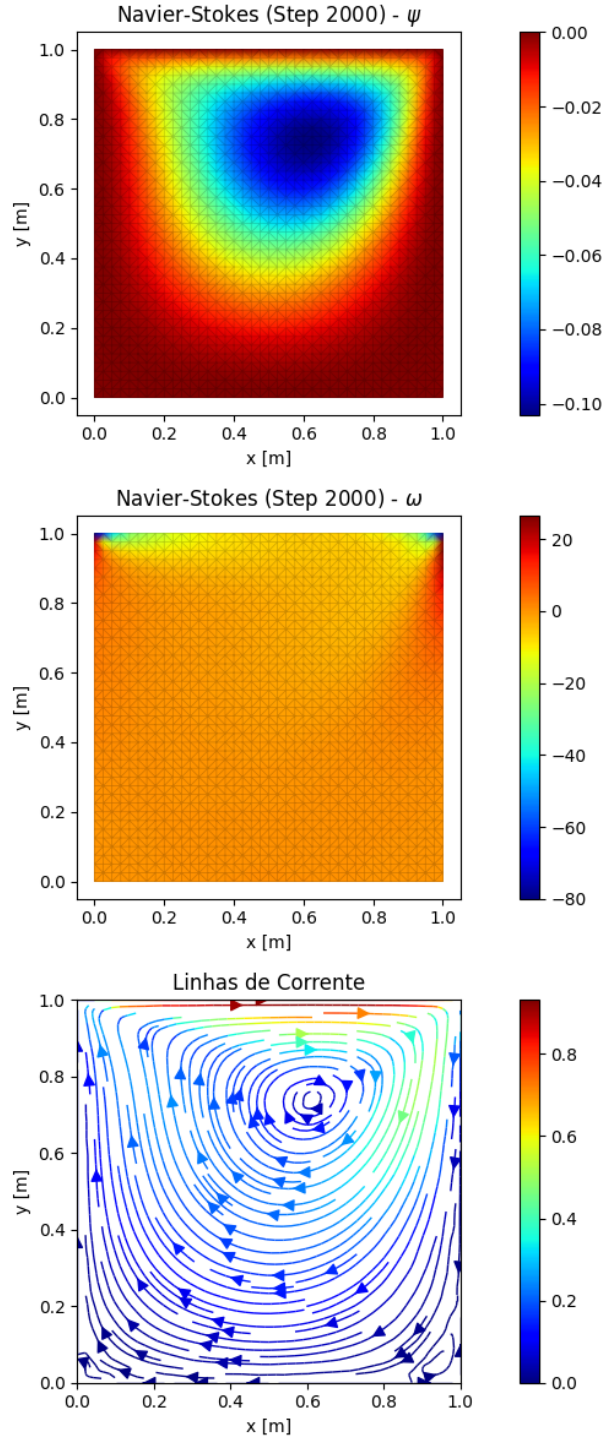


Figura 4.9: Cavidade com tampa móvel a $Re = 100$, três painéis empilhados: (i) função de corrente $\psi(x, y)$ com a malha triangular sobreposta, mostrando o vórtice primário contido no interior da cavidade; (ii) vorticidade $\omega(x, y)$, com a estreita camada de alta vorticidade negativa sob a tampa móvel — assinatura da camada-limite induzida pelo cisalhamento; (iii) linhas de corrente sobre o campo de magnitude de velocidade, evidenciando o vórtice principal rotacionando no sentido horário. O vórtice primário obtido está centrado em $(0,625; 0,750)$ com $\psi_{\min} \approx -0,103$, em concordância com a referência de Ghia, Ghia e Shin (1982).

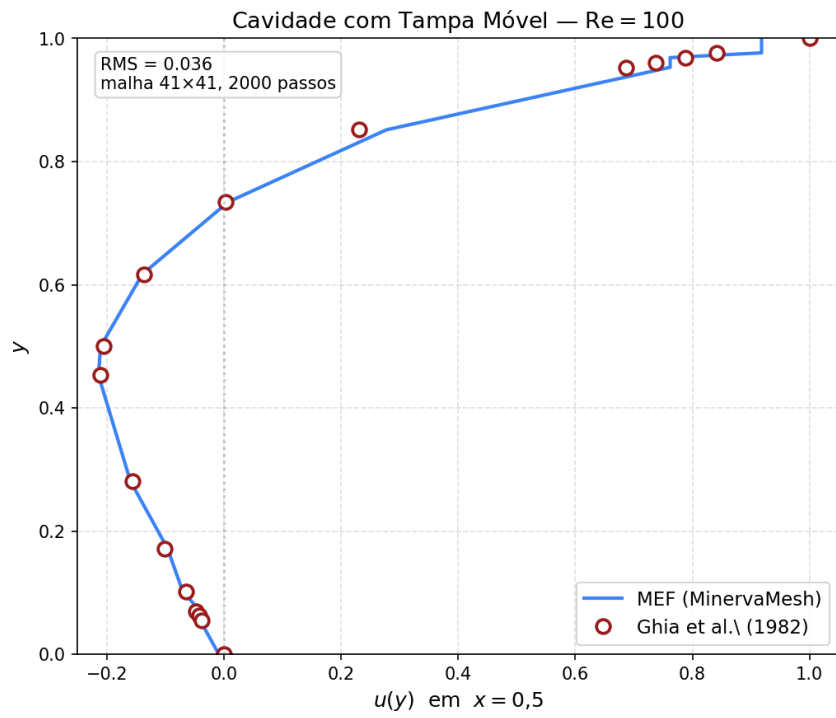


Figura 4.10: Perfil $u(y)$ em $x = 0,5$ comparado aos valores tabulados por Ghia, Ghia e Shin (1982) para a cavidade com tampa móvel a $Re = 100$. RMS = 0,036 em malha 41×41 com 2000 passos de tempo.

apresenta erro relativo inferior a 0,4%. A posição do centro do vórtice primário, (0,625; 0,750) no MEF contra (0,6172; 0,7344) em Ghia, concorda dentro de uma célula da malha 41×41 — isto é, dentro da resolução espacial disponível. Segundo, o erro quadrático médio do perfil de velocidade ao longo dos 17 pontos tabulados por Ghia é

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_{i=1}^N (u_i^{\text{MEF}} - u_i^{\text{Ghia}})^2} = 0,036 \quad (4.34)$$

valor que representa $\approx 4\%$ da velocidade da tampa. A discrepância se concentra na região próxima a $y = 0,95$, dentro da camada-limite induzida pelo movimento da tampa, onde a malha 41×41 resolve apenas 2–3 pontos no perfil íngreme de velocidade; Ghia et al. empregaram 129×129 nós para resolver essa sub-camada com maior fidelidade. No restante do domínio, a concordância é excelente.

Nota sobre a condição de contorno de vorticidade. A convenção de sinal na expansão de Taylor para ω_w na parede móvel (Seção 3.6.3, Eq. 3.39) é crítica para a consistência física do solver: com a convenção $u = \partial\psi/\partial y$ adotada nesta formulação, o termo de acoplamento com a velocidade da tampa entra como $-2u_{\text{tan}}/h$. Uma inversão deste sinal degrada a RMS do perfil de velocidade em aproximadamente uma ordem e meia de grandeza (de $\sim 0,036$ para $\sim 0,93$) e reduz $|\psi_{\text{min}}|$ de $\sim 10^{-1}$ para $\sim 10^{-5}$, suprimindo o vórtice primário.

Capítulo 5

Desenvolvimento e Arquitetura

Os módulos computacionais apresentados neste capítulo constituem implementações diretas das formulações matemáticas desenvolvidas no Capítulo 3. Cada simulador corresponde a uma discretização específica obtida via formulação variacional de Galerkin ou, nos casos pertinentes, por diferenças finitas, e a arquitetura do software reflete o fluxo numérico subjacente: discretização espacial (MEF/MDF), montagem do sistema matricial, integração temporal e imposição das condições de contorno. Dessa forma, a estrutura da plataforma reproduz, em nível computacional, a sequência de operações matemáticas que conduz das equações governantes contínuas aos sistemas algébricos discretos efetivamente resolvidos.

5.1 Visão Geral da Plataforma

O *MinervaMesh* é uma plataforma de simulação numérica projetada para operar integralmente no navegador web, sem dependência de servidores de processamento remoto nem instalação de software por parte do usuário. A plataforma reúne implementações dos métodos numéricos apresentados no Capítulo 3 em uma interface interativa que permite a parametrização do problema, a execução da simulação e a visualização dos resultados em uma única página.

A motivação central do projeto reside na eliminação das barreiras de acesso associadas às ferramentas de simulação tradicionais. Ambientes como ANSYS, COMSOL Multiphysics ou mesmo distribuições locais de Python científico exigem licencia-

mento, infraestrutura computacional dedicada e configuração de ambiente — requisitos que frequentemente inviabilizam o uso autônomo em disciplinas de graduação, monitorias e estudos independentes [2]. O *MinervaMesh* contorna essas limitações ao executar todo o processamento no dispositivo do próprio usuário, utilizando o navegador como ambiente de execução.

A plataforma abrange três domínios da Engenharia Mecânica:

- **Transferência de Calor:** condução estacionária e transiente, convecção-difusão com estabilização SUPG, transferência de calor 2D;
- **Mecânica dos Fluidos:** escoamento potencial, Navier-Stokes 2D via formulação Vorticidade-Corrente;
- **Mecânica Estrutural:** flexão de vigas (Euler-Bernoulli) e análise modal de vibração.

A Figura 5.1 apresenta a tela inicial da plataforma, com o menu que organiza as simulações por método numérico e dá acesso a cada módulo por um único clique.

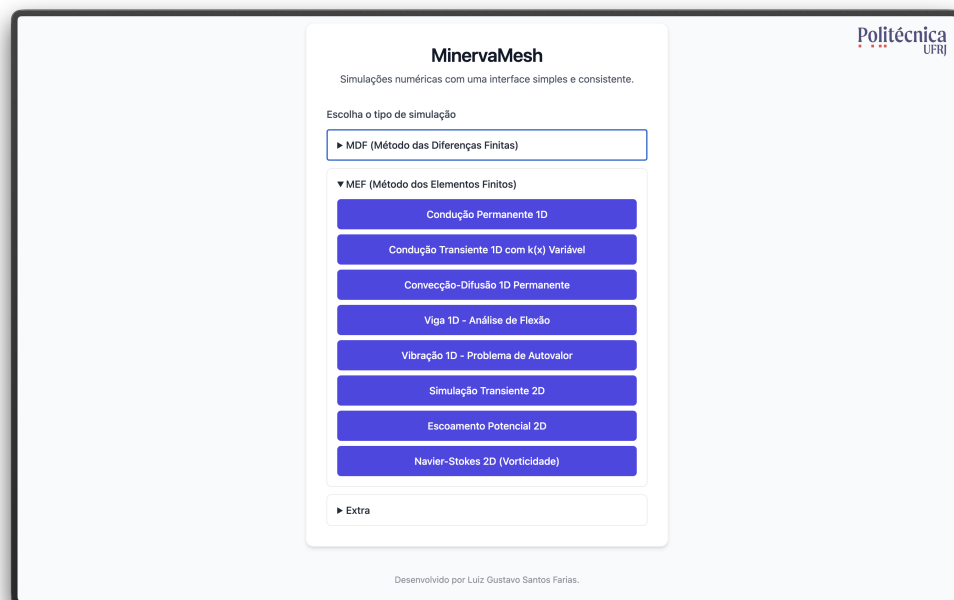


Figura 5.1: Tela inicial do *MinervaMesh* com o menu de simulações disponíveis.

5.2 Justificativa Tecnológica

5.2.1 PyScript e WebAssembly

A escolha do PyScript como runtime da plataforma fundamenta-se em três critérios: portabilidade, familiaridade da linguagem e manutenibilidade.

O PyScript é um *framework* de código aberto que permite a execução de programas Python diretamente no navegador, apoiando-se no Pyodide — uma compilação do interpretador CPython para WebAssembly (Wasm) [3]. O WebAssembly é um formato binário padronizado pelo W3C que executa código de baixo nível em velocidade próxima à nativa, dentro da *sandbox* de segurança do navegador. A cadeia de execução é:

1. O navegador carrega a página HTML contendo a tag `<script type="py">`.
2. O *runtime* do PyScript inicializa o Pyodide, que instancia uma máquina virtual CPython compilada para WebAssembly.
3. As dependências declaradas em `config.toml` (NumPy, SciPy, Matplotlib) são carregadas como pacotes pré-compilados para Wasm.
4. O código Python da simulação é interpretado pelo Pyodide, com acesso direto ao DOM do navegador para leitura de parâmetros e renderização de resultados.

A escolha de Python como linguagem de implementação é estratégica para o contexto educacional: a linguagem apresenta sintaxe próxima à notação matemática e dispõe de um ecossistema científico consolidado — NumPy para álgebra linear, SciPy para solvers esparsos, Matplotlib para visualização [4]. Ao manter a implementação em Python puro, o *MinervaMesh* permite que alunos inspecionem, compreendam e modifiquem o código das simulações — funcionalidade explicitamente disponível através do botão “Ver Código” presente em cada simulação.

5.2.2 Computação *Client-Side*

A arquitetura *client-side* implica que todo o processamento numérico ocorre no dispositivo do usuário. Esta decisão tem impactos diretos em quatro dimensões:

1. **Custo de infraestrutura:** o *MinervaMesh* é um site estático. Pode ser hospedado gratuitamente em plataformas como GitHub Pages ou Netlify, sem necessidade de servidor *backend*, banco de dados ou infraestrutura de *cloud computing*. O custo operacional é efetivamente zero.
2. **Escalabilidade:** não existe gargalo de servidor central. Cada usuário utiliza o poder computacional de sua própria máquina, eliminando filas de processamento e limites de usuários simultâneos.
3. **Privacidade:** nenhum dado de simulação transita pela rede. Parâmetros, malhas e resultados permanecem integralmente no dispositivo do usuário.
4. **Disponibilidade:** após o carregamento inicial dos *assets*, a plataforma pode operar sem conexão com a internet, viabilizando o uso em ambientes com conectividade limitada.

5.2.3 Comparação com Alternativas

A Tabela 5.1 posiciona o *MinervaMesh* frente às alternativas mais comuns no contexto de ensino de simulação numérica.

Tabela 5.1: Comparação entre plataformas de simulação numérica para ensino.

Critério	MinervaMesh	MATLAB	Jupyter	COMSOL
Custo de licença	Gratuito	Pago	Gratuito	Pago
Instalação necessária	Não	Sim	Sim	Sim
Execução no navegador	Sim	Não	Parcial	Não
Código-fonte acessível	Sim	Parcial	Sim	Não
Infraestrutura de servidor	Não	Não	Sim	Sim

A principal limitação da abordagem *client-side* reside na capacidade computacional: problemas com malhas muito refinadas (dezenas de milhares de nós) podem apresentar tempo de execução elevado, uma vez que o desempenho do WebAssembly, embora próximo ao nativo, ainda é inferior ao de implementações compiladas em C/Fortran. Para os casos de interesse deste trabalho, envolvendo malhas de centenas a poucos milhares de nós, essa limitação não se mostrou restritiva. Uma análise

quantitativa desse custo é apresentada na Seção 5.2.4, e a Seção 5.6 sistematiza as limitações da plataforma.

5.2.4 Análise Computacional

Para quantificar o custo computacional da execução em ambiente *client-side* e o impacto da camada *WebAssembly*, foi conduzido um benchmark cruzado entre dois ambientes: *Pyodide* no navegador (mesmo cenário em que a plataforma opera para o usuário final) e *CPython* nativo, ambos rodando na mesma máquina (Apple M1 Max, 64 GB de RAM, macOS), com idênticas versões de NumPy e SciPy. Foram instrumentados dois solvers MEF representativos do espectro de complexidade da plataforma: a condução permanente 1D (sistema denso resolvido por `np.linalg.solve`) e a transferência de calor 2D transiente (sistema esparsa CSR com 50 passos de `spsolve`). As medições reportam mediana e MAD (*median absolute deviation*) de cinco rodadas (três para os tamanhos maiores), descartando uma rodada de *warm-up*. As fases de montagem da matriz e resolução do sistema são cronometradas separadamente, com toda a etapa de pós-processamento (geração de gráficos, escrita no DOM) excluída da medição. As listagens completas dos scripts de benchmark constam no Apêndice C.

Tabela 5.2: Tempo total do solver (mediana, com MAD reportado abaixo de 1% em todas as células) em CPython nativo e Pyodide (navegador), na mesma máquina (Apple M1 Max). t_{mont} é o tempo de montagem (incluindo geração de malha quando aplicável) e t_{slv} é o tempo da resolução linear (`np.linalg.solve` no caso 1D denso, ou o total dos passos de `spsolve` nos casos 2D esparsos). “CP” = CPython nativo; “Py” = Pyodide.

Caso	N	t_{mont} CP	t_{slv} CP	t_{tot} CP	t_{tot} Py	Py/CP
Cond. 1D MEF	101	0,15 ms	0,06 ms	0,21 ms	1,6 ms	7,1×
Cond. 1D MEF	1 001	1,56 ms	9,89 ms	11,5 ms	16,0 ms	1,4×
Cond. 1D MEF	10 001	23,8 ms	4 887 ms	4 911 ms	1 394 ms	0,3×
Transcalor 2D MEF	400	56,8 ms	25,1 ms	81,9 ms	244 ms	3,0×
Transcalor 2D MEF	1 600	236 ms	143 ms	379 ms	1 138 ms	3,0×
Transcalor 2D MEF	6 400	994 ms	849 ms	1 843 ms	5 814 ms	3,2×

A Figura 5.2 ilustra a escalabilidade dos dois solvers em escala logarítmica. No caso 1D, o crescimento aproximadamente cúbico do tempo de `solve` (de 0,06 ms para 4,89 s ao multiplicar N por cem) reflete a complexidade $\mathcal{O}(N^3)$ do solver direto denso adotado, que se torna proibitivo já em torno de $N = 10\,000$ nós — precisamente o regime em que se justificaria a migração para um solver iterativo esparso. No caso transcalor 2D, a inclinação observada é substancialmente mais branda: o tempo total cresce aproximadamente com $N^{1,2}$, comportamento típico de fatorações esparsas em malhas estruturais 2D, e mantém-se em poucos segundos mesmo com $N \approx 6\,400$ nós e 50 passos temporais. Os dois casos cobrem, portanto, os dois caminhos de código distintos da plataforma: solver denso em problemas 1D e solver esparso com avanço temporal nos problemas 2D. A simulação de Navier-Stokes 2D, embora mais pesada por exigir remontagem da matriz de convecção a cada passo, segue qualitativamente o mesmo padrão da transcalor; a validação do caso de cavidade contra Ghia *et al.* [24] apresentada no Capítulo 4 é executada na própria plataforma com tempos compatíveis com uso interativo (poucos minutos no navegador para malha 41×41).

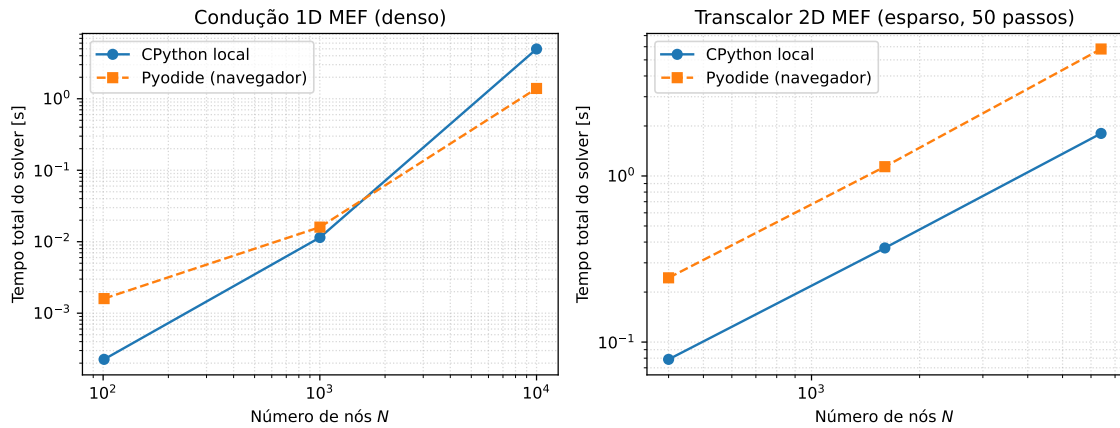


Figura 5.2: Escalabilidade do tempo total do solver com o número de nós, em escala log-log, comparando CPython nativo (linha sólida) e Pyodide no navegador (linha tracejada). À esquerda, condução 1D MEF (denso) com inclinação compatível com $\mathcal{O}(N^3)$; à direita, transcalor 2D MEF (esparso, 50 passos) com inclinação próxima de $\mathcal{O}(N^{1,2})$.

A leitura da Tabela 5.2 revela três regimes. Quando o tempo total é dominado por loops Python sobre elementos — caso da transcalor 2D em todos os

tamanhos — a razão Pyodide/CPython mantém-se aproximadamente constante em torno de 3,0–3,2×, refletindo o fato de que o *WebAssembly* executa o interpretador CPython compilado para Wasm com penalidade modesta, mas *não* acelera o *bytecode* interpretado em si. Em problemas muito pequenos — como a condução 1D com $N = 101$ — a razão sobe para cerca de 7×, refletindo o sobrecusto fixo de inicialização das chamadas Python que se torna dominante quando o tempo absoluto é da ordem de milissegundos. Já em problemas grandes onde o tempo migra para rotinas BLAS/LAPACK compiladas — como a condução 1D em $N = 10\,001$, em que a fase de `np.linalg.solve` consome aproximadamente 99% do tempo total — observou-se uma inversão: o Pyodide foi cerca de 3,5× mais rápido que o CPython nativo da máquina de teste. A explicação plausível é que a versão de NumPy instalada localmente está utilizando uma combinação BLAS subótima para esta carga específica, enquanto o Pyodide entrega uma build de OpenBLAS portada para Wasm sintonizada para precisão dupla densa. Em síntese, a penalidade típica de execução no navegador é um pequeno fator multiplicativo (entre 1,4 e 3,2× nos casos representativos da plataforma), e *não uma ordem de magnitude* — ainda compatível, portanto, com a interatividade esperada em uso didático.

5.3 Arquitetura de Software

5.3.1 Organização de Diretórios

O repositório do *MinervaMesh* segue uma estrutura modular, onde cada simulação é autocontida em seu próprio diretório. A organização geral é apresentada a seguir:

```
minervamesh/  
|-- index.html           # Menu principal  
|-- assets/             # Logos e imagens globais  
|-- simulations/  
|   |-- mef/            # Elementos Finitos  
|   |   |-- <nome-do-modulo>/  
|   |   |   |-- index.html    # Interface do usuario  
|   |   |   |-- simulation.py  # Logica numerica  
|   |   |   +-- config.toml    # Dependencias Python
```



```

| | |-- ... # (8 módulos MEF)
| | +-- escoamento-fluido-2d/
| | |-- common.py # Matrizes MEF 2D
| | +-- geometry.py # Geometrias de obstaculo
| |-- mdf/ # Diferencas Finitas
| +-- extra/ # Ferramentas auxiliares

```

Cada módulo de simulação segue um padrão uniforme de três arquivos:

- `index.html`: interface do usuário, contendo o formulário de parâmetros, o contêiner de resultados e a carga do PyScript;
- `simulation.py`: implementação da lógica numérica em Python, incluindo montagem de matrizes, solução do sistema e geração de gráficos;
- `config.toml`: declaração das dependências Python necessárias (por exemplo, `numpy`, `scipy`, `matplotlib`).

5.3.2 Fluxo de Execução de uma Simulação

O ciclo de vida de uma simulação no *MinervaMesh* pode ser descrito em cinco etapas:

1. **Carregamento:** o navegador requisita o `index.html` de um servidor HTTP simples. A tag `<script type="py">` instrui o PyScript a inicializar o Pyodide e carregar as dependências declaradas em `config.toml`.
2. **Parametrização:** o usuário configura os parâmetros físicos e numéricos através de campos de entrada na interface.
3. **Execução:** ao clicar em “Rodar”, o código Python é executado pelo Pyodide. As matrizes do MEF são montadas, o sistema linear é resolvido e os gráficos são gerados — tudo no navegador.
4. **Visualização:** os gráficos resultantes são convertidos para formato de imagem codificado em Base64 e inseridos dinamicamente no DOM da página.
5. **Inspeção:** o botão “Ver Código” exhibe o código-fonte Python da simulação com *syntax highlighting*, permitindo ao aluno estudar a implementação.

A Figura 5.3 ilustra a etapa de parametrização em um módulo representativo (Condução Permanente 1D), com os campos de entrada preenchidos e o resultado ainda não renderizado.

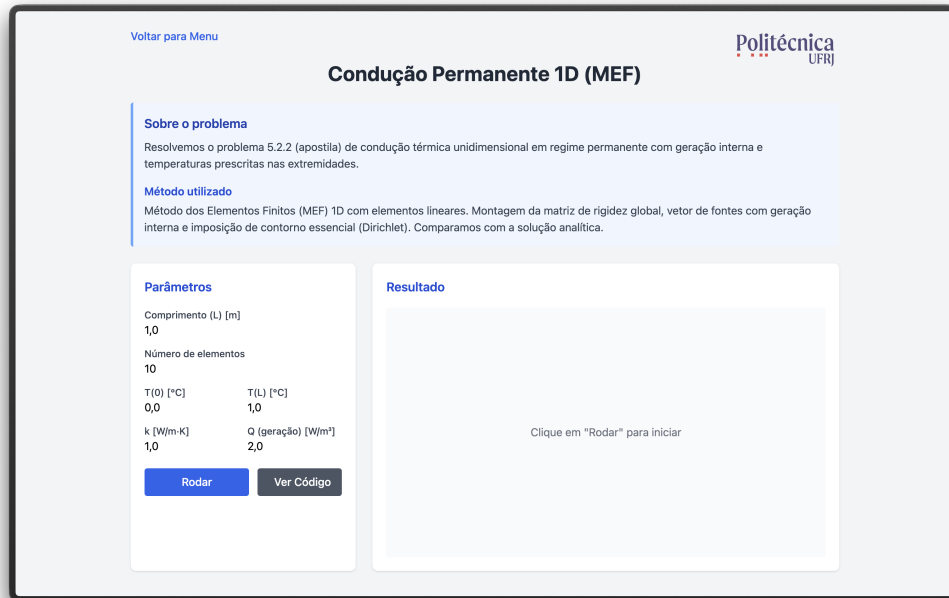


Figura 5.3: Interface de parametrização de uma simulação (Condução Permanente 1D) antes da execução.

5.4 Módulos de Simulação MEF

Esta seção descreve a implementação de cada módulo de simulação do *MinervaMesh*, com ênfase nas decisões de código e na correspondência entre operadores matemáticos e numéricos. A formulação detalhada e a validação de cada caso — equação governante, condições de contorno, solução de referência e análise de erro — já constam no Capítulo 4; aqui o foco recai sobre *como* cada método é codificado e executado no navegador. Os trechos de código apresentados referem-se às funções centrais de cada simulação; as listagens completas encontram-se nos Apêndices.

Todos os módulos implementam diretamente as formulações variacionais e discretizações desenvolvidas no Capítulo 3. As matrizes elementares de rigidez, massa e carga são construídas a partir das expressões derivadas nas Seções 3.4.5 e 3.5.4, e cada trecho de código apresentado nesta seção referencia explicitamente a equação

teórica correspondente, mantendo correspondência direta entre operadores matemáticos e operadores numéricos.

5.4.1 Condução Permanente 1D

O primeiro módulo resolve o problema de condução térmica unidimensional em regime permanente com geração interna de calor, validado na Seção 4.2. A discretização por elementos finitos lineares resulta na matriz de rigidez e no vetor de carga elementares (Seção 3.4.5), implementados como:

```
1 # Elemento linear 1D: ke e be
2 ke = (k / h) * np.array([[1.0, -1.0], [-1.0, 1.0]])
3 be = (Q * h / 2.0) * np.array([1.0, 1.0])
```

A montagem percorre os N_{el} elementos, acumulando as contribuições nas posições globais. Após a imposição das condições de Dirichlet (zerando linhas e colunas correspondentes), o sistema $\mathbf{KT} = \mathbf{b}$ é resolvido por `numpy.linalg.solve`. A Figura 5.4 mostra a comparação entre a solução MEF e a solução analítica.

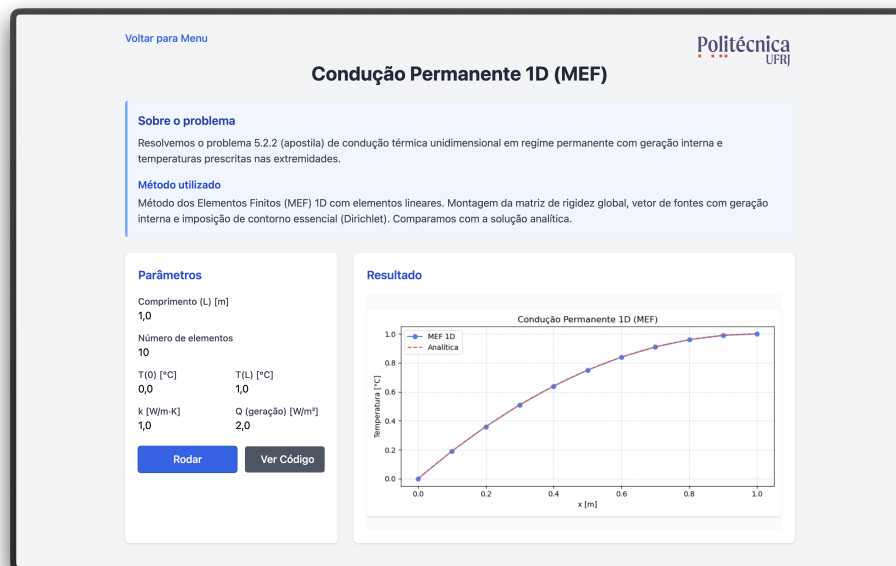


Figura 5.4: Condução permanente 1D: solução MEF ($N_{el} = 10$) comparada à solução analítica.

5.4.2 Condução Transiente 1D com $k(x)$ Variável

Este módulo estende o problema anterior para o regime transiente com condutividade térmica variável $k(x)$, incorporando a matriz de massa e a integração temporal conforme o sistema semidiscreto estabelecido na Seção 3.6. A formulação e a validação completas constam na Seção 4.3. As matrizes elementares são calculadas por:

```
1 # Matrizes de elemento 1D transiente
2 ke = (k_avg / h) * np.array([[1, -1], [-1, 1]])
3 me = (rho_cp * h / 6) * np.array([[2, 1], [1, 2]])
```

A integração temporal utiliza o método θ generalizado (Eq. 3.37), seguindo rigorosamente o esquema de avanço temporal derivado na Seção 3.6, e permitindo ao usuário selecionar entre Euler explícito, Crank-Nicolson e Euler implícito. A Figura 5.5 ilustra a evolução temporal do campo de temperatura.

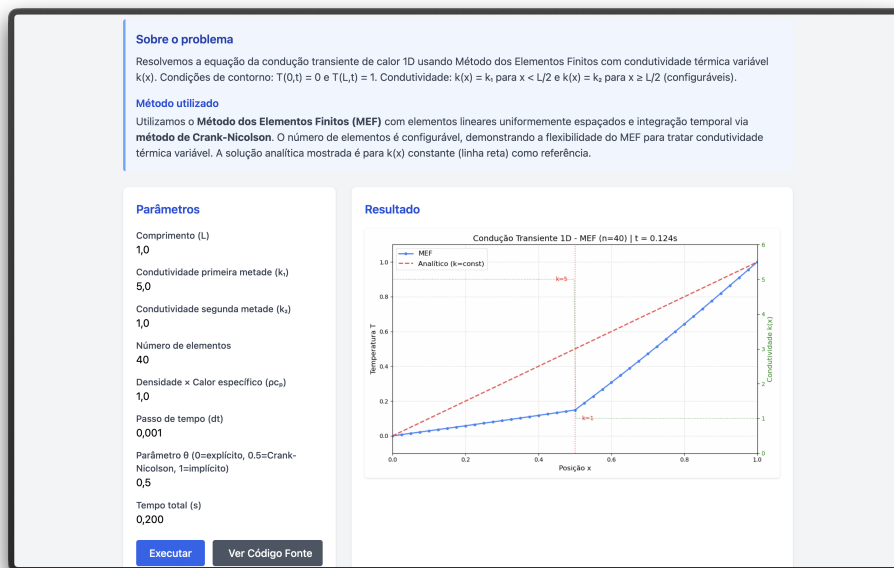


Figura 5.5: Evolução temporal do campo de temperatura com condutividade variável $k(x)$.

5.4.3 Convecção-Difusão 1D com Estabilização SUPG

O módulo de convecção-difusão 1D resolve a equação de advecção-difusão estacionária aplicando a técnica de estabilização SUPG (formulação desenvolvida

na Seção 3.5.4; caso validado na Seção 4.4). A discretização MEF gera três matrizes elementares: difusão, convecção e estabilização SUPG. O parâmetro τ_{SUPG} é calculado segundo a expressão analítica da Eq. 3.35, implementado como:

```
1 # Numero de Peclet local e tau SUPG
2 Pe_local = (rho_cp * u * h) / (2 * k)
3 tau = h / (2*abs(u)) * (1/np.tanh(abs(Pe_local))
4                               - 1/abs(Pe_local))
```

A Figura 5.6 demonstra o efeito da estabilização SUPG em um problema com número de Péclet elevado. Sem estabilização, a formulação de Galerkin padrão produz oscilações espúrias; com SUPG, a solução numérica converge suavemente para a solução analítica.

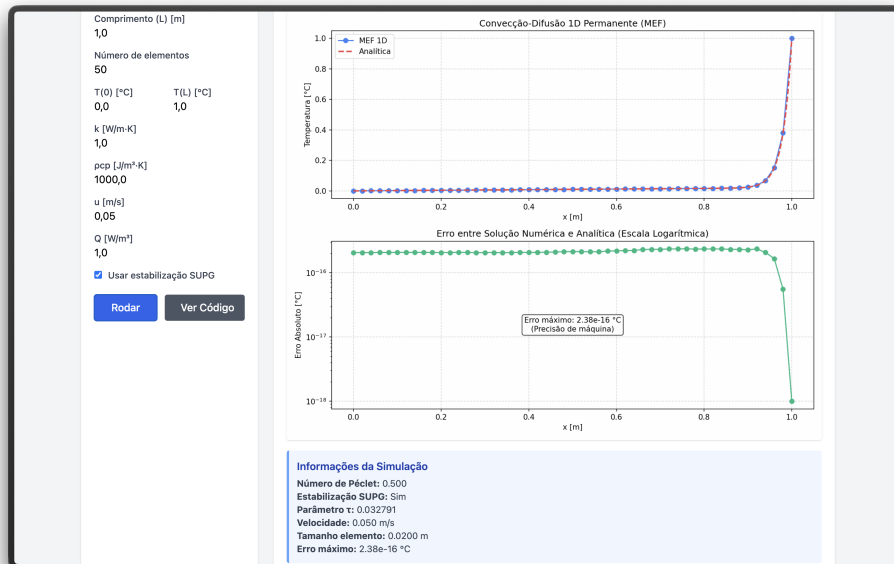


Figura 5.6: Convecção-difusão 1D com estabilização SUPG ($Pe \gg 1$): solução MEF comparada à analítica.

5.4.4 Viga 1D — Análise de Flexão (Euler-Bernoulli)

O módulo de viga 1D implementa elementos finitos de Euler-Bernoulli com dois graus de liberdade por nó: deslocamento vertical w e rotação θ , seguindo o procedimento de Galerkin apresentado na Seção 3.4.3 com funções de interpolação cúbicas de Hermite (caso validado na Seção 4.5). A matriz de rigidez elementar 4×4

é construída com essas funções:

```
1  # Matriz de rigidez do elemento de viga
2  ke = (EI / h**3) * np.array([
3      [12,    6*h,   -12,    6*h  ],
4      [6*h,   4*h**2, -6*h,   2*h**2],
5      [-12,  -6*h,    12,   -6*h  ],
6      [6*h,   2*h**2, -6*h,   4*h**2]
7  ])
```

O módulo suporta quatro condições de contorno (simplesmente apoiada, balanço, engastada-engastada e engastada-apoiada) e cinco tipos de carregamento (uniforme, linear, triangular, senoidal e pontual), além de propriedades materiais variáveis ao longo do comprimento. A Figura 5.7 apresenta a análise completa de uma viga, incluindo deflexão, diagramas de momento fletor e esforço cortante.

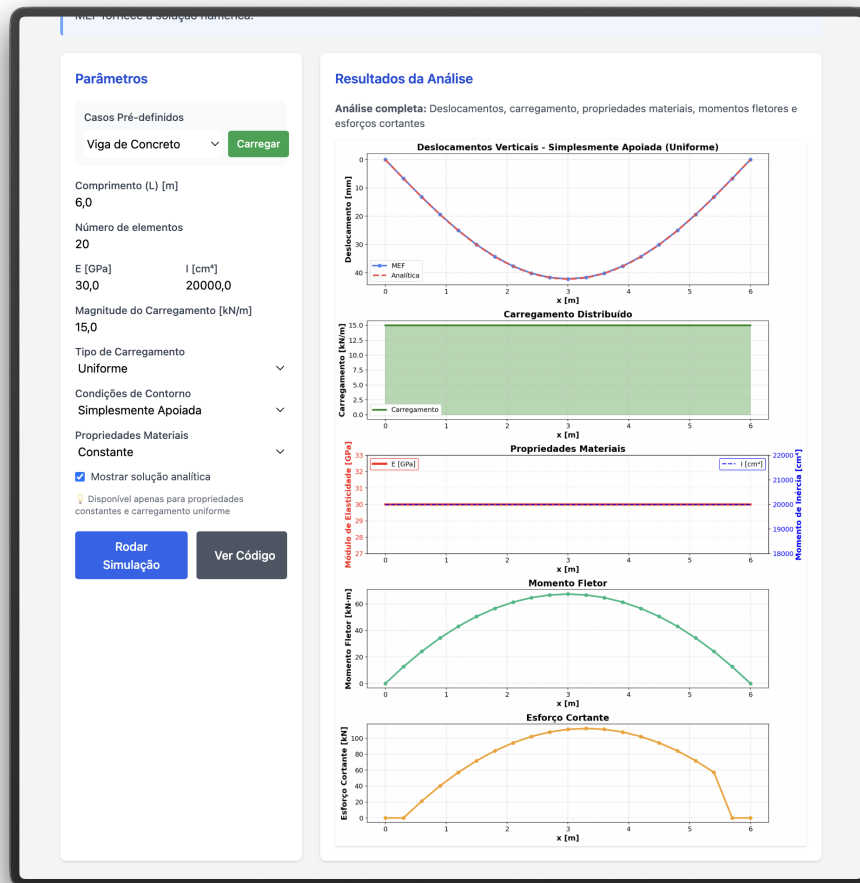


Figura 5.7: Análise de flexão de viga 1D (Euler-Bernoulli): deflexão, carregamento, propriedades, momento fletor e esforço cortante.

5.4.5 Vibração 1D — Problema de Autovalor

O módulo de vibração resolve o problema de autovalor generalizado $\mathbf{K}\phi = \lambda \mathbf{M}\phi$ para uma barra elástica 1D (caso validado na Seção 4.6), onde as frequências naturais são $f_n = \sqrt{\lambda_n}/(2\pi)$ e os autovetores ϕ_n representam os modos de vibração. As matrizes de rigidez e massa são construídas conforme o procedimento de montagem global descrito na Seção 3.4.6, empregando massa concentrada (*lumped*, Eq. 3.27):

```
1 # Matrizes de rigidez e massa (barra 1D)
2 ke = (E*A / h) * np.array([[1, -1], [-1, 1]])
3 me = (rho*A*h / 2) * np.array([[1, 0], [0, 1]])
```

O problema de autovalor é resolvido pela rotina `scipy.linalg.eigh`, que explora a simetria das matrizes para eficiência e estabilidade numérica. A solução fornece n frequências naturais e os correspondentes modos de vibração, comparáveis com as expressões analíticas clássicas — por exemplo, $f_n = nc/(2L)$ para barras engastadas em ambas as extremidades, onde $c = \sqrt{E/\rho}$ é a velocidade de propagação de ondas. A Figura 5.8 ilustra os cinco primeiros modos.



Figura 5.8: Frequências naturais e cinco primeiros modos de vibração de uma barra 1D.

5.4.6 Transferência de Calor 2D

O módulo de transferência de calor bidimensional resolve a equação de condução transiente em uma placa quadrada discretizada com elementos triangulares lineares (Seção 3.4.4; caso validado na Seção 4.7). As matrizes elementares de rigidez e massa são construídas pela função compartilhada `element_matrices`, que implementa diretamente as expressões derivadas na Seção 3.4.5 (Eqs. 3.25 e 3.26):

```
1 def element_matrices(coords, k, rho, cv):
```



```

2     x0, x1, x2 = coords
3     area = 0.5 * abs((x1[0]-x0[0])*(x2[1]-x0[1])
4                   -(x2[0]-x0[0])*(x1[1]-x0[1]))
5     b = np.array([x1[1]-x2[1], x2[1]-x0[1], x0[1]-x1[1]])
6     c = np.array([x2[0]-x1[0], x0[0]-x2[0], x1[0]-x0[0]])
7     ke = (k/(4*area)) * (np.outer(b,b) + np.outer(c,c))
8     me = (rho*cv*area/12) * (np.ones((3,3)) + np.eye(3))
9     return ke, me

```

A integração temporal emprega o método de Crank-Nicolson ($\theta = 1/2$), conforme a formulação do método θ generalizado apresentada na Seção 3.6 (Eq. 3.37), formulado com matrizes esparsas (`scipy.sparse`) para viabilizar a execução no navegador com malhas de até 40×40 nós. A cada passo de tempo, um frame do campo de temperatura é gerado e acumulado para compor uma animação GIF. A Figura 5.9 ilustra um instante da evolução temporal.

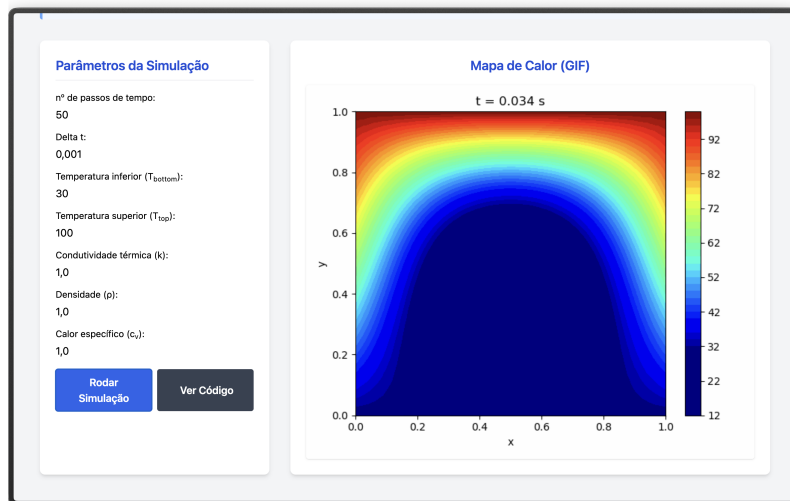


Figura 5.9: Campo de temperatura em placa 2D durante a evolução transiente (método de Crank-Nicolson).

5.4.7 Escoamento Potencial 2D

O módulo de escoamento potencial resolve a equação de Laplace $\nabla^2 \psi = 0$ em domínios 2D com obstáculos, caso particular da equação de Poisson para a função corrente (Eq. 3.32) com vorticidade nula (caso validado na Seção 4.8). A malha triangular é gerada pelo módulo `common.py`, que também assembla as

matrizes globais de rigidez e massa conforme o procedimento descrito na Seção 3.4.6.

O módulo `geometry.py` fornece diferentes geometrias de obstáculo (cilindro, retângulo, degrau), sendo os nós do contorno do obstáculo inseridos na malha com a condição $\psi = \text{constante}$. As velocidades são recuperadas por:

$$u = \frac{\partial \psi}{\partial y} \approx \frac{1}{2A^e} \sum_j \psi_j c_j, \quad v = -\frac{\partial \psi}{\partial x} \approx -\frac{1}{2A^e} \sum_j \psi_j b_j \quad (5.1)$$

A Figura 5.10 apresenta as linhas de corrente para escoamento ao redor de um obstáculo.

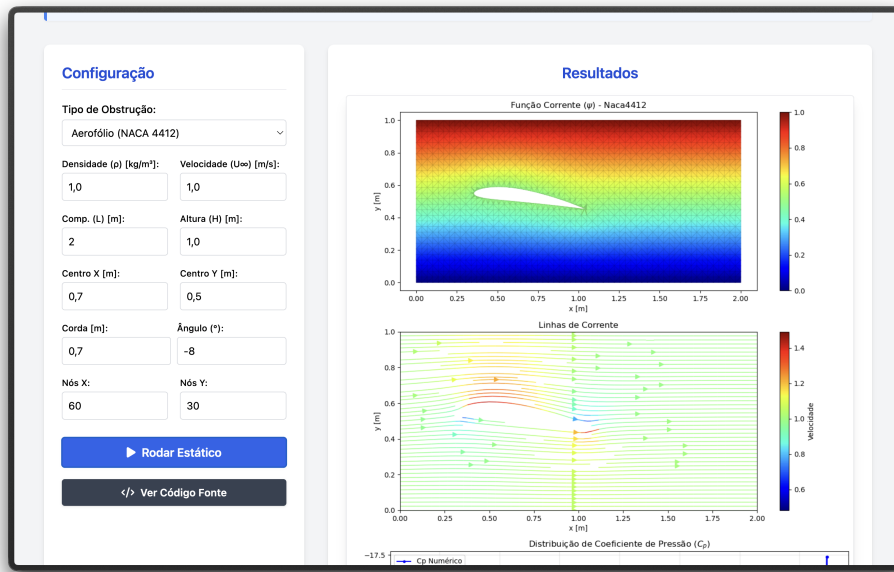


Figura 5.10: Escoamento potencial 2D: linhas de corrente ao redor de um obstáculo.

5.4.8 Navier-Stokes 2D (Vorticidade-Corrente)

O módulo mais complexo da plataforma implementa o solver de Navier-Stokes 2D na formulação Vorticidade-Corrente (ω - ψ), conforme descrito na Seção 3.5 (caso validado na Seção 4.9). A discretização temporal segue o esquema semi-implícito derivado na Seção 3.6.2 (Eq. 3.38), enquanto as condições de contorno de vorticidade nas paredes são impostas conforme a Eq. 3.39. O algoritmo segue o procedimento iterativo:

1. Resolver a equação de Poisson $\mathbf{K}\psi = \mathbf{M}\omega$ para ψ ;
2. Atualizar a vorticidade nas paredes sólidas conforme a Eq. 3.39;

3. Calcular (u, v) a partir de ψ (Eq. 5.1);
4. Assemblar a matriz de convecção $\mathbf{C}^n$ com o campo de velocidades atual;
5. Resolver o transporte da vorticidade pelo esquema semi-implícito (Eq. 3.38).

O solver suporta dois cenários: escoamento em canal com obstáculos (cilindro, retângulo, degrau) e cavidade com tampa móvel (*lid-driven cavity*). No caso da cavidade, todas as paredes possuem $\psi = 0$ e a parede superior impõe velocidade tangencial $u_{\text{lid}} = 1$, acrescentando o termo $-2u_{\text{tan}}/h$ à condição de contorno da vorticidade. A Figura 5.11 mostra os campos de ψ , ω e as linhas de corrente para a cavidade a $\text{Re} = 100$.

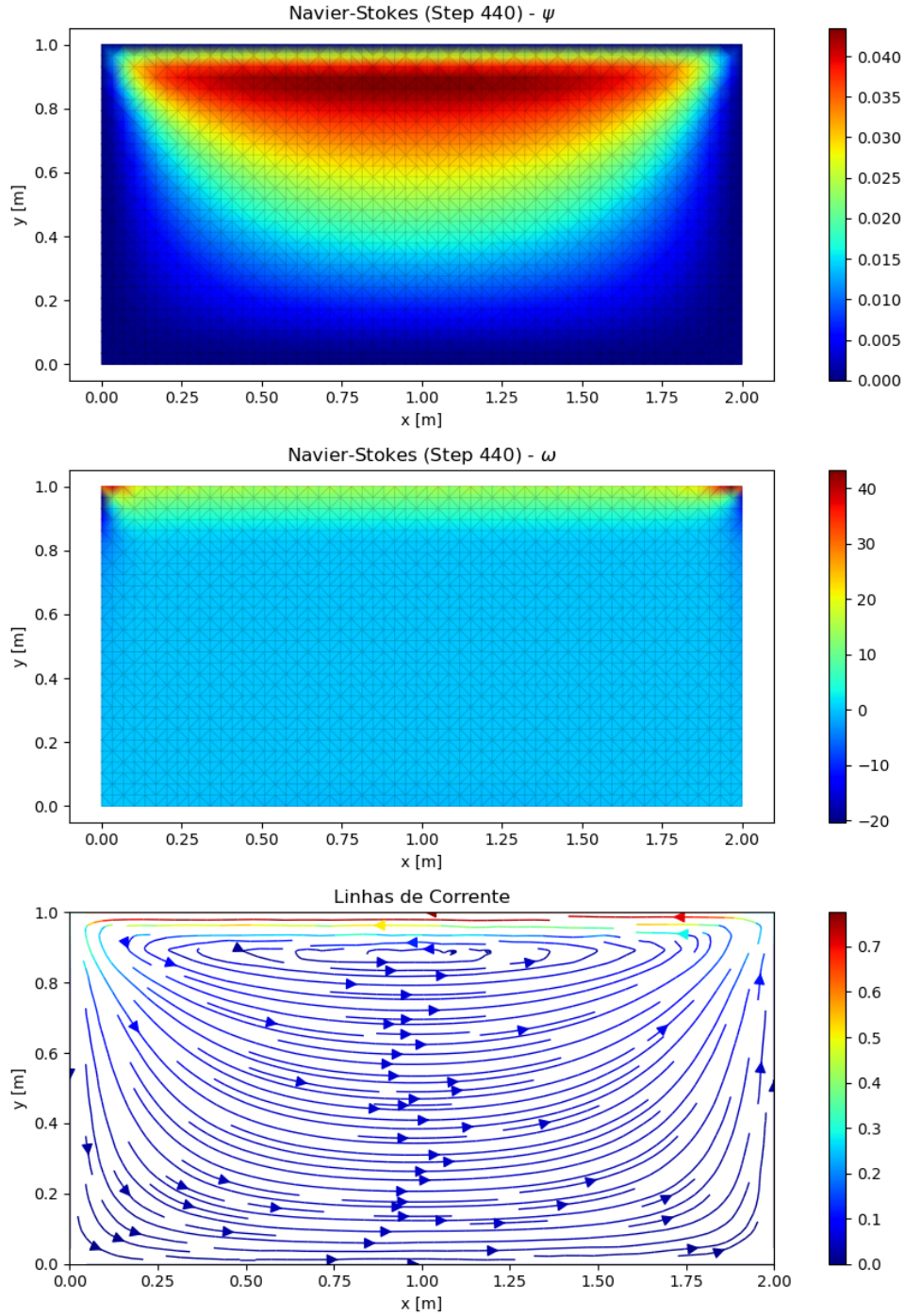


Figura 5.11: Navier-Stokes 2D — cavidade com tampa móvel ($Re = 100$): campos de ψ , ω e linhas de corrente.

5.5 Módulos Complementares em MDF

Além dos oito módulos MEF descritos na seção anterior, a plataforma disponibiliza dois módulos implementados pelo Método das Diferenças Finitas, desenvolvidos nas

etapas iniciais do projeto e mantidos no portfólio por seu valor didático-comparativo:

- **Condução Permanente 1D com Geração Interna:** resolve o mesmo problema térmico da Seção 5.4.1, substituindo a discretização variacional pela aproximação de derivadas via expansão de Taylor (Seção 3.3), o que permite contrastar as duas formulações para um fenômeno físico idêntico.
- **Equação da Onda 1D:** problema hiperbólico linear com avanço temporal explícito, incluído como exercício conceitual de integração temporal em malha estruturada.

Ambos os módulos reproduzem suas respectivas soluções analíticas com erro numérico compatível com a precisão de máquina. Como constituem implementações iniciais de caráter preparatório e o escopo central do trabalho recai sobre o MEF, a validação quantitativa apresentada no Capítulo 4 concentra-se exclusivamente nos módulos de elementos finitos.

5.6 Limitações da Plataforma

A análise computacional apresentada na Seção 5.2.4 torna explícitas as restrições impostas pela arquitetura *client-side*. Esta seção sistematiza tais restrições, distinguindo-as em dois grupos: limitações *intrínsecas* ao modelo de execução em *WebAssembly* (que motivariam, em uma plataforma de produção em larga escala, uma migração para arquitetura híbrida) e limitações *contornáveis*, mitigáveis dentro do próprio paradigma *client-side* via extensões já viáveis tecnicamente.

Limitações intrínsecas.

- **Memória disponível:** o processo *Pyodide* executa em *sandbox* com *heap WebAssembly* tipicamente limitado a poucos gigabytes (4 GB no padrão Wasm32, ampliável apenas via Wasm64 com suporte parcial nos navegadores), e a pressão prática de coleta de lixo torna-se relevante a partir de algumas centenas de MB.

- **Escalabilidade do problema:** a combinação entre o limite de memória da *sandbox* e a complexidade dos solvers diretos adotados estabelece um teto efetivo de tamanho de problema. Considerando o benchmark da Seção 5.2.4, este teto é da ordem de 10^4 nós para formulações com matriz densa (acima desse valor, o tempo de `np.linalg.solve` ultrapassa cinco segundos e o consumo de memória aproxima-se da fronteira da *sandbox*) e da ordem de 10^5 nós para formulações esparsas com `spsolve`. A extensão para problemas tridimensionais com malhas refinadas, comum em engenharia profissional, encontra-se fora desta faixa.
- **Execução em *thread* única:** o *Pyodide* ocupa apenas a *thread* principal de JavaScript, o que impede o uso direto de `multiprocessing`, `joblib` ou de bibliotecas científicas em modo *multithread*. Operações longas bloqueiam a renderização da página até concluírem, e o ganho potencial de paralelismo de domínio (decomposição da malha entre múltiplos núcleos) não é acessível dentro do paradigma atual.
- **Latência de inicialização (*cold start*):** o carregamento do *runtime WebAssembly* e dos pacotes científicos (`numpy`, `scipy`, `matplotlib`) demanda tipicamente entre cinco e quinze segundos no primeiro acesso, dependendo da banda do usuário. Esse custo é amortizado pelo cache do navegador em visitas subsequentes, mas não é eliminado.
- **Geração de malha:** esta é, na prática, a mais sensível das limitações da plataforma para fins de engenharia. O `Gmsh` — gerador de malhas não estruturadas adotado como referência *de facto* em pré-processamento de MEF — não está disponível no *Pyodide*, pois depende de extensões em C++ ainda não portadas para *WebAssembly*. O mesmo se aplica a `Triangle`, `TetGen`, `MeshPy` e demais ferramentas consagradas. Como substituto, o *MinervaMesh* utiliza a triangulação de Delaunay nativa do `matplotlib.tri`, o que impõe três restrições importantes: (i) a malha resultante é sempre 2D — não há geração de malha tetraédrica 3D nativamente disponível; (ii) o algoritmo de Delaunay opera sobre um conjunto de pontos pré-determinado, sem refinamento adaptativo a posteriori, sem refinamento dirigido por gradiente da solução, e

sem geração de *boundary layer mesh* (camada limite refinada próxima a paredes), recursos centrais em escoamentos de alto Reynolds; (iii) geometrias com obstáculos ou contornos curvilíneos exigem que os pontos da fronteira sejam definidos manualmente em coordenadas cartesianas, como ilustram os módulos de escoamento 2D do MinervaMesh, em que cilindros e degraus são parametrizados ponto a ponto. Esta restrição limita o repertório prático da plataforma a geometrias geometricamente simples.

- **Suporte parcial à pilha científica:** de modo análogo ao *Gmsh*, outros pacotes que dependem de extensões nativas ainda não portadas para *WebAssembly* — como *petsc4py* (solvers iterativos de larga escala) e diversos pacotes de aprendizado de máquina — não estão disponíveis no *Pyodide*, embora a cobertura da pilha numérica fundamental (NumPy, SciPy, Matplotlib) seja completa.

Limitações contornáveis.

- **Bloqueio da *thread* principal:** mitigável por meio de *Web Workers*, que permitem mover a execução do *Pyodide* para uma *thread* paralela ao DOM, preservando a responsividade da interface durante simulações longas. A reorganização exigida é localizada à camada de orquestração.
- **Solvers diretos esparsos:** o uso atual de *spsolve* no caso transiente 2D refatora a matriz **A** a cada chamada e, embora robusto, é mais custoso do que o necessário para sistemas simétricos definidos positivos. A substituição por solvers iterativos (*scipy.sparse.linalg.cg*, *gmres*) com pré-condicionamento adequado — ou o reuso explícito da fatoração via *splu* — viabilizaria malhas substancialmente maiores dentro da mesma fronteira de memória.
- **Geração de malha avançada:** a integração com geradores externos consagrados é viável à medida que esses geradores recebam *ports* para *WebAssembly*. O projeto *triangle-wasm* [46] comprova essa viabilidade ao distribuir o gerador *Triangle*, de J. Shewchuk, compilado via *Emscripten* e executável diretamente no navegador — amplia substancialmente o repertório 2D em relação

à triangulação simples do `matplotlib.tri`, ao oferecer triangulação de Delaunay com restrições de contorno e refinamento controlado por qualidade. A portabilização do `Gmsh` para *WebAssembly*, ainda inexistente em forma oficial mas tecnicamente factível pelo mesmo caminho, eliminaria simultaneamente as três restrições atuais de pré-processamento (geometrias complexas, refinamento adaptativo e extensão tridimensional), preservando integralmente o paradigma *client-side*.

- **Eficiência da montagem:** os *loops* Python sobre elementos — predominantes na fase de montagem dos casos 2D — constituem o gargalo mais sensível à execução em Wasm, conforme indica a Tabela 5.2. Vetorização adicional via operações `numpy.add.at` ou refatoração para `Cython/Numba` (não disponíveis em Wasm no momento) reduziriam parte significativa dessa diferença.

Para os módulos didáticos e os tamanhos de malha tipicamente empregados em ensino de Engenharia Mecânica (centenas a poucos milhares de nós), nenhuma dessas limitações se mostrou restritiva ao longo das validações apresentadas no Capítulo 4. As restrições enumeradas acima são, portanto, fronteiras de ampliação futura, e não barreiras à utilização atual.

5.7 Síntese do Capítulo

Do ponto de vista da integração teórico-computacional, todos os módulos descritos neste capítulo compartilham o mesmo núcleo numérico: construção de matrizes elementares, montagem global por conectividade nodal, solução de sistemas lineares e, nos casos transientes, avanço temporal via método θ ou esquema semi-implícito. Dessa forma, a plataforma *MinervaMesh* operacionaliza integralmente o arcabouço teórico estabelecido no Capítulo 3, mantendo correspondência direta entre as equações governantes contínuas, os sistemas discretizados obtidos pela formulação de Galerkin e os *solvers* efetivamente implementados. As listagens completas dos códigos, apresentadas nos Apêndices A e B, documentam essa correspondência em nível de implementação linha a linha, e os *scripts* de *benchmark* computacional cruzado entre CPython e Pyodide constam no Apêndice C.

Este capítulo apresentou a arquitetura e a implementação do *MinervaMesh*. A escolha do PyScript e da computação *client-side* viabiliza uma plataforma de simulação numérica com custo de infraestrutura zero, portabilidade total e código aberto para inspeção acadêmica. A organização modular do repositório — com cada simulação autocontida em três arquivos (HTML, Python, TOML) — permite a extensão da plataforma com novos casos sem impacto nos módulos existentes. Os oito módulos MEF descritos cobrem três domínios da Engenharia Mecânica (térmico, estrutural e fluidodinâmico), demonstrando a versatilidade da formulação de elementos finitos apresentada no Capítulo 3. A validação quantitativa desses módulos — comparação com soluções analíticas e *benchmarks* da literatura — foi apresentada no Capítulo 4, confirmando a corretude das discretizações que a arquitetura aqui descrita disponibiliza na web.

Capítulo 6

Conclusões

6.1 Conclusões

Este trabalho apresentou o desenvolvimento e a validação do *MinervaMesh*, uma plataforma *client-side* para simulação numérica em Engenharia Mecânica, executada integralmente no navegador por meio da combinação de *PyScript*, *Pyodide* e *WebAssembly*. Os objetivos estabelecidos na Introdução foram alcançados: implementaram-se oito módulos de Método dos Elementos Finitos descritos no Capítulo 5, dois módulos complementares de Método das Diferenças Finitas (Seção 5.5) e uma interface interativa de pré/pós-processamento comum a todas as simulações, validada prioritariamente contra soluções analíticas e, para os casos 2D sem solução fechada, contra benchmark consagrado da literatura. Os dois resultados centrais do trabalho confirmam-se: as equações discretizadas pelo MEF e pelo MDF **reproduzem corretamente as soluções de referência** em todos os casos verificados, e a **implementação na web flexibiliza a resolução desses problemas para outras pessoas** — estudantes, monitores e pesquisadores —, que podem parametrizar, executar e inspecionar cada simulação diretamente no navegador, sem instalação nem custo de licenciamento.

Os resultados de validação apresentados no Capítulo 4 demonstram que a plataforma preserva a ordem de convergência esperada para cada discretização: erros nodais da ordem da precisão de máquina ($\sim 10^{-13}$) para problemas com solução polinomial — condução 1D e viga de Euler-Bernoulli — como consequência da super-

convergência de elementos lineares e da reprodução polinomial dos elementos cúbicos de Hermite; concordância visual com soluções analíticas em problemas parabólicos e hiperbólicos; e, para o escoamento em cavidade com tampa móvel a $Re = 100$, erro relativo inferior a 0,4% na vorticidade mínima comparada à referência de Ghia et al. [24], utilizando apenas malha 41×41 . A estabilização SUPG reproduz quantitativamente o parâmetro τ canônico da formulação de Brooks e Hughes [5], e a formulação Vorticidade-Corrente com a condição de contorno de Thom [45] na parede reproduz o vórtice primário da cavidade dentro da resolução da malha utilizada.

A contribuição didática do trabalho articula-se em três eixos: (i) **acessibilidade** — a plataforma é gratuita, opera sem instalação e pode ser hospedada em serviços de páginas estáticas sem custo operacional; (ii) **transparência** — o código-fonte Python de cada simulação é inspecionável pelo usuário diretamente na página, permitindo o uso tanto como ferramenta de simulação quanto como objeto de estudo em métodos numéricos; e (iii) **unificação** — os três domínios clássicos abordados (térmico, fluidodinâmico e estrutural) são cobertos em uma única interface consistente, facilitando comparações pedagógicas entre métodos e formulações para um mesmo fenômeno físico. A esses três eixos didáticos soma-se uma contribuição metodológica: o *benchmark* cruzado entre *Pyodide* no navegador e *CPython* nativo apresentado na Seção 5.2.4 caracteriza quantitativamente o custo da execução *client-side* e oferece um protocolo reproduzível para análises semelhantes em outras plataformas didáticas baseadas em *PyScript/Pyodide*.

A principal limitação da arquitetura *client-side* é a capacidade computacional: embora o *WebAssembly* opere em velocidade próxima à nativa para as rotinas de *NumPy* e *SciPy* utilizadas, malhas com dezenas de milhares de nós impõem tempos de resposta incompatíveis com a interatividade esperada em ambiente didático. O benchmark cruzado entre *Pyodide* e *CPython* nativo apresentado na Seção 5.2.4 confirma quantitativamente essa restrição, e a Seção 5.6 sistematiza as fronteiras práticas da plataforma. Para os casos deste trabalho (centenas a poucos milhares de nós), essa restrição não se mostrou limitante, mas condiciona a estratégia de extensão discutida a seguir.

6.2 Trabalhos Futuros

A partir da base estabelecida, identificam-se sete linhas naturais de continuidade:

- **Análise de convergência quantitativa:** produzir gráficos log-log de erro L^2 em função do refinamento de malha (h) para os casos 2D, validando numericamente a ordem de convergência teórica discutida na fundamentação.
- **Elementos de ordem superior:** estender a biblioteca de funções de forma para elementos quadráticos ($\mathbb{P}_2$) e quadriláteros bilineares ($\mathbb{Q}_1$), permitindo estudos comparativos entre discretizações distintas para o mesmo problema físico.
- **Modelos de turbulência:** incorporar formulações RANS ou de *Large Eddy Simulation* ao solver de Navier-Stokes 2D, ampliando o alcance da plataforma para regimes não laminares.
- **Escalabilidade tridimensional:** investigar a viabilidade de solvers 3D sob a restrição de memória e tempo do *WebAssembly*, possivelmente explorando execução assíncrona via *Web Workers* e solvers iterativos esparsos (cf. Seção 5.6).
- **Validação experimental:** complementar o atual processo de verificação numérica com comparações contra dados experimentais, introduzindo a nomenclatura formal de V&V (Verificação e Validação) e a análise de incerteza na discussão metodológica.
- **Integração com geradores de malha externos:** ampliar o repertório de geometrias utilizáveis incorporando à plataforma o `triangle-wasm` [46] — *port WebAssembly* do *Triangle* já disponível e funcional — e, à medida que se viabilize, um eventual *port* do `Gmsh`, atualmente inexistente em forma oficial. Tal integração endereçaria simultaneamente as restrições de geometria complexa, refinamento adaptativo e extensão tridimensional discutidas na Seção 5.6, preservando a característica *client-side* da plataforma.
- **Condições de contorno de Robin:** incorporar a imposição de condições mistas (Seção 3.4.2) ao módulo de transferência de calor 2D, permitindo a representação de troca convectiva $-k \partial T / \partial n = h(T - T_\infty)$ entre o domínio

sólido e um meio externo a temperatura prescrita — extensão natural pedagogicamente relevante para problemas de resfriamento de aletas e dispositivos eletrônicos.

Referências Bibliográficas

- [1] MALISKA, C. R., *Transferência de Calor e Mecânica dos Fluidos Computacional*. 2 ed. Rio de Janeiro, LTC, 2004.
- [2] ANJOS, G. R., *Computação Científica para Engenheiros*. Rio de Janeiro, PEM/COPPE/UFRJ, 2024. Notas de Aula, versão de 23 de maio de 2024.
- [3] PyScript Development Team, “PyScript Documentation”, Disponível em: <https://pyscript.net>, 2023, Acesso em: 20 fev. 2026.
- [4] HARRIS, C. R., MILLMAN, K. J., WALT, S. J. V. D., *et al.*, “Array programming with NumPy”, *Nature*, v. 585, n. 7825, pp. 357–362, 2020.
- [5] BROOKS, A. N., HUGHES, T. J. R., “Streamline upwind/Petrov-Galerkin formulations for convection dominated flows with particular emphasis on the incompressible Navier-Stokes equations”, *Computer Methods in Applied Mechanics and Engineering*, v. 32, n. 1-3, pp. 199–259, 1982.
- [6] FISH, J., BELYTSCHKO, T., *A First Course in Finite Elements*. West Sussex, John Wiley & Sons, 2007.
- [7] INCROPERA, F. P., DEWITT, D. P., BERGMAN, T. L., *et al.*, *Fundamentals of Heat and Mass Transfer*. 6th ed. Hoboken, John Wiley & Sons, 2006.
- [8] CENGEL, Y. A., GHAJAR, A. J., *Heat and Mass Transfer: Fundamentals and Applications*. 5 ed. New York, McGraw-Hill Education, 2015.
- [9] FOX, R. W., MCDONALD, A. T., PRITCHARD, P. J., *Introdução à Mecânica dos Fluidos*. 7th ed. Rio de Janeiro, LTC, 2012.
- [10] WHITE, F. M., *Viscous Fluid Flow*. 3 ed. New York, McGraw-Hill, 2006.

- [11] VERSTEEG, H. K., MALALASEKERA, W., *An Introduction to Computational Fluid Dynamics: The Finite Volume Method*. 2 ed. Harlow, Pearson Education, 2007.
- [12] PATANKAR, S. V., *Numerical Heat Transfer and Fluid Flow*. New York, Hemisphere Publishing Corporation, 1980.
- [13] ZIENKIEWICZ, O. C., TAYLOR, R. L., ZHU, J. Z., *The Finite Element Method: Its Basis and Fundamentals*. 7th ed. Oxford, Butterworth-Heinemann, 2013.
- [14] LEWIS, R. W., NITHIARASU, P., SEETHARAMU, K. N., *Fundamentals of the Finite Element Method for Heat and Fluid Flow*. Chichester, John Wiley & Sons, 2004.
- [15] REDDY, J. N., *An Introduction to the Finite Element Method*. 3 ed. New York, McGraw-Hill, 2005.
- [16] GALERKIN, B. G., “Series in some questions of elastic equilibrium of beams and plates”, *Vestnik Inzhenerov i Tekhnikov*, v. 19, pp. 897–908, 1915.
- [17] COURANT, R., “Variational methods for the solution of problems of equilibrium and vibrations”, *Bulletin of the American Mathematical Society*, v. 49, n. 1, pp. 1–23, 1943.
- [18] HRENNIKOFF, A., “Solution of problems of elasticity by the framework method”, *Journal of Applied Mechanics*, v. 8, n. 4, pp. A169–A175, 1941.
- [19] TURNER, M. J., CLOUGH, R. W., MARTIN, H. C., *et al.*, “Stiffness and deflection analysis of complex structures”, *Journal of the Aeronautical Sciences*, v. 23, n. 9, pp. 805–823, 1956.
- [20] CLOUGH, R. W., “The finite element method in plane stress analysis”. In: *Proceedings of the 2nd ASCE Conference on Electronic Computation*, Pittsburgh, 1960.
- [21] CHRISTIE, I., GRIFFITHS, D. F., MITCHELL, A. R., *et al.*, “Finite element methods for second order differential equations with significant first de-

- rivatives”, *International Journal for Numerical Methods in Engineering*, v. 10, pp. 1389–1396, 1976.
- [22] DONEA, J., “A Taylor-Galerkin method for convective transport problems”, *International Journal for Numerical Methods in Engineering*, v. 20, n. 1, pp. 101–119, 1984.
- [23] LÖHNER, R., MORGAN, K., ZIENKIEWICZ, O. C., “The solution of non-linear hyperbolic equation systems by the finite element method”, *International Journal for Numerical Methods in Fluids*, v. 4, pp. 1043–1063, 1984.
- [24] GHIA, U., GHIA, K. N., SHIN, C. T., “High-Re solutions for incompressible flow using the Navier-Stokes equations and a multigrid method”, *Journal of Computational Physics*, v. 48, pp. 387–411, 1982.
- [25] MARCHI, C. H., SUERO, R., ARAKI, L. K., “The lid-driven square cavity flow: numerical solution with a 1024 x 1024 grid”, *Journal of the Brazilian Society of Mechanical Sciences and Engineering*, v. 31, n. 3, pp. 186–198, 2009.
- [26] ARMALY, B. F., DURST, F., PEREIRA, J. C. F., *et al.*, “Experimental and theoretical investigation of backward-facing step flow”, *Journal of Fluid Mechanics*, v. 127, pp. 473–496, 1983.
- [27] THOMAS, C., MORGAN, K., TAYLOR, C., “A finite element analysis of flow over a backward facing step”, *Computers & Fluids*, v. 9, n. 3, pp. 265–278, 1981.
- [28] PIRONNEAU, O., “On the transport-diffusion algorithm and its applications to the Navier-Stokes equations”, *Numerische Mathematik*, v. 38, pp. 309–332, 1982.
- [29] GEUZAIN, C., REMACLE, J.-F., “Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities”, *International Journal for Numerical Methods in Engineering*, v. 79, n. 11, pp. 1309–1331, 2009.
- [30] AGUIAR, G. D. S. O. N. D., *Simulação Numérica da Transferência de Calor em um Disco de Freio Durante Frenagem usando o Método de Elementos Finitos.*

Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2020.

- [31] ALVES, F. R. D. M., *Solução da equação tridimensional transiente do calor através do método de elementos finitos aplicado a discos de freio*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2021.
- [32] FERREIRA, G. P., *Criação de um código em Python para simular a transferência de calor em um processador computacional*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2022.
- [33] PINHEIRO, V. G., *Análise de condução de calor em componentes eletrônicos de material heterogêneo utilizando método de elementos finitos*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2022.
- [34] CAPITANIO, T. A., *Implementação do método de elementos finitos em Python para o estudo paramétrico de dissipadores de calor de processadores computacionais*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2023.
- [35] MIRANDA, L. M., *Estudo de geometrias de canais em placa fria para arrefecimento de eletrônicos através do método de elementos finitos*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2023.
- [36] TEIXEIRA, M. M., *Solução da equação de Laplace tridimensional para a temperatura aplicada a um bloco de motor através do Método dos Elementos Finitos*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2023.
- [37] OLIVEIRA, M. D. D., *Estudo da transferência de calor em dissipador de calor de aletas retas por meio do Método dos Elementos Finitos*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2025.
- [38] FLORENTINO, Y. D. S., *Estudo de escoamentos para arrefecimento de eletrônicos através de Método de Elementos Finitos*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2022.

- [39] SANTOS, J. D. O. D., *Investigação dos campos de velocidade, vorticidade e função corrente em escoamento livre utilizando o Método de Elementos Finitos*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2022.
- [40] SILVA, J. A., *Método de Elementos Finitos em Python com a abordagem lagrangiana-euleriana para as oscilações de um cilindro*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2020.
- [41] AMARAL, T. A. C. D., *Análise CFD de difusão de espécie química para diferentes geometrias de stent farmacológico*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2020.
- [42] GOMES, M. F., *Análise CFD de escoamento em artérias coronárias para diferentes configurações de restrição de fluxo*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2021.
- [43] ARAUJO, R. S. D. A., *Comparação e validação de métodos numéricos aplicados à Transferência de Calor*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2022.
- [44] GOMES, Y. P., *Análise Variacional e Numérica da Equação de Poisson em Problemas de Transferência de Calor*. Projeto de graduação, Universidade Federal do Rio de Janeiro, Rio de Janeiro, 2025.
- [45] THOM, A., “The Flow Past Circular Cylinders at Low Speeds”, *Proceedings of the Royal Society A*, v. 141, n. 845, pp. 651–669, 1933.
- [46] IMBRIZI, B., “triangle-wasm: Javascript wrapper around Triangle compiled to WebAssembly”, Repositório GitHub. Disponível em: <https://github.com/brunoimbrizi/triangle-wasm>, Acesso em: 09 maio 2026.

Apêndice A

Listagens dos Módulos de Simulação 1D

Este apêndice apresenta as listagens completas dos módulos de simulação unidimensionais implementados no *MinervaMesh*. Os códigos são incluídos com o propósito de garantir a reprodutibilidade científica dos resultados apresentados neste trabalho: cada listagem corresponde diretamente às formulações matemáticas derivadas no Capítulo 3 e às implementações descritas no Capítulo 5. Cabe ressaltar que este apêndice não constitui um repositório de software, mas sim documentação técnica destinada à verificação e validação das soluções numéricas obtidas. Os códigos Python a seguir correspondem aos arquivos `simulation.py` de cada caso, executados integralmente no navegador via PyScript/WebAssembly.

A.1 Condução Permanente 1D

Listagem A.1: Condução permanente 1D — `simulation.py`

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Condução Permanente 1D (MEF) --- Problema 5.2.2
5  Barra 1D com geração interna Q, T(0)=T0 e T(L)=TL.
6  Elementos lineares, solução analítica para comparação.
7  """
8
```

```

9  import numpy as np
10 import matplotlib.pyplot as plt
11 from js import document
12 from pyscript import when
13 import asyncio
14 import io
15 import imageio.v2 as imageio
16 import base64
17
18
19 # =====
20 # 1. Solução analítica
21 # =====
22 def analytical_solution(x, Q, k, L, T0, TL):
23     # Solução da EDO:  $-k T'' = Q$ ,  $T(0)=T0$ ,  $T(L)=TL$ 
24     C1 = (TL - T0 + (Q * L * L) / (2 * k)) / L
25     C2 = T0
26     return - (Q / (2 * k)) * x * x + C1 * x + C2
27
28
29 # =====
30 # 2. Executar simulação MEF 1D
31 # =====
32 @when("click", "#runMEF1D")
33 async def run_mef_1d(event=None):
34     # Abrir modal de execução
35     sim_loading = document.getElementById("sim-loading")
36     try:
37         sim_loading.showModal()
38     except Exception:
39         pass
40     await asyncio.sleep(0.05)
41
42     # Parâmetros
43     L = float(document.getElementById("L").value)
44     nel = int(document.getElementById("nel").value)
45     T0 = float(document.getElementById("T0").value)
46     TL = float(document.getElementById("TL").value)
47     k = float(document.getElementById("k").value)

```

```

48     Q = float(document.getElementById("Q").value)
49
50     # Malha 1D
51     nn = nel + 1
52     x = np.linspace(0.0, L, nn)
53     h = L / nel
54
55     # Matrizes/vetores globais
56     K = np.zeros((nn, nn))           # matriz de rigidez (apostila:
        K)
57     b = np.zeros(nn)                # vetor de carregamento (
        apostila: b)
58
59     # Elemento linear: ke = k/h * [[1,-1],[-1,1]]; be = Q*h/2 *
        [1,1]
60     ke = (k / h) * np.array([[1.0, -1.0], [-1.0, 1.0]])
61     be = (Q * h / 2.0) * np.array([1.0, 1.0])
62
63     # Montagem
64     for e in range(nel):
65         n1, n2 = e, e + 1
66         K[n1:n2+1, n1:n2+1] += ke
67         b[n1:n2+1] += be
68
69     # Contornos essenciais (Dirichlet)
70     # T(0)=T0
71     K[0, :] = 0.0
72     K[0, 0] = 1.0
73     b[0] = T0
74     # T(L)=TL
75     K[-1, :] = 0.0
76     K[-1, -1] = 1.0
77     b[-1] = TL
78
79     # Resolver
80     T = np.linalg.solve(K, b)
81
82     # Analítico (grade densa para curva lisa)
83     x_dense = np.linspace(0.0, L, max(600, min(2400, 8 * nn)))

```

```

84     T_ana_dense = analytical_solution(x_dense, Q, k, L, T0, TL)
85
86     # Analítico nos nós da malha para métricas de erro
87     T_ana_mesh = analytical_solution(x, Q, k, L, T0, TL)
88     erro_nodal = np.abs(T - T_ana_mesh)
89     e_max = float(np.max(erro_nodal))
90     T_max_mef = float(np.max(T))
91     x_max_mef = float(x[int(np.argmax(T))])
92     T_max_ana = float(np.max(T_ana_dense))
93     x_max_ana = float(x_dense[int(np.argmax(T_ana_dense))])
94
95     # Plot
96     fig, ax = plt.subplots(figsize=(8, 4))
97     ax.plot(x, T, 'o-', label='MEF 1D', color='#3B82F6')
98     ax.plot(x_dense, T_ana_dense, '--', label='Analítica', color
99             = '#EF4444')
100    ax.set_xlabel('x [m]')
101    ax.set_ylabel('Temperatura [°C]')
102    ax.set_title('Condução Permanente 1D (MEF)')
103    ax.grid(True, linestyle='--', alpha=0.5)
104    ax.legend()
105    plt.tight_layout()
106
107    buf = io.BytesIO()
108    plt.savefig(buf, format='png')
109    plt.close(fig)
110    buf.seek(0)
111    img = imageio.imread(buf)
112    gif_buffer = io.BytesIO()
113    imageio.mimsave(gif_buffer, [img], format='GIF', duration
114                    =1.0)
115    gif_buffer.seek(0)
116    gif_base64 = base64.b64encode(gif_buffer.read()).decode('utf
117                                -8')
118
119    container = document.getElementById("mef1d-output")
120    metrics_html = f"""
121    <div class="bg-blue-50 border-l-4 border-blue-400 p-3
122            rounded w-full">

```

```

119         <h4 class="font-semibold text-blue-800 mb-1">Validação</
            h4>
120         <p class="text-sm text-gray-700"><strong>Malha:</strong>
            {nel} elementos lineares, h = {h:.4f} m</p>
121         <p class="text-sm text-gray-700"><strong>T_max (MEF):</
            strong> {T_max_mef:.6f} °C em x = {x_max_mef:.3f} m</
            p>
122         <p class="text-sm text-gray-700"><strong>T_max (analí
            tica):</strong> {T_max_ana:.6f} °C em x = {x_max_ana
            :.3f} m</p>
123         <p class="text-sm text-gray-700"><strong>Erro máximo
            nodal:</strong> {e_max:.2e} °C</p>
124     </div>
125     """
126     container.innerHTML = (
127         f"<div class='flex flex-col gap-3 w-full'>"
128         f"<img src='data:image/gif;base64,{gif_base64}' class='
            rounded shadow w-full'/>"
129         f"{metrics_html}"
130         f"</div>"
131     )
132
133     try:
134         sim_loading.close()
135     except Exception:
136         pass

```

A.2 Condução Transiente 1D com $k(x)$ Variável

Listagem A.2: Condução transiente 1D — simulation.py

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Problema Térmico Transiente 1D (MEF)
5  -----
6  Modelo:  $\rho c \frac{\partial T}{\partial t} = \frac{\partial}{\partial x}(k(x) \frac{\partial T}{\partial x})$ ,  $0 < x < L$ ,  $t > 0$ 

```

```

7  Condições de contorno:  $T(0,t) = 0$ ,  $T(L,t) = 1$  (fixas)
8  Condutividade térmica descontínua em  $x=L/2$ , com valores  $k_1$  (
    primeira metade)
9  e  $k_2$  (segunda metade) configuráveis via interface.
10
11 Este script simula a condução transiente de calor 1D usando Mé
    todo dos Elementos Finitos
12 com condutividade térmica variável e número de elementos
    configurável.
13 Gera um GIF com os frames e embute no DOM via PyScript.
14 """
15
16 import numpy as np
17 import matplotlib.pyplot as plt
18 from js import document
19 from pyscript import when
20 import asyncio
21 import imageio.v2 as imageio
22 import io
23 import base64
24
25
26 # =====
27 # 1. Função de condutividade térmica  $k(x)$ 
28 # =====
29 def thermal_conductivity(x, L, k1=5.0, k2=1.0):
30     """
31     Retorna a condutividade térmica  $k(x)$ .
32     Condutividade abrupta (não suavizada).
33     """
34     return np.where(x < L/2, k1, k2)
35
36
37 # =====
38 # 2. Solução analítica de regime permanente para  $k(x)$  descontí
    nuo
39 # =====
40 def analytical_steady_state(x, L, k1, k2):
41     """

```



```

42     Solução analítica de regime permanente para k(x) descontínuo
        em x=L/2
43     com T(0)=0 e T(L)=1.
44
45     Em permanente, o fluxo q = -k·dT/dx é constante. Integrando
        em cada região
46     e impondo continuidade de T em L/2 e T(L)=1:
47         C      = 2·k1·k2 / (L·(k1+k2))
48         T(x)    = (C/k1)·x                                para x < L
        /2
49         T(x)    = (C·L/(2·k1)) + (C/k2)·(x - L/2)        para x >=
        L/2
50     """
51     C = 2.0 * k1 * k2 / (L * (k1 + k2))
52     T = np.zeros_like(x, dtype=float)
53     mask_left = x < L / 2
54     T[mask_left] = (C / k1) * x[mask_left]
55     T_half = (C / k1) * (L / 2)
56     T[~mask_left] = T_half + (C / k2) * (x[~mask_left] - L / 2)
57     return T
58
59
60 # =====
61 # 2. Matrizes de elemento para MEF 1D
62 # =====
63 def element_matrices_1d(x1, x2, k_avg, rho_cp):
64     """Calcula as matrizes de elemento para condução 1D."""
65     h = x2 - x1 # comprimento do elemento
66
67     # Matriz de rigidez do elemento
68     ke = (k_avg / h) * np.array([[1, -1], [-1, 1]])
69
70     # Matriz de massa do elemento (capacidade térmica)
71     me = (rho_cp * h / 6) * np.array([[2, 1], [1, 2]])
72
73     return ke, me
74
75
76 # =====

```

```

77 # 3. Montar sistema global
78 # =====
79 def assemble_system(n_elements, L, rho_cp=1.0, k1=5.0, k2=1.0):
80     """Monta as matrizes globais K e M do sistema MEF."""
81     n_nodes = n_elements + 1
82
83     x_nodes = np.zeros(n_nodes)
84
85     # Primeira metade (x < L/2) - pontos uniformes
86     n_left = n_elements // 2
87     x_nodes[:n_left+1] = np.linspace(0, L/2, n_left+1)
88
89     # Segunda metade (x >= L/2) - pontos uniformes
90     n_right = n_elements - n_left
91     x_nodes[n_left:] = np.linspace(L/2, L, n_right+1)
92
93     # Inicializar matrizes globais
94     K = np.zeros((n_nodes, n_nodes))
95     M = np.zeros((n_nodes, n_nodes))
96
97     # Montar sistema elemento por elemento
98     for i in range(n_elements):
99         x1, x2 = x_nodes[i], x_nodes[i+1]
100
101         # Calcular condutividade média do elemento
102         # Para elementos que cruzam a fronteira x=L/2, usar mé
103         dia ponderada
104         if x1 < L/2 and x2 > L/2:
105             # Elemento cruza a fronteira - calcular média
106             ponderada
107             frac_left = (L/2 - x1) / (x2 - x1) # fração à
108             esquerda da fronteira
109             frac_right = (x2 - L/2) / (x2 - x1) # fração à
110             direita da fronteira
111             k_avg = frac_left * k1 + frac_right * k2
112         else:
113             # Elemento inteiramente em uma região
114             x_center = (x1 + x2) / 2
115             k_avg = thermal_conductivity(x_center, L, k1, k2)

```

```

112
113         ke, me = element_matrizes_1d(x1, x2, k_avg, rho_cp)
114
115         # Montar matrizes globais
116         K[i:i+2, i:i+2] += ke
117         M[i:i+2, i:i+2] += me
118
119     return K, M, x_nodes
120
121
122     # =====
123     # 4. Renderizar frame
124     # =====
125     def render_frame(T, x_nodes, t, n_elements, k1=1.0, k2=5.0):
126         """Desenha um frame e retorna a imagem como array para o GIF
127         ."""
128
129         fig, ax = plt.subplots(figsize=(10, 6))
130
131         # Plotar temperatura numérica
132         ax.plot(x_nodes, T, 'o-', label="MEF", color="#3B82F6",
133                 linewidth=2, markersize=4)
134
135         # Plotar solução analítica de regime permanente (referência)
136         x_dense = np.linspace(0, x_nodes[-1], 200)
137         T_analytical = analytical_steady_state(x_dense, x_nodes[-1],
138                 k1, k2)
139         ax.plot(x_dense, T_analytical, '--', label="Regime
140                 permanente (analítico)", color="#EF4444", linewidth=2)
141
142         # Plotar condutividade térmica como fundo
143         k_values = thermal_conductivity(x_dense, x_nodes[-1], k1, k2
144                 )
145
146         # Criar segundo eixo para k(x)
147         ax2 = ax.twinx()
148         ax2.plot(x_dense, k_values, ':', color='green', alpha=0.7,
149                 linewidth=1)
150         ax2.set_ylabel('Condutividade k(x)', color='green', fontsize
151                 =12)

```

```

144     ax2.tick_params(axis='y', labelcolor='green')
145     ax2.set_ylim(0, max(k1, k2) + 1)
146
147     ax.set_ylim(-0.1, 1.1)
148     ax.set_xlim(0, x_nodes[-1])
149     ax.set_xlabel("Posição x", fontsize=12)
150     ax.set_ylabel("Temperatura T", fontsize=12)
151     ax.set_title(f"Condução Transiente 1D - MEF (n={n_elements})
        | t = {t:.3f}s", fontsize=14)
152     ax.grid(True, linestyle='--', alpha=0.5)
153     ax.legend(fontsize=12)
154
155     # Adicionar linha vertical no meio para mostrar mudança de k
156     # Linha vertical no meio
157     ax.axvline(x=x_nodes[-1]/2, color='red', linestyle=':',
        alpha=0.5)
158
159     # Rótulos em y = k1 e y = k2 no eixo de k(x)
160     eps = 0.02 * x_nodes[-1] # pequeno deslocamento horizontal
        para não colidir com a linha
161     ax2.text(x_nodes[-1]/2 - eps, k1, f'k={k1:g}', ha='right',
        va='center',
162             color='red', fontsize=10)
163     ax2.text(x_nodes[-1]/2 + eps, k2, f'k={k2:g}', ha='left', va
        ='center',
164             color='red', fontsize=10)
165
166     plt.tight_layout()
167     buf = io.BytesIO()
168     plt.savefig(buf, format='png', dpi=100)
169     plt.close(fig)
170     buf.seek(0)
171     return imageio.imread(buf)
172
173
174     # =====
175     # 5. Executar simulação
176     # =====
177     @when("click", "#runHeatBtn")

```

```

178 async def run_simulation(event=None):
179     """Executa a simulação MEF, monta frames e publica um GIF no
        container HTML."""
180     # parâmetros
181     L = float(document.getElementById("L").value)
182     n_elements = int(document.getElementById("n_elements").value
        )
183     k1 = float(document.getElementById("k1").value)
184     k2 = float(document.getElementById("k2").value)
185     rho_cp = float(document.getElementById("rho_cp").value)
186     dt = float(document.getElementById("dt").value)
187     theta = float(document.getElementById("theta").value)
188     tmax = float(document.getElementById("tmax").value)
189
190     # Montar sistema MEF
191     K, M, x_nodes = assemble_system(n_elements, L, rho_cp, k1,
        k2)
192     n_nodes = len(x_nodes)
193
194     # Condições iniciais (temperatura zero em todo domínio,
        exceto bordas)
195     T = np.zeros(n_nodes)
196
197     # Aplicar condições de contorno
198     T[0] = 0.0      #  $T(0,t) = 0$ 
199     T[-1] = 1.0     #  $T(L,t) = 1$ 
200
201     # Identificar nós de contorno
202     boundary_nodes = [0, n_nodes-1]
203     interior_nodes = list(range(1, n_nodes-1))
204
205     # Preparar sistema para integração temporal conforme equação
        da apostila
206     #  $((M/\Delta t) + \theta K) T^{(n+1)} = [(M/\Delta t) - (1-\theta)K] T^n$ 
207     A = M/dt + theta * K
208     B = M/dt - (1 - theta) * K
209
210     # Aplicar condições de contorno no sistema
211     for i in boundary_nodes:

```

```

212         A[i, :] = 0
213         A[i, i] = 1
214         B[i, :] = 0
215         B[i, i] = 1
216
217         # abrir modal de execução durante a simulação
218         container = document.getElementById("heat-output")
219         sim_loading = document.getElementById("sim-loading")
220         if sim_loading is not None:
221             try:
222                 sim_loading.showModal()
223             except Exception:
224                 pass
225         await asyncio.sleep(0.05)
226
227         frames = []
228         nt = max(1, int(tmax / dt))
229
230         # Frame inicial
231         frames.append(render_frame(T, x_nodes, 0.0, n_elements, k1,
232                                   k2))
233
234         # =====
235         # 6. Loop temporal (Crank-Nicolson com suavização suave)
236         # =====
237         T_prev = T.copy() # Para suavização temporal
238
239         for n in range(1, nt + 1):
240             # Resolver sistema: A * T^(n+1) = B * T^n + f
241             b = B @ T
242
243             # Aplicar condições de contorno no vetor b
244             b[0] = 0.0 # T(0,t) = 0
245             b[-1] = 1.0 # T(L,t) = 1
246
247             # Resolver sistema linear
248             T_new = np.linalg.solve(A, b)
249
250             # Atualizar temperatura

```

```

250         T = T_new
251
252         # Garantir condições de contorno exatas
253         T[0] = 0.0
254         T[-1] = 1.0
255
256         if n % max(1, nt // 50) == 0 or n == nt:
257             frames.append(render_frame(T, x_nodes, n*dt,
258                                     n_elements, k1, k2))
259
260         # =====
261         # 7. Montar GIF e publicar no DOM
262         # =====
263         gif_buffer = io.BytesIO()
264         imageio.mimsave(gif_buffer, frames, format='GIF', duration
265                         =0.1, loop=0)
266         gif_buffer.seek(0)
267         gif_base64 = base64.b64encode(gif_buffer.read()).decode("utf
268                               -8")
269
270         # Metricas de validacao: T(L/2) vs analitica de regime
271         # permanente
272         # Analitica (fluxo contínuo em x=L/2): T(L/2) = k2 / (k1 +
273         k2) quando
274         # T(0)=0 e T(L)=1. Para k1=5, k2=1 -> T(L/2) = 1/6 aprox
275         0,1667.
276         idx_meio = int(np.argmin(np.abs(x_nodes - L / 2.0)))
277         T_meio_mef = float(T[idx_meio])
278         T_meio_ana = float(k2 / (k1 + k2))
279         erro_meio = abs(T_meio_mef - T_meio_ana)
280         tau_dif = L * L * rho_cp / max(k1, k2)
281         t_final = nt * dt
282         frac_tau = t_final / tau_dif if tau_dif > 0 else 0.0
283
284         metrics_html = f"""
285         <div class="bg-blue-50 border-l-4 border-blue-400 p-3
286             rounded w-full">
287             <h4 class="font-semibold text-blue-800 mb-1">Validação</
288             h4>

```

```

281         <p class="text-sm text-gray-700"><strong>Tempo final:</
           strong> {t_final:.3f} s ({frac_tau*100:.1f})% de  $\tau_{dif}$ 
           = {tau_dif:.2f} s</p>
282         <p class="text-sm text-gray-700"><strong>T(L/2) MEF:</
           strong> {T_meio_mef:.6f} °C</p>
283         <p class="text-sm text-gray-700"><strong>T(L/2) analí
           tica (regime permanente,  $k_2/(k_1+k_2)$ ):</strong> {
           T_meio_ana:.6f} °C</p>
284         <p class="text-sm text-gray-700"><strong>Desvio em  $x = L$ 
           /2:</strong> {erro_meio:.2e} °C</p>
285     </div>
286     """
287     container.innerHTML = (
288         f"<div class='flex flex-col gap-3 w-full'>"
289         f"<img src='data:image/gif;base64,{gif_base64}' class='
           rounded shadow w-full' />"
290         f"{metrics_html}"
291         f"</div>"
292     )
293
294     if sim_loading is not None:
295         try:
296             sim_loading.close()
297         except Exception:
298             pass

```

A.3 Convecção-Difusão 1D com SUPG

Listagem A.3: Convecção-difusão 1D com SUPG — simulation.py

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Convecção-Difusão 1D Permanente (MEF) --- Problema 5.2.4
5  Equação:  $-k \frac{d^2 T}{dx^2} + \rho c_p u \frac{dT}{dx} = Q$ 
6  Condições:  $T(0)=T_0$ ,  $T(L)=T_L$ 
7  Elementos lineares com estabilização SUPG.
8  """

```



```

9
10 import numpy as np
11 import matplotlib.pyplot as plt
12 from js import document
13 from pyscript import when
14 import asyncio
15 import io
16 import imageio.v2 as imageio
17 import base64
18
19
20 # =====
21 # 1. Solução analítica
22 # =====
23 def analytical_solution(x, Q, k, rho_cp, u, L, T0, TL):
24     """
25     Solução analítica da equação de convecção-difusão 1D
26     permanente.
27     Equação:  $-k \frac{d^2 T}{dx^2} + \rho_{cp} u \frac{dT}{dx} = Q$ 
28     Condições:  $T(0)=T_0$ ,  $T(L)=T_L$ 
29
30     Forma estável numericamente mesmo para grandes números de Pé
31     clet: a razão
32      $(\exp(\alpha x) - 1) / (\exp(\alpha L) - 1)$  é reescrita fatorando  $\exp(\alpha L)$  quando  $\alpha > 0$ ,
33     evitando overflow de exp para  $\alpha \cdot L$  grande.
34     """
35     # Caso puramente difusivo:  $-k T'' = Q$ 
36     if abs(rho_cp * u) < 1e-14:
37         C1 = (TL - T0 + (Q * L * L) / (2 * k)) / L
38         return -(Q / (2 * k)) * x * x + C1 * x + T0
39
40     alpha = rho_cp * u / k
41     Tp = Q * x / (rho_cp * u)
42     Tp_L = Q * L / (rho_cp * u)
43
44     #  $g(x) := (\exp(\alpha x) - 1) / (\exp(\alpha L) - 1)$ 
45     if alpha > 0:
46         # fatora  $\exp(\alpha L)$  para evitar overflow

```

```

45         g = np.exp(alpha * (x - L)) * (1.0 - np.exp(-alpha * x))
46             \
47                 / (1.0 - np.exp(-alpha * L))
48     else:
49         # para  $\alpha \leq 0$ , expm1 já é estável
50         g = np.expm1(alpha * x) / np.expm1(alpha * L)
51
52     return T0 + Tp + (TL - T0 - Tp_L) * g
53
54 # =====
55 # 2. Executar simulação MEF 1D com convecção-difusão
56 # =====
57 @when("click", "#runConveccaoDifusao1D")
58 async def run_conveccao_difusao_1d(event=None):
59     # Abrir modal de execução
60     sim_loading = document.getElementById("sim-loading")
61     try:
62         sim_loading.showModal()
63     except Exception:
64         pass
65     await asyncio.sleep(0.05)
66
67     # Parâmetros
68     L = float(document.getElementById("L").value)
69     nel = int(document.getElementById("nel").value)
70     T0 = float(document.getElementById("T0").value)
71     TL = float(document.getElementById("TL").value)
72     k = float(document.getElementById("k").value)
73     rho_cp = float(document.getElementById("rho_cp").value)
74     u = float(document.getElementById("u").value)
75     Q = float(document.getElementById("Q").value)
76     use_supg = document.getElementById("use_supg").checked
77
78     # Malha 1D uniforme
79     nn = nel + 1
80     x = np.linspace(0.0, L, nn)
81     h = L / nel
82

```

```

83     # Número de Péclet local
84     Pe_local = (rho_cp * u * h) / (2 * k)
85
86     # Parâmetro de estabilização SUPG
87     if use_supg and abs(Pe_local) > 1e-10:
88         tau = h / (2 * abs(u)) * (1 / np.tanh(abs(Pe_local)) - 1
89             / abs(Pe_local))
90     else:
91         tau = 0.0
92
93     # Matrizes/vetores globais
94     K = np.zeros((nn, nn))          # matriz de rigidez
95     b = np.zeros(nn)                # vetor de carregamento
96
97     # Elemento linear com convecção-difusão
98     # Matriz de difusão: ke_diff = k/h * [[1,-1],[-1,1]]
99     ke_diff = (k / h) * np.array([[1.0, -1.0], [-1.0, 1.0]])
100
101     # Matriz de convecção: ke_conv = rho_cp*u/2 * [[-1,1],[-1,1]]
102     ke_conv = (rho_cp * u / 2.0) * np.array([[-1.0, 1.0], [-1.0,
103         1.0]])
104
105     # Vetor de fontes: be = Q*h/2 * [1,1]
106     be = (Q * h / 2.0) * np.array([1.0, 1.0])
107
108     # Estabilização SUPG (se habilitada)
109     if use_supg and tau > 0:
110         # Matriz de estabilização SUPG
111         ke_supg = tau * rho_cp * u * u / h * np.array([[1.0,
112             -1.0], [-1.0, 1.0]])
113         # Vetor de estabilização SUPG
114         be_supg = tau * Q * u * np.array([-1.0, 1.0])
115
116         # Combinar todas as contribuições
117         ke_total = ke_diff + ke_conv + ke_supg
118         be_total = be + be_supg
119     else:
120         ke_total = ke_diff + ke_conv
121         be_total = be

```

```

119
120     # Montagem
121     for e in range(nel):
122         n1, n2 = e, e + 1
123         K[n1:n2+1, n1:n2+1] += ke_total
124         b[n1:n2+1] += be_total
125
126     # Contornos essenciais (Dirichlet)
127     # T(0)=T0
128     K[0, :] = 0.0
129     K[0, 0] = 1.0
130     b[0] = T0
131     # T(L)=TL
132     K[-1, :] = 0.0
133     K[-1, -1] = 1.0
134     b[-1] = TL
135
136     # Resolver
137     T = np.linalg.solve(K, b)
138
139     # Analítico (grade densa para curva lisa)
140     x_dense = np.linspace(0.0, L, max(600, min(2400, 8 * nn)))
141     T_ana_dense = analytical_solution(x_dense, Q, k, rho_cp, u,
142                                     L, T0, TL)
143
144     # Solução analítica nos pontos da malha numérica para cá
145     lcu do erro
146     T_ana_mesh = analytical_solution(x, Q, k, rho_cp, u, L, T0,
147                                     TL)
148
149     # Plot
150     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8))
151
152     # Gráfico principal
153     ax1.plot(x, T, 'o-', label='MEF 1D', color='#3B82F6',
154             markersize=6)
155     ax1.plot(x_dense, T_ana_dense, '--', label='Analítica',
156             color='#EF4444', linewidth=2)
157     ax1.set_xlabel('x [m]')

```

```

153     ax1.set_ylabel('Temperatura [°C]')
154     ax1.set_title('Convecção-Difusão 1D Permanente (MEF)')
155     ax1.grid(True, linestyle='--', alpha=0.5)
156     ax1.legend()
157
158     # Gráfico de erro
159     erro = np.abs(T - T_ana_mesh)
160     # Evitar erro zero para escala logarítmica
161     erro = np.maximum(erro, 1e-18)
162     ax2.semilogy(x, erro, 'o-', color='#10B981', markersize=6)
163     ax2.set_xlabel('x [m]')
164     ax2.set_ylabel('Erro Absoluto [°C]')
165     ax2.set_title('Erro entre Solução Numérica e Analítica (
        Escala Logarítmica)')
166     ax2.grid(True, linestyle='--', alpha=0.5)
167     # Adicionar texto explicativo
168     ax2.text(0.5, 0.5, f'Erro máximo: {np.max(erro):.2e} °C\n(
        Precisão de máquina)',
169             transform=ax2.transAxes, ha='center', va='center',
170             bbox=dict(boxstyle='round', facecolor='white',
171                     alpha=0.8),
172             fontsize=10)
173
174
175     plt.tight_layout()
176
177     buf = io.BytesIO()
178     plt.savefig(buf, format='png', dpi=150)
179     plt.close(fig)
180     buf.seek(0)
181     img = imageio.imread(buf)
182     gif_buffer = io.BytesIO()
183     imageio.mimsave(gif_buffer, [img], format='GIF', duration
184                     =1.0)
185     gif_buffer.seek(0)
186     gif_base64 = base64.b64encode(gif_buffer.read()).decode('utf
187                             -8')
188
189     container = document.getElementById("conveccao-difusao-
190                             output")

```

```

186     container.innerHTML = f"<img src='data:image/gif;base64,{
        gif_base64}' class='rounded shadow w-full'/'>"
187
188     # Atualizar informações sobre Péclet
189     pe_info = document.getElementById("pe-info")
190     pe_info.innerHTML = f"""
191     <div class="bg-blue-50 border-l-4 border-blue-400 p-3
        rounded">
192         <h4 class="font-semibold text-blue-800 mb-1">Informações
            da Simulação</h4>
193         <p class="text-sm text-gray-700"><strong>Número de Pé
            clet:</strong> {Pe_local:.3f}</p>
194         <p class="text-sm text-gray-700"><strong>Estabilização
            SUPG:</strong> {'Sim' if use_supg else 'Não'}</p>
195         <p class="text-sm text-gray-700"><strong>Parâmetro  $\tau$ :</
            strong> {tau:.6f}</p>
196         <p class="text-sm text-gray-700"><strong>Velocidade:</
            strong> {u:.3f} m/s</p>
197         <p class="text-sm text-gray-700"><strong>Tamanho
            elemento:</strong> {h:.4f} m</p>
198         <p class="text-sm text-gray-700"><strong>Erro máximo:</
            strong> {np.max(erro):.2e} °C</p>
199     </div>
200     """
201
202     try:
203         sim_loading.close()
204     except Exception:
205         pass

```

A.4 Viga 1D — Euler-Bernoulli

Listagem A.4: Viga 1D (Euler-Bernoulli) — simulation.py

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Viga 1D Avançada (MEF) --- Problema 5.2.5 Expandido

```

```

5  Análise completa de flexão de vigas usando MEF com elementos de
    viga de Euler-Bernoulli.
6  Inclui múltiplos tipos de carregamento, condições de contorno,
    propriedades variáveis e análises completas.
7  """
8
9  import numpy as np
10 import matplotlib.pyplot as plt
11 from js import document
12 from pyscript import when
13 import asyncio
14 import io
15 import imageio.v2 as imageio
16 import base64
17
18
19 # =====
20 # 1. Funções de carregamento
21 # =====
22 def load_uniform(x, q0, L):
23     """Carregamento uniforme"""
24     return np.full_like(x, q0)
25
26 def load_linear(x, q0, qL, L):
27     """Carregamento linearmente variável"""
28     return q0 + (qL - q0) * x / L
29
30 def load_triangular(x, q_max, L):
31     """Carregamento triangular (máximo no centro)"""
32     return q_max * (1 - 2 * np.abs(x - L/2) / L)
33
34 def load_sinusoidal(x, q0, L):
35     """Carregamento senoidal"""
36     return q0 * np.sin(np.pi * x / L)
37
38 def load_point(x, P, x_pos, L):
39     """Carregamento pontual (aproximado como distribuição
        localizada)"""
40     # Aproximação: distribuição gaussiana muito concentrada

```

```

41     sigma = L / 100 # Largura da distribuição
42     return P * np.exp(-0.5 * ((x - x_pos) / sigma)**2) / (sigma
43         * np.sqrt(2 * np.pi))
44
45 # =====
46 # 2. Soluções analíticas
47 # =====
48 def analytical_solution_simply_supported(x, q_func, E, I, L):
49     """Solução analítica para viga simplesmente apoiada"""
50     if q_func == "uniform":
51         q = 1.0 # Valor normalizado
52         return (q / (24 * E * I)) * (L**3 * x - 2 * L * x**3 + x
53             **4)
54     elif q_func == "triangular":
55         q_max = 1.0
56         return (q_max / (120 * E * I)) * (L**4 * x - 2 * L**2 *
57             x**3 + x**5)
58     return np.zeros_like(x)
59
60 def analytical_solution_cantilever(x, q_func, E, I, L):
61     """Solução analítica para viga em balanço"""
62     if q_func == "uniform":
63         q = 1.0
64         return (q / (24 * E * I)) * (6 * L**2 * x**2 - 4 * L * x
65             **3 + x**4)
66     elif q_func == "triangular":
67         q_max = 1.0
68         return (q_max / (120 * E * I)) * (10 * L**3 * x**2 - 10
69             * L**2 * x**3 + 5 * L * x**4 - x**5)
70     return np.zeros_like(x)
71
72 def analytical_solution_fixed_fixed(x, q_func, E, I, L):
73     """Solução analítica para viga engastada-engastada"""
74     if q_func == "uniform":
75         q = 1.0
76         return (q / (24 * E * I)) * (L**2 * x**2 - 2 * L * x**3
77             + x**4)
78     return np.zeros_like(x)

```



```

74
75
76 # =====
77 # 3. Funções auxiliares para propriedades variáveis
78 # =====
79 def get_variable_properties(x, prop_type, base_value, L):
80     """Calcula propriedades variáveis ao longo da viga"""
81     if prop_type == "constant":
82         return np.full_like(x, base_value)
83     elif prop_type == "linear":
84         # Propriedade varia linearmente de base_value a 0.5*
85         # base_value
86         return base_value * (1 - 0.5 * x / L)
87     elif prop_type == "parabolic":
88         # Propriedade varia parabolicamente (máximo no centro)
89         return base_value * (1 - 0.3 * (x - L/2)**2 / (L/2)**2)
90     return np.full_like(x, base_value)
91
92 # =====
93 # 4. Executar simulação MEF avançada para viga 1D
94 # =====
95 @when("click", "#runViga1D")
96 async def run_viga_1d(event=None):
97     # Abrir modal de execução
98     sim_loading = document.getElementById("sim-loading")
99     try:
100         sim_loading.showModal()
101     except Exception:
102         pass
103     await asyncio.sleep(0.05)
104
105     # Parâmetros básicos
106     L = float(document.getElementById("L").value)
107     nel = int(document.getElementById("nel").value)
108     E_base = float(document.getElementById("E").value) * 1e9 #
109         Converter para Pa
110     I_base = float(document.getElementById("I").value) * 1e-8 #
111         Converter para m4

```

```

110     q_magnitude = float(document.getElementById("q").value) *
        1000    # Converter para N/m
111     bc_type = document.getElementById("bc_type").value
112     load_type = document.getElementById("load_type").value
113     prop_type = document.getElementById("prop_type").value
114     show_analytical = document.getElementById("show_analytical")
        .checked
115
116     # Malha 1D
117     nn = nel + 1
118     x = np.linspace(0.0, L, nn)
119     h = L / nel
120
121     # Propriedades variáveis
122     E = get_variable_properties(x, prop_type, E_base, L)
123     I = get_variable_properties(x, prop_type, I_base, L)
124
125     # Carregamento
126     if load_type == "uniform":
127         q = load_uniform(x, q_magnitude, L)
128     elif load_type == "linear":
129         q0 = q_magnitude
130         qL = 0.5 * q_magnitude
131         q = load_linear(x, q0, qL, L)
132     elif load_type == "triangular":
133         q = load_triangular(x, q_magnitude, L)
134     elif load_type == "sinusoidal":
135         q = load_sinusoidal(x, q_magnitude, L)
136     elif load_type == "point":
137         P = q_magnitude * L    # Força pontual equivalente
138         x_pos = L / 2    # Posição da força
139         q = load_point(x, P, x_pos, L)
140     else:
141         q = load_uniform(x, q_magnitude, L)
142
143     # Para elementos de viga, cada nó tem 2 DOFs: deslocamento
        vertical (w) e rotação (θ)
144     ndof = 2 * nn
145     K = np.zeros((ndof, ndof))

```

```

146     f = np.zeros(ndof)
147
148     # Montagem da matriz global com propriedades variáveis
149     for e in range(nel):
150         # Propriedades do elemento (média dos nós)
151         E_elem = (E[e] + E[e+1]) / 2
152         I_elem = (I[e] + I[e+1]) / 2
153         EI = E_elem * I_elem
154
155         # Carregamento do elemento (média dos nós)
156         q_elem = (q[e] + q[e+1]) / 2
157
158         # Matriz de rigidez do elemento
159         ke = (EI / h**3) * np.array([
160             [12,      6*h,    -12,      6*h],
161             [6*h,     4*h**2, -6*h,     2*h**2],
162             [-12,    -6*h,     12,    -6*h],
163             [6*h,     2*h**2, -6*h,     4*h**2]
164         ])
165
166         # Vetor de forças do elemento
167         fe = (q_elem * h / 12) * np.array([6, h, 6, -h])
168
169         # DOFs do elemento
170         dofs = [2*e, 2*e+1, 2*(e+1), 2*(e+1)+1]
171
172         # Montar matriz de rigidez
173         for i in range(4):
174             for j in range(4):
175                 K[dofs[i], dofs[j]] += ke[i, j]
176
177         # Montar vetor de forças
178         for i in range(4):
179             f[dofs[i]] += fe[i]
180
181     # Aplicar condições de contorno
182     if bc_type == "simply_supported":
183         # Simplesmente apoiada: w(0) = 0, w(L) = 0
184         K[0, :] = 0; K[0, 0] = 1; f[0] = 0

```

```

185         K[2*nn-2, :] = 0; K[2*nn-2, 2*nn-2] = 1; f[2*nn-2] = 0
186
187     elif bc_type == "cantilever":
188         # Viga em balanço:  $w(0) = 0$ ,  $\theta(0) = 0$ 
189         K[0, :] = 0; K[0, 0] = 1; f[0] = 0
190         K[1, :] = 0; K[1, 1] = 1; f[1] = 0
191
192     elif bc_type == "fixed_fixed":
193         # Engastada-engastada:  $w(0) = 0$ ,  $\theta(0) = 0$ ,  $w(L) = 0$ ,  $\theta(L)$ 
194              $= 0$ 
195         K[0, :] = 0; K[0, 0] = 1; f[0] = 0
196         K[1, :] = 0; K[1, 1] = 1; f[1] = 0
197         K[2*nn-2, :] = 0; K[2*nn-2, 2*nn-2] = 1; f[2*nn-2] = 0
198         K[2*nn-1, :] = 0; K[2*nn-1, 2*nn-1] = 1; f[2*nn-1] = 0
199
200     elif bc_type == "fixed_supported":
201         # Engastada-apoiada:  $w(0) = 0$ ,  $\theta(0) = 0$ ,  $w(L) = 0$ 
202         K[0, :] = 0; K[0, 0] = 1; f[0] = 0
203         K[1, :] = 0; K[1, 1] = 1; f[1] = 0
204         K[2*nn-2, :] = 0; K[2*nn-2, 2*nn-2] = 1; f[2*nn-2] = 0
205
206     # Resolver sistema
207     u = np.linalg.solve(K, f)
208
209     # Extrair deslocamentos verticais e rotações
210     w = u[:, 2] # deslocamentos verticais
211     theta = u[:, 1] # rotações
212
213     # Pos-processamento de M (momento) e V (cortante) via
214         derivadas canonicas
215     # das funcoes de Hermite cubica. Para cada elemento de
216         comprimento h com
217     # DOFs [w1, theta1, w2, theta2]:
218     #  $M(x) = -EI * w''(x)$  e linear em xi (coordenada local em
219         [0, 1])
220     #  $V(x) = -EI * w'''(x)$  e constante no elemento (3a
221         derivada de cubico)
222     # Avaliamos M nos dois nos locais e V uma vez por elemento,
223         somando nos

```

```

218     # nos globais e depois dividindo pelo numero de elementos
        adjacentes.
219     # Isso elimina o salto espurio que um pos-processamento por
        diferencas
220     # finitas introduzia nos 2 nos de cada extremidade.
221     M = np.zeros(nn)
222     V = np.zeros(nn)
223     count = np.zeros(nn)
224     for e in range(nel):
225         E_elem = (E[e] + E[e+1]) / 2
226         I_elem = (I[e] + I[e+1]) / 2
227         EI = E_elem * I_elem
228         w1, th1 = w[e], theta[e]
229         w2, th2 = w[e+1], theta[e+1]
230         # d2N/dxi2 em xi=0: [-6, -4h, 6, -2h]      -> M no no
            esquerdo
231         M_left  = -EI / h**2 * (-6*w1 - 4*h*th1 + 6*w2 - 2*h*th2
            )
232         # d2N/dxi2 em xi=1: [ 6,  2h, -6,  4h]      -> M no no
            direito
233         M_right = -EI / h**2 * ( 6*w1 + 2*h*th1 - 6*w2 + 4*h*th2
            )
234         # d3N/dxi3 constante: [12, 6h, -12, 6h]     -> V no
            elemento
235         V_elem  = -EI / h**3 * (12*w1 + 6*h*th1 - 12*w2 + 6*h*
            th2)
236         M[e]    += M_left
237         M[e+1]  += M_right
238         V[e]    += V_elem
239         V[e+1]  += V_elem
240         count[e]    += 1
241         count[e+1]  += 1
242     M /= count
243     V /= count
244
245     # Solução analítica para comparação (se disponível)
246     x_dense = np.linspace(0.0, L, max(600, min(2400, 8 * nn)))
247     w_ana_dense = np.zeros_like(x_dense)
248

```

```

249     if show_analytical and prop_type == "constant":
250         if bc_type == "simply_supported" and load_type == "
            uniform":
251             # Solução analítica para viga simplesmente apoiada
                com carregamento uniforme
252             q_ana = q_magnitude # Usar a magnitude real do
                carregamento
253             w_ana_dense = (q_ana / (24 * E_base * I_base)) * (L
                **3 * x_dense - 2 * L * x_dense**3 + x_dense**4)
254         elif bc_type == "cantilever" and load_type == "uniform":
255             # Solução analítica para viga em balanço com
                carregamento uniforme
256             q_ana = q_magnitude
257             w_ana_dense = (q_ana / (24 * E_base * I_base)) * (6
                * L**2 * x_dense**2 - 4 * L * x_dense**3 +
                x_dense**4)
258         elif bc_type == "fixed_fixed" and load_type == "uniform"
            :
259             # Solução analítica para viga engastada-engastada
                com carregamento uniforme
260             q_ana = q_magnitude
261             w_ana_dense = (q_ana / (24 * E_base * I_base)) * (L
                **2 * x_dense**2 - 2 * L * x_dense**3 + x_dense
                **4)
262
263         # Determinar título da condição de contorno
264         bc_titles = {
265             "simply_supported": "Simplesmente Apoiada",
266             "cantilever": "Em Balanço",
267             "fixed_fixed": "Engastada-Engastada",
268             "fixed_supported": "Engastada-Apoiada"
269         }
270         bc_title = bc_titles.get(bc_type, "Desconhecida")
271
272         # Determinar título do carregamento
273         load_titles = {
274             "uniform": "Uniforme",
275             "linear": "Linear",
276             "triangular": "Triangular",

```

```

277         "sinusoidal": "Senoidal",
278         "point": "Pontual"
279     }
280     load_title = load_titles.get(load_type, "Desconhecido")
281
282     # Plot avançado com múltiplos gráficos
283     fig = plt.figure(figsize=(12, 18))
284
285     # Layout: 5x1 grid
286     gs = fig.add_gridspec(5, 1, height_ratios=[2, 1.5, 1.5, 1.5,
287         1.5])
288
289     # 1. Deslocamentos
290     ax1 = fig.add_subplot(gs[0, :])
291     ax1.plot(x, w*1000, 'o-', label='MEF', color='#3B82F6',
292         markersize=6, linewidth=3)
293     if show_analytical and np.any(w_ana_dense):
294         ax1.plot(x_dense, w_ana_dense*1000, '--', label='Analí
295             tica', color='#EF4444', linewidth=3)
296     ax1.set_xlabel('x [m]', fontsize=14, fontweight='bold')
297     ax1.set_ylabel('Deslocamento [mm]', fontsize=14, fontweight=
298         'bold')
299     ax1.set_title(f'Deslocamentos Verticais - {bc_title} ({
300         load_title})', fontsize=16, fontweight='bold')
301     ax1.grid(True, linestyle='--', alpha=0.5)
302     ax1.legend(fontsize=12)
303     ax1.invert_yaxis()
304     ax1.tick_params(labelsize=12)
305
306     # 2. Carregamento
307     ax2 = fig.add_subplot(gs[1, :])
308     ax2.plot(x, q/1000, 'g-', linewidth=3, label='Carregamento')
309     ax2.fill_between(x, 0, q/1000, alpha=0.3, color='green')
310     ax2.set_xlabel('x [m]', fontsize=14, fontweight='bold')
311     ax2.set_ylabel('Carregamento [kN/m]', fontsize=14,
312         fontweight='bold')
313     ax2.set_title('Carregamento Distribuído', fontsize=16,
314         fontweight='bold')
315     ax2.grid(True, linestyle='--', alpha=0.5)

```

```

309     ax2.legend(fontsize=12)
310     ax2.tick_params(labelsize=12)
311
312     # 3. Propriedades
313     ax3 = fig.add_subplot(gs[2, :])
314
315     # Plotar linha vermelha primeiro (E) com estilo mais visível
316     line1 = ax3.plot(x, E/1e9, 'r-', linewidth=4, label='E [GPa]
        ', alpha=0.9, zorder=3)
317
318     # Criar eixo secundário para linha azul (I) - tracejada e
        mais fina
319     ax3_twin = ax3.twinx()
320     line2 = ax3_twin.plot(x, I*1e8, 'b--', linewidth=2, label='I
        [cm4]', alpha=0.8, zorder=2)
321
322     # Configurar eixos e labels
323     ax3.set_xlabel('x [m]', fontsize=14, fontweight='bold')
324     ax3.set_ylabel('Módulo de Elasticidade [GPa]', color='r',
        fontweight='bold', fontsize=14)
325     ax3_twin.set_ylabel('Momento de Inércia [cm4]', color='b',
        fontweight='bold', fontsize=14)
326     ax3.set_title('Propriedades Materiais', fontweight='bold',
        fontsize=16)
327
328     # Configurar cores dos ticks
329     ax3.tick_params(axis='y', labelcolor='r', labelsize=12)
330     ax3_twin.tick_params(axis='y', labelcolor='b', labelsize=12)
331     ax3.tick_params(axis='x', labelsize=12)
332
333     # Grid mais sutil
334     ax3.grid(True, linestyle='--', alpha=0.3)
335
336     # Ajustar limites dos eixos para melhor visualização
337     E_min, E_max = np.min(E/1e9), np.max(E/1e9)
338     I_min, I_max = np.min(I*1e8), np.max(I*1e8)
339
340     ax3.set_ylim(0.9 * E_min, 1.1 * E_max)
341     ax3_twin.set_ylim(0.9 * I_min, 1.1 * I_max)

```



```

342
343     # Adicionar legendas com cores corretas e mais visíveis
344     ax3.legend(loc='upper left', fontsize=12, framealpha=0.9,
345               edgecolor='r')
346
347     ax3_twin.legend(loc='upper right', fontsize=12, framealpha
348                   =0.9, edgecolor='b')
349
350     # 4. Momentos fletores
351     ax4 = fig.add_subplot(gs[3, :])
352     ax4.plot(x, M/1000, 'o-', color='#10B981', markersize=6,
353             linewidth=3)
354     ax4.set_xlabel('x [m]', fontsize=14, fontweight='bold')
355     ax4.set_ylabel('Momento Fletor [kN·m]', fontsize=14,
356                   fontweight='bold')
357     ax4.set_title('Momento Fletor', fontsize=16, fontweight='
358                   bold')
359     ax4.grid(True, linestyle='--', alpha=0.5)
360     ax4.tick_params(labelsize=12)
361
362     # 5. Esforços cortantes
363     ax5 = fig.add_subplot(gs[4, :])
364     ax5.plot(x, V/1000, 'o-', color='#F59E0B', markersize=6,
365             linewidth=3)
366     ax5.set_xlabel('x [m]', fontsize=14, fontweight='bold')
367     ax5.set_ylabel('Esforço Cortante [kN]', fontsize=14,
368                   fontweight='bold')
369     ax5.set_title('Esforço Cortante', fontsize=16, fontweight='
370                   bold')
371     ax5.grid(True, linestyle='--', alpha=0.5)
372     ax5.tick_params(labelsize=12)
373
374     plt.tight_layout()
375
376     # Converter para GIF
377     buf = io.BytesIO()
378     plt.savefig(buf, format='png', dpi=150, bbox_inches='tight')
379     plt.close(fig)
380     buf.seek(0)
381     img = imageio.imread(buf)

```

```

373 gif_buffer = io.BytesIO()
374 imageio.mimsave(gif_buffer, [img], format='GIF', duration
    =1.0)
375 gif_buffer.seek(0)
376 gif_base64 = base64.b64encode(gif_buffer.read()).decode('utf
    -8')
377
378 # Metricas de validacao
379 w_abs = np.abs(w)
380 i_wmax = int(np.argmax(w_abs))
381 w_max_mef = float(w[i_wmax]) # m (com sinal)
382 x_wmax = float(x[i_wmax])
383 M_abs_max = float(np.max(np.abs(M))) # N*m
384 V_abs_max = float(np.max(np.abs(V))) # N
385 EI_const = E_base * I_base
386
387 # Analiticos fechados (apenas quando prop=constante, carga=
    uniforme, 3 BCs)
388 w_max_ana = None
389 M_max_ana = None
390 V_max_ana = None
391 if prop_type == "constant" and load_type == "uniform":
392     q_ana = q_magnitude
393     if bc_type == "simply_supported":
394         w_max_ana = 5.0 * q_ana * L**4 / (384.0 * EI_const)
395         M_max_ana = q_ana * L * L / 8.0
396         V_max_ana = q_ana * L / 2.0
397     elif bc_type == "cantilever":
398         w_max_ana = q_ana * L**4 / (8.0 * EI_const)
399         M_max_ana = q_ana * L * L / 2.0
400         V_max_ana = q_ana * L
401     elif bc_type == "fixed_fixed":
402         w_max_ana = q_ana * L**4 / (384.0 * EI_const)
403         M_max_ana = q_ana * L * L / 12.0
404         V_max_ana = q_ana * L / 2.0
405
406 def fmt_cmp(mef, ana, unit, scale=1.0):
407     if ana is None:
408         return f"{mef*scale:.4f} {unit}"

```

```

409     err_rel = abs(mef - ana) / abs(ana) if abs(ana) > 1e-12
        else abs(mef - ana)
410     return (
411         f"{mef*scale:.4f} {unit} "
412         f"(analítica {ana*scale:.4f} {unit}, erro relativo {
            err_rel:.2e})"
413     )
414
415     rows = [
416         f"<p class=\"text-sm text-gray-700\"><strong>Deflexão má
            xima:</strong> {fmt_cmp(w_max_mef, w_max_ana, 'mm',
            1000)} em x = {x_wmax:.3f} m</p>",
417         f"<p class=\"text-sm text-gray-700\"><strong>|M| máximo
            :</strong> {fmt_cmp(M_abs_max, M_max_ana, 'kN·m',
            1/1000)}</p>",
418         f"<p class=\"text-sm text-gray-700\"><strong>|V| máximo
            :</strong> {fmt_cmp(V_abs_max, V_max_ana, 'kN',
            1/1000)}</p>",
419     ]
420     metrics_html = (
421         "<div class=\"bg-blue-50 border-l-4 border-blue-400 p-3
            rounded w-full\">"
422         "<h4 class=\"font-semibold text-blue-800 mb-1\">Validaçã
            o</h4>"
423         f"<p class=\"text-sm text-gray-700\"><strong>Caso:</
            strong> {bc_title} com carga {load_title.lower()},
            propriedades {prop_type}</p>"
424         + "".join(rows)
425         + "</div>"
426     )
427
428     container = document.getElementById("vigaId-output")
429     container.innerHTML = (
430         f"<div class='flex flex-col gap-3 w-full'>"
431         f"<img src='data:image/gif;base64,{gif_base64}' class='
            rounded shadow w-full'>"
432         f"{metrics_html}"
433         f"</div>"
434     )

```

```

435
436     try:
437         sim_loading.close()
438     except Exception:
439         pass
440
441
442     # =====
443     # 5. Função para casos pré-definidos
444     # =====
445     @when("click", "#loadCase")
446     async def load_preset_case(event=None):
447         case = document.getElementById("preset_cases").value
448
449         if case == "concrete_beam":
450             document.getElementById("L").value = "6.0"
451             document.getElementById("E").value = "30.0"
452             document.getElementById("I").value = "20000.0"
453             document.getElementById("q").value = "15.0"
454             document.getElementById("bc_type").value = "
                simply_supported"
455             document.getElementById("load_type").value = "uniform"
456             document.getElementById("prop_type").value = "constant"
457
458         elif case == "steel_beam":
459             document.getElementById("L").value = "8.0"
460             document.getElementById("E").value = "200.0"
461             document.getElementById("I").value = "50000.0"
462             document.getElementById("q").value = "25.0"
463             document.getElementById("bc_type").value = "
                simply_supported"
464             document.getElementById("load_type").value = "uniform"
465             document.getElementById("prop_type").value = "constant"
466
467         elif case == "cantilever_roof":
468             document.getElementById("L").value = "4.0"
469             document.getElementById("E").value = "200.0"
470             document.getElementById("I").value = "15000.0"
471             document.getElementById("q").value = "8.0"

```

```

472         document.getElementById("bc_type").value = "cantilever"
473         document.getElementById("load_type").value = "triangular"
474         "
475         document.getElementById("prop_type").value = "linear"
476
477     elif case == "bridge_beam":
478         document.getElementById("L").value = "20.0"
479         document.getElementById("E").value = "200.0"
480         document.getElementById("I").value = "100000.0"
481         document.getElementById("q").value = "50.0"
482         document.getElementById("bc_type").value = "
483             simply_supported"
484         document.getElementById("load_type").value = "point"
485         document.getElementById("prop_type").value = "constant"

```

A.5 Vibração 1D — Problema de Autovalor

Listagem A.5: Vibração 1D — simulation.py

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Problema de Autovalor 1D - Vibração em Corpo Sólido (MEF)
5  Problema 5.2.6: Solução de problema de autovalor 1D - vibração
6  em um corpo sólido
7
8  Este código resolve o problema de vibração livre de uma barra 1D
9  usando MEF,
10 calculando frequências naturais e modos de vibração através da
11 solução do problema de autovalor.
12 """
13
14 import numpy as np
15 import matplotlib.pyplot as plt
16 from js import document
17 from pyscript import when
18 import asyncio
19 import io

```

```

17 import imageio.v2 as imageio
18 import base64
19 from scipy.linalg import eigh
20
21
22 # =====
23 # 1. Funções auxiliares para propriedades variáveis
24 # =====
25 def get_variable_properties(x, prop_type, base_value, L):
26     """Calcula propriedades variáveis ao longo da barra"""
27     if prop_type == "constant":
28         return np.full_like(x, base_value)
29     elif prop_type == "linear":
30         # Propriedade varia linearmente de base_value a 0.5*
31         # base_value
32         return base_value * (1 - 0.5 * x / L)
33     elif prop_type == "parabolic":
34         # Propriedade varia parabolicamente (máximo no centro)
35         return base_value * (1 - 0.3 * (x - L/2)**2 / (L/2)**2)
36     elif prop_type == "exponential":
37         # Propriedade varia exponencialmente
38         return base_value * np.exp(-0.5 * x / L)
39     return np.full_like(x, base_value)
40
41 # =====
42 # 2. Soluções analíticas para frequências naturais
43 # =====
44 def analytical_frequencies_bars(L, E, rho, n_modes=5):
45     """Frequências naturais analíticas para barra livre-livre"""
46     c = np.sqrt(E / rho) # Velocidade de onda
47     frequencies = []
48     for n in range(1, n_modes + 1):
49         # Para barra livre-livre:  $f_n = (n-1) * c / (2*L)$  para  $n > 1$ 
50         # Primeiro modo é movimento de corpo rígido ( $f = 0$ )
51         if n == 1:
52             frequencies.append(0.0)
53         else:

```

```

54         frequencies.append((n-1) * c / (2*L))
55     return np.array(frequencies)
56
57
58 def analytical_frequencies_fixed_fixed(L, E, rho, n_modes=5):
59     """Frequências naturais analíticas para barra engastada-
60         engastada"""
61     c = np.sqrt(E / rho) # Velocidade de onda
62     frequencies = []
63     for n in range(1, n_modes + 1):
64         frequencies.append(n * c / (2*L))
65     return np.array(frequencies)
66
67 def analytical_frequencies_fixed_free(L, E, rho, n_modes=5):
68     """Frequências naturais analíticas para barra engastada-
69         livre"""
70     c = np.sqrt(E / rho) # Velocidade de onda
71     frequencies = []
72     for n in range(1, n_modes + 1):
73         frequencies.append((2*n-1) * c / (4*L))
74     return np.array(frequencies)
75
76 # =====
77 # 3. Executar simulação MEF para vibração 1D
78 # =====
79 @when("click", "#runVibacao1D")
80 async def run_vibacao_1d(event=None):
81     # Abrir modal de execução
82     sim_loading = document.getElementById("sim-loading")
83     try:
84         sim_loading.showModal()
85     except Exception:
86         pass
87     await asyncio.sleep(0.05)
88
89     # Parâmetros básicos
90     L = float(document.getElementById("L").value)

```

```

91     nel = int(document.getElementById("nel").value)
92     E_base = float(document.getElementById("E").value) * 1e9 #
           Converter para Pa
93     rho_base = float(document.getElementById("rho").value) # kg
           /m³
94     A_base = float(document.getElementById("A").value) * 1e-4 #
           Converter para m²
95     bc_type = document.getElementById("bc_type").value
96     prop_type = document.getElementById("prop_type").value
97     n_modes = 5 # Fixo em 5 modos
98     show_analytical = document.getElementById("show_analytical")
           .checked
99
100
101     nn = nel + 1
102     x = np.linspace(0.0, L, nn)
103     h = L / nel
104
105     # Propriedades variáveis
106     E = get_variable_properties(x, prop_type, E_base, L)
107     rho = get_variable_properties(x, prop_type, rho_base, L)
108     A = get_variable_properties(x, prop_type, A_base, L)
109
110     # Para elementos de barra, cada nó tem 1 DOF: deslocamento
           axial (u)
111     ndof = nn
112     K = np.zeros((ndof, ndof)) # Matriz de rigidez
113     M = np.zeros((ndof, ndof)) # Matriz de massa
114
115     # Montagem das matrizes globais
116     for e in range(nel):
117         # Propriedades do elemento (média dos nós)
118         E_elem = (E[e] + E[e+1]) / 2
119         rho_elem = (rho[e] + rho[e+1]) / 2
120         A_elem = (A[e] + A[e+1]) / 2
121
122         # Matriz de rigidez do elemento de barra
123         ke = (E_elem * A_elem / h) * np.array([
124             [1, -1],

```



```

125         [-1, 1]
126     ])
127
128     # Matriz de massa do elemento de barra (massa
129     concentrada)
129     me = (rho_elem * A_elem * h / 2) * np.array([
130         [1, 0],
131         [0, 1]
132     ])
133
134     # DOFs do elemento
135     dofs = [e, e+1]
136
137     # Montar matriz de rigidez
138     for i in range(2):
139         for j in range(2):
140             K[dofs[i], dofs[j]] += ke[i, j]
141
142     # Montar matriz de massa
143     for i in range(2):
144         for j in range(2):
145             M[dofs[i], dofs[j]] += me[i, j]
146
147     # Aplicar condições de contorno
148     if bc_type == "free_free":
149         # Barra livre-livre: sem restrições
150         pass
151
152     elif bc_type == "fixed_fixed":
153         # Barra engastada-engastada:  $u(0) = 0$ ,  $u(L) = 0$ 
154         # Aplicar penalidade nas diagonais principais
155         penalty = 1e12
156         K[0, 0] += penalty
157         K[nn-1, nn-1] += penalty
158
159     elif bc_type == "fixed_free":
160         # Barra engastada-livre:  $u(0) = 0$ 
161         penalty = 1e12
162         K[0, 0] += penalty

```

```

163
164     elif bc_type == "fixed_supported":
165         # Barra engastada-apoiada:  $u(0) = 0$ ,  $u(L) = 0$ 
166         penalty = 1e12
167         K[0, 0] += penalty
168         K[nn-1, nn-1] += penalty
169
170     # Resolver problema de autovalor:  $K * \phi = \lambda * M * \phi$ 
171     # Usar scipy.linalg.eigh para matrizes simétricas
172     eigenvalues, eigenvectors = eigh(K, M)
173
174     # Calcular frequências naturais (Hz)
175     frequencies = np.sqrt(eigenvalues) / (2 * np.pi)
176
177     # Ordenar por frequência crescente
178     idx = np.argsort(frequencies)
179     frequencies = frequencies[idx]
180     eigenvectors = eigenvectors[:, idx]
181
182     # Pegar apenas os primeiros n_modes
183     frequencies = frequencies[:n_modes]
184     eigenvectors = eigenvectors[:, :n_modes]
185
186     # Normalizar modos de vibração
187     for i in range(n_modes):
188         eigenvectors[:, i] = eigenvectors[:, i] / np.max(np.abs(
189             eigenvectors[:, i]))
190
191     # Solução analítica para comparação (se disponível)
192     freq_analytical = np.zeros(n_modes)
193
194     if show_analytical and prop_type == "constant":
195         if bc_type == "free_free":
196             freq_analytical = analytical_frequencies_bars(L,
197                 E_base, rho_base, n_modes)
198         elif bc_type == "fixed_fixed":
199             freq_analytical = analytical_frequencies_fixed_fixed
200                 (L, E_base, rho_base, n_modes)
201         elif bc_type == "fixed_free":

```

```

199         freq_analytical = analytical_frequencies_fixed_free(
200             L, E_base, rho_base, n_modes)
201
202     # Determinar título da condição de contorno
203     bc_titles = {
204         "free_free": "Livre-Livre",
205         "fixed_fixed": "Engastada-Engastada",
206         "fixed_free": "Engastada-Livre",
207         "fixed_supported": "Engastada-Apoiada"
208     }
209     bc_title = bc_titles.get(bc_type, "Desconhecida")
210
211     # Plot avançado com múltiplos gráficos
212     fig = plt.figure(figsize=(14, 22))
213
214     # Layout: 7x1 grid (1 tabela + 5 modos + 1 propriedades)
215     gs = fig.add_gridspec(7, 1, height_ratios=[1.2, 2.2, 2.2,
216         2.2, 2.2, 2.2, 1.5], hspace=0.5)
217
218     # 1. Tabela de frequências
219     ax1 = fig.add_subplot(gs[0, :])
220     ax1.axis('off')
221
222     # Criar tabela de frequências
223     table_data = []
224     show_analytical_table = show_analytical and prop_type == "
225         constant"
226
227     for i in range(min(5, n_modes)):
228         row = [f'Modo {i+1}', f'{frequencies[i]:.2f} Hz']
229         if show_analytical_table and i < len(freq_analytical)
230             and freq_analytical[i] > 0:
231             error = abs(frequencies[i] - freq_analytical[i]) /
232                 freq_analytical[i] * 100
233             row.append(f'{freq_analytical[i]:.2f} Hz')
234             row.append(f'{error:.1f}%')
235         elif show_analytical_table:
236             row.extend(['-', '-'])
237     table_data.append(row)

```

```

233
234     # Definir cabeçalhos e larguras baseado nas condições
235     if show_analytical_table:
236         headers = ['Modo', 'MEF [Hz]', 'Analítica [Hz]', 'Erro
                [%]']
237         col_widths = [0.2, 0.2, 0.2, 0.2]
238     else:
239         headers = ['Modo', 'MEF [Hz]']
240         col_widths = [0.3, 0.3]
241
242     table = ax1.table(cellText=table_data, colLabels=headers,
243                      cellLoc='center', loc='center',
244                      colWidths=col_widths)
245     table.auto_set_font_size(False)
246     table.set_fontsize(12)
247     table.scale(1, 2)
248
249     # Colorir cabeçalho
250     for i in range(len(headers)):
251         table[(0, i)].set_facecolor('#4F46E5')
252         table[(0, i)].set_text_props(weight='bold', color='white
                ')
253
254     ax1.set_title(f'Frequências Naturais - {bc_title}', fontsize
                =16, fontweight='bold', pad=50)
255
256     # 2-6. Modos de vibração (5 modos fixos)
257     colors = ['#3B82F6', '#EF4444', '#10B981', '#F59E0B', '#8
                B5CF6']
258     for i in range(min(5, n_modes)):
259         ax = fig.add_subplot(gs[i+1, :])
260
261         # Plotar modo de vibração
262         ax.plot(x, eigenvectors[:, i], 'o-', color=colors[i],
                markersize=6, linewidth=3,
263                label=f'Modo {i+1}: {frequencies[i]:.2f} Hz')
264
265         # Adicionar linha de referência em zero

```

```

266         ax.axhline(y=0, color='black', linestyle='--', alpha
267                     =0.3)
268
269         ax.set_xlabel('x [m]', fontsize=14, fontweight='bold')
270         ax.set_ylabel('Amplitude Normalizada', fontsize=14,
271                     fontweight='bold')
272         ax.set_title(f'Modo de Vibração {i+1}', fontsize=16,
273                     fontweight='bold')
274         ax.grid(True, linestyle='--', alpha=0.5)
275         ax.legend(fontsize=12)
276         ax.tick_params(labelsize=12)
277
278         # Adicionar nós da malha
279         ax.scatter(x, eigenvectors[:, i], color=colors[i], s=50,
280                 zorder=5)
281
282         # 7. Propriedades materiais
283         ax_props = fig.add_subplot(gs[6, :])
284
285         # Plotar linha vermelha primeiro (E) com estilo mais visível
286         line1 = ax_props.plot(x, E/1e9, 'r-', linewidth=4, label='E
287                             [GPa]', alpha=0.9, zorder=3)
288
289         # Criar eixo secundário para linha azul (rho) - tracejada e
290         # mais fina
291         ax_props_twin = ax_props.twinx()
292         line2 = ax_props_twin.plot(x, rho, 'b--', linewidth=2, label
293                                 =f'ρ [kg/m³]', alpha=0.8, zorder=2)
294
295         # Configurar eixos e labels
296         ax_props.set_xlabel('x [m]', fontsize=14, fontweight='bold')
297         ax_props.set_ylabel('Módulo de Elasticidade [GPa]', color='r
298                             ', fontweight='bold', fontsize=14)
299         ax_props_twin.set_ylabel('Densidade [kg/m³]', color='b',
300                                fontweight='bold', fontsize=14)
301         ax_props.set_title('Propriedades Materiais', fontweight='
302                             bold', fontsize=16)
303
304         # Configurar cores dos ticks

```

```

295     ax_props.tick_params(axis='y', labelcolor='r', labelsiz=12)
296     ax_props_twin.tick_params(axis='y', labelcolor='b',
297                               labelsiz=12)
298
299     # Grid mais sutil
300     ax_props.grid(True, linestyle='--', alpha=0.3)
301
302     # Ajustar limites dos eixos para melhor visualização
303     E_min, E_max = np.min(E/1e9), np.max(E/1e9)
304     rho_min, rho_max = np.min(rho), np.max(rho)
305
306     ax_props.set_ylim(0.9 * E_min, 1.1 * E_max)
307     ax_props_twin.set_ylim(0.9 * rho_min, 1.1 * rho_max)
308
309     # Adicionar legendas com cores corretas e mais visíveis
310     ax_props.legend(loc='upper left', fontsize=12, framealpha
311                    =0.9, edgecolor='r')
312
313     ax_props_twin.legend(loc='upper right', fontsize=12,
314                         framealpha=0.9, edgecolor='b')
315
316     # Converter para GIF
317     buf = io.BytesIO()
318     plt.savefig(buf, format='png', dpi=150, bbox_inches='tight')
319     plt.close(fig)
320     buf.seek(0)
321     img = imageio.imread(buf)
322     gif_buffer = io.BytesIO()
323     imageio.mimsave(gif_buffer, [img], format='GIF', duration
324                     =1.0)
325     gif_buffer.seek(0)
326     gif_base64 = base64.b64encode(gif_buffer.read()).decode('utf
327                               -8')
328
329     container = document.getElementById("vibracao1d-output")
330     container.innerHTML = f"<img src='data:image/gif;base64,{
331                               gif_base64}' class='rounded shadow w-full'/"
332
333     try:

```

```

328         sim_loading.close()
329     except Exception:
330         pass
331
332
333     # =====
334     # 4. Função para casos pré-definidos
335     # =====
336     @when("click", "#loadCase")
337     async def load_preset_case(event=None):
338         case = document.getElementById("preset_cases").value
339
340         if case == "steel_bar":
341             document.getElementById("L").value = "2.0"
342             document.getElementById("E").value = "200.0"
343             document.getElementById("rho").value = "7850.0"
344             document.getElementById("A").value = "100.0"
345             document.getElementById("bc_type").value = "fixed_fixed"
346             document.getElementById("prop_type").value = "constant"
347
348         elif case == "aluminum_bar":
349             document.getElementById("L").value = "1.5"
350             document.getElementById("E").value = "70.0"
351             document.getElementById("rho").value = "2700.0"
352             document.getElementById("A").value = "50.0"
353             document.getElementById("bc_type").value = "fixed_free"
354             document.getElementById("prop_type").value = "constant"
355
356         elif case == "concrete_column":
357             document.getElementById("L").value = "3.0"
358             document.getElementById("E").value = "30.0"
359             document.getElementById("rho").value = "2400.0"
360             document.getElementById("A").value = "400.0"
361             document.getElementById("bc_type").value = "fixed_fixed"
362             document.getElementById("prop_type").value = "linear"
363
364         elif case == "composite_bar":
365             document.getElementById("L").value = "1.0"
366             document.getElementById("E").value = "100.0"

```

```
367     document.getElementById("rho").value = "2000.0"
368     document.getElementById("A").value = "25.0"
369     document.getElementById("bc_type").value = "free_free"
370     document.getElementById("prop_type").value = "parabolic"
```


Apêndice B

Listagens dos Módulos de Simulação 2D

Este apêndice apresenta as listagens completas dos módulos de simulação bidimensionais implementados no *MinervaMesh*, incluindo os módulos compartilhados de geração de malha e geometria. Os códigos são incluídos com o propósito de garantir a reprodutibilidade científica dos resultados apresentados neste trabalho: cada listagem corresponde diretamente às formulações matemáticas derivadas no Capítulo 3 e às implementações descritas no Capítulo 5. Cabe ressaltar que este apêndice não constitui um repositório de software, mas sim documentação técnica destinada à verificação e validação das soluções numéricas obtidas. Os códigos Python a seguir são executados integralmente no navegador via PyScript/WebAssembly.

B.1 Malha e Matrizes MEF

Listagem B.1: Módulo compartilhado de malha e matrizes MEF — `common.py`

```
1 import numpy as np
2 import matplotlib.tri as tri
3 from scipy.sparse import lil_matrix, csr_matrix
4 import matplotlib.pyplot as plt
5 import io
6 import base64
7
8 # =====
```

```

9  # 1. Geração da Malha (Genérica)
10 # =====
11 def generate_mesh_generic(L, H, boundary_points, mask_function,
12                             cx, cy, nx, ny):
13     """
14     Gera malha triangular com um buraco definido por
15     boundary_points.
16     boundary_points: array (N, 2) com pontos da fronteira do
17     obstáculo.
18     mask_function: função f(x, y) -> bool que retorna True se o
19     ponto deve ser MANTIDO (fora do obstáculo).
20     """
21     # 1. Grid de fundo
22     x = np.linspace(0, L, nx)
23     y = np.linspace(0, H, ny)
24     dx = x[1] - x[0]
25     dy = y[1] - y[0]
26     h_mesh = min(dx, dy)
27
28     Xg, Yg = np.meshgrid(x, y)
29     X_flat = Xg.flatten()
30     Y_flat = Yg.flatten()
31
32     # 2. Nós da fronteira do obstáculo (passados como argumento)
33     x_bound = boundary_points[:, 0]
34     y_bound = boundary_points[:, 1]
35
36     # 3. Filtrar nós do grid usando a função de máscara
37     # Buffer para evitar triângulos muito finos (slivers)
38     # A função de máscara deve ser conservadora (remover apenas
39     o que está garantidamente dentro)
40     # Mas aqui vamos aplicar diretamente
41     mask_keep = mask_function(X_flat, Y_flat)
42
43     X_grid_valid = X_flat[mask_keep]
44     Y_grid_valid = Y_flat[mask_keep]
45
46     # 4. Combinar nós
47     X = np.concatenate([X_grid_valid, x_bound])

```

```

43     Y = np.concatenate([Y_grid_valid, y_bound])
44
45     # 5. Triangulação
46     triang = tri.Triangulation(X, Y)
47
48     # 6. Mascaramos triângulos espúrios (dentro do obstáculo)
49     x_tri = X[triang.triangles].mean(axis=1)
50     y_tri = Y[triang.triangles].mean(axis=1)
51
52     # Se o centróide não passa na máscara de "manter", então
removemos
53     # Mas cuidado: a máscara de manter pode ser "dist > r +
buffer".
54     # Para triângulos, queremos remover se estiver DENTRO do
obstáculo exato.
55     # Vamos assumir que mask_function(x, y, strict=True) remove
o interior.
56     # Por simplicidade, vamos passar uma segunda função ou usar
a mesma com lógica interna.
57     # Vamos usar a mesma mask_function. Se retornar False (
remover), mascaramos.
58
59     triang.set_mask(~mask_function(x_tri, y_tri))
60
61     return X, Y, triang
62
63     # =====
64     # 2. Matrizes MEF (Triângulo Linear)
65     # =====
66     def element_matrices(coords):
67         x0, x1, x2 = coords
68         detJ = (x1[0] - x0[0]) * (x2[1] - x0[1]) - (x2[0] - x0[0]) *
            (x1[1] - x0[1])
69         area = 0.5 * abs(detJ)
70
71         b = np.array([x1[1] - x2[1], x2[1] - x0[1], x0[1] - x1[1]])
72         c = np.array([x2[0] - x1[0], x0[0] - x2[0], x1[0] - x0[0]])
73
74         ke = (1.0 / (4 * area)) * (np.outer(b, b) + np.outer(c, c))

```

```

75     me = (area / 12.0) * (np.ones((3, 3)) + np.eye(3))
76
77     return ke, me, area, b, c
78
79 def assemble_matrices(X, Y, triang):
80     n_nodes = len(X)
81     K = lil_matrix((n_nodes, n_nodes))
82     M = lil_matrix((n_nodes, n_nodes))
83
84     elements = triang.triangles
85     mask = triang.mask if triang.mask is not None else np.zeros(
86         len(elements), dtype=bool)
87
88     for i, el in enumerate(elements):
89         if mask[i]: continue
90
91         coords = np.column_stack((X[el], Y[el]))
92         ke, me, area, b, c = element_matrices(coords)
93
94         for row in range(3):
95             for col in range(3):
96                 K[el[row], el[col]] += ke[row, col]
97                 M[el[row], el[col]] += me[row, col]
98
99     return K.tocsr(), M.tocsr()
100
101 # =====
102 # 3. Helpers de Plotagem
103 # =====
104 def plot_to_base64(fig):
105     buf = io.BytesIO()
106     plt.savefig(buf, format='png', bbox_inches='tight')
107     plt.close(fig)
108     buf.seek(0)
109     return base64.b64encode(buf.read()).decode("utf-8")

```

B.2 Geometrias de Obstáculo

```

1  import numpy as np
2
3  def get_cylinder(params):
4      cx, cy, r = params["cx"], params["cy"], params["r"]
5      n_pts = 100
6      theta = np.linspace(0, 2*np.pi, n_pts, endpoint=False)
7      x_bound = cx + r * np.cos(theta)
8      y_bound = cy + r * np.sin(theta)
9      boundary_pts = np.column_stack([x_bound, y_bound])
10
11     def mask_func(x, y):
12         return (x - cx)**2 + (y - cy)**2 > (r * 1.01)**2
13
14     return boundary_pts, mask_func
15
16 def get_rectangle(params):
17     cx, cy = params["cx"], params["cy"]
18     w, h = params["w"], params["h"]
19
20     perim = 2 * (w + h)
21     n_w = int(100 * (w / perim))
22     n_h = int(100 * (h / perim))
23     n_w = max(5, n_w)
24     n_h = max(5, n_h)
25
26     hw = w / 2
27     hh = h / 2
28
29     x_b = np.linspace(cx - hw, cx + hw, n_w, endpoint=False)
30     y_b = np.full_like(x_b, cy - hh)
31
32     y_r = np.linspace(cy - hh, cy + hh, n_h, endpoint=False)
33     x_r = np.full_like(y_r, cx + hw)
34
35     x_t = np.linspace(cx + hw, cx - hw, n_w, endpoint=False)
36     y_t = np.full_like(x_t, cy + hh)
37
38     y_l = np.linspace(cy + hh, cy - hh, n_h, endpoint=False)

```

```

39     x_l = np.full_like(y_l, cx - hw)
40
41     x_bound = np.concatenate([x_b, x_r, x_t, x_l])
42     y_bound = np.concatenate([y_b, y_r, y_t, y_l])
43     boundary_pts = np.column_stack([x_bound, y_bound])
44
45     def mask_func(x, y):
46         buffer = 1.01
47         return (np.abs(x - cx) > hw * buffer) | (np.abs(y - cy)
48             > hh * buffer)
49
50     return boundary_pts, mask_func
51
52 def get_step(params):
53     # Backward facing step
54     # Step is at bottom left corner.
55     # Dimensions: step_h, step_l
56     step_h = params["step_h"]
57     step_l = params["step_l"]
58
59     # Boundary points for the step (L-shape)
60     # 1. Top of step (0, step_h) -> (step_l, step_h)
61     # 2. Back of step (step_l, step_h) -> (step_l, 0)
62
63     n_pts = 50
64     x_top = np.linspace(0, step_l, n_pts)
65     y_top = np.full_like(x_top, step_h)
66
67     y_back = np.linspace(step_h, 0, n_pts)
68     x_back = np.full_like(y_back, step_l)
69
70     # Concatenate
71     x_bound = np.concatenate([x_top, x_back])
72     y_bound = np.concatenate([y_top, y_back])
73     boundary_pts = np.column_stack([x_bound, y_bound])
74
75     def mask_func(x, y):
76         # Remove if x < step_l AND y < step_h
77         # Keep if x >= step_l OR y >= step_h

```

```

77         # Add small buffer to avoid removing boundary nodes
78         # We want to remove the block (0,0) to (step_l, step_h)
79         # So keep if NOT (x < step_l-eps AND y < step_h-eps)
80         eps = 1e-3
81         return ~((x < step_l - eps) & (y < step_h - eps))
82
83     return boundary_pts, mask_func
84
85
86
87 def get_none(params):
88     # Empty (for Cavity or pure channel)
89     return np.empty((0, 2)), lambda x, y: np.ones_like(x, dtype=
        bool)
90
91 def get_geometry(geo_type, params):
92     if geo_type == "cylinder":
93         return get_cylinder(params)
94     elif geo_type == "rectangle":
95         return get_rectangle(params)
96     elif geo_type == "step":
97         return get_step(params)
98     else:
99         return get_none(params)

```

B.3 Transferência de Calor 2D

Listagem B.3: Transferência de calor 2D — simulation.py

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Simulação Transiente de Condução de Calor em Placa 2D
5  @author: lgsfarias
6  """
7
8  # =====
9  # 0. Importações

```

```

10 # =====
11 import numpy as np
12 import matplotlib.pyplot as plt
13 import matplotlib.tri as tri
14 from scipy.sparse import lil_matrix, csr_matrix
15 from scipy.sparse.linalg import spsolve
16 from pyscript import when, document
17 import imageio.v2 as imageio
18 import io
19 import base64
20 import asyncio
21
22 # =====
23 # 1. Geração da Malha
24 # =====
25 nx, ny = 40, 40
26 x = np.linspace(0, 1, nx)
27 y = np.linspace(0, 1, ny)
28 Xg, Yg = np.meshgrid(x, y)
29 X = Xg.flatten()
30 Y = Yg.flatten()
31 points = np.column_stack((X, Y))
32 triang = tri.Triangulation(X, Y)
33 elements = triang.triangles
34 n_nodes = len(points)
35
36 # =====
37 # 2. Matrizes Locais
38 # =====
39 def element_matrices(coords, k, rho, cv):
40     x0, x1, x2 = coords
41     area = 0.5 * abs((x1[0] - x0[0]) * (x2[1] - x0[1]) - (x2[0]
42         - x0[0]) * (x1[1] - x0[1]))
43     b = np.array([x1[1] - x2[1], x2[1] - x0[1], x0[1] - x1[1]])
44     c = np.array([x2[0] - x1[0], x0[0] - x2[0], x1[0] - x0[0]])
45     ke = (k / (4 * area)) * (np.outer(b, b) + np.outer(c, c))
46     me = (rho * cv * area / 12.0) * (np.ones((3, 3)) + np.eye(3))
47
48     return ke, me

```



```

47
48 # =====
49 # 3. Gerar imagem do frame
50 # =====
51 def generate_frame_image(u, step, dt, Tbottom, Ttop):
52     fig, ax = plt.subplots(figsize=(6, 5))
53     tpc = ax.tricontourf(triang, u, levels=50, cmap='jet', vmin=
        Tbottom, vmax=Ttop)
54     ax.set_title(f"t = {step * dt:.3f} s")
55     ax.set_xlabel("x")
56     ax.set_ylabel("y")
57     fig.colorbar(tpc, ax=ax)
58     buf = io.BytesIO()
59     plt.tight_layout()
60     plt.savefig(buf, format='png')
61     plt.close(fig)
62     buf.seek(0)
63     return imageio.imread(buf)
64
65 # =====
66 # 4. Rodar Simulação
67 # =====
68 @when("click", "#run-btn")
69 async def run_simulation(event=None):
70     # Mostrar spinner de loading dentro do plot-output
71     document.getElementById("plot-output").innerHTML = ""
72     <div class="flex flex-col items-center justify-center py-6
73         ">
74         <div class="animate-spin h-10 w-10 border-4 border-blue
75             -500 border-t-transparent rounded-full"></div>
76         <p class="mt-4 text-blue-700 font-medium">Calculando
77             simulação...</p>
78     </div>
79     ""
80
81     await asyncio.sleep(0.05) # permite que o DOM atualize antes
82         de continuar
83
84     # Parâmetros

```

```

81     dt = float(document.getElementById("dt").value)
82     n_steps = int(document.getElementById("n_steps").value)
83     Tbottom = float(document.getElementById("Tbottom").value)
84     Ttop = float(document.getElementById("Ttop").value)
85     k = float(document.getElementById("k").value)
86     rho = float(document.getElementById("rho").value)
87     cv = float(document.getElementById("cv").value)
88
89     # Matrices globais
90     M = lil_matrix((n_nodes, n_nodes))
91     K = lil_matrix((n_nodes, n_nodes))
92     for el in elements:
93         coords = points[el]
94         ke, me = element_matrices(coords, k, rho, cv)
95         for i in range(3):
96             for j in range(3):
97                 K[el[i], el[j]] += ke[i, j]
98                 M[el[i], el[j]] += me[i, j]
99     M = M.tocsr()
100    K = K.tocsr()
101
102    # Condições iniciais
103    u = np.zeros(n_nodes)
104    for i in range(n_nodes):
105        xi, yi = X[i], Y[i]
106        if np.isclose(xi, 0) or np.isclose(xi, 1):
107            u[i] = Tbottom + (Ttop - Tbottom) * yi
108        elif np.isclose(yi, 0):
109            u[i] = Tbottom
110        elif np.isclose(yi, 1):
111            u[i] = Ttop
112
113    u_init = u.copy()
114    boundary_nodes = [i for i in range(n_nodes) if
115                      np.isclose(X[i], 0) or np.isclose(X[i], 1)
116                      or
117                      np.isclose(Y[i], 0) or np.isclose(Y[i], 1)
118                      ]

```

```

118     A = M + dt / 2 * K
119     B = M - dt / 2 * K
120     A = A.tolil()
121     for i in boundary_nodes:
122         A[i, :] = 0
123         A[i, i] = 1
124     A = A.tocsr()
125
126     # Loop de tempo
127     frames = []
128     for step in range(n_steps):
129         b = B @ u
130         b[boundary_nodes] = u_init[boundary_nodes]
131         u = spsolve(A, b)
132         frames.append(generate_frame_image(u, step, dt, Tbottom,
133                                           Ttop))
134
135     # Salvar GIF
136     gif_buffer = io.BytesIO()
137     imageio.mimsave(gif_buffer, frames, format='GIF', duration
138                     =0.1, loop=0)
139     gif_buffer.seek(0)
140     gif_base64 = base64.b64encode(gif_buffer.read()).decode("utf
141                               -8")
142
143     # Metricas de validacao
144     # T no centro vs regime permanente analitico T_ss = Tbottom
145     + (Ttop-Tbottom) y
146     # Escala difusiva tau_dif = L^2 * rho * cv / k (dominio
147     unitario: L=1)
148     idx_centro = int(np.argmin((X - 0.5) ** 2 + (Y - 0.5) ** 2))
149     T_centro_mef = float(u[idx_centro])
150     T_centro_ss = float(Tbottom + (Ttop - Tbottom) * Y[
151                               idx_centro])
152     erro_centro = abs(T_centro_mef - T_centro_ss)
153     tau_dif = 1.0 * rho * cv / k if k > 0 else float('inf')
154     t_final = n_steps * dt
155     frac_tau = t_final / tau_dif if tau_dif > 0 else 0.0

```

```

151     metrics_html = (
152         "<div class=\"bg-blue-50 border-l-4 border-blue-400 p-3
           rounded w-full\">"
153         "<h4 class=\"font-semibold text-blue-800 mb-1\">Validaçã
           o</h4>"
154         f"<p class=\"text-sm text-gray-700\"><strong>Tempo final
           :</strong> "
155         f"{t_final:.3f} s ({frac_tau*100:.1f})% da escala
           difusiva "
156         f"τ_dif = {tau_dif:.2f} s)</p>"
157         f"<p class=\"text-sm text-gray-700\"><strong>T(0,5; 0,5)
           MEF:</strong> "
158         f"{T_centro_mef:.4f} °C</p>"
159         f"<p class=\"text-sm text-gray-700\"><strong>T(0,5; 0,5)
           regime permanente "
160         f"(T_bot + (T_top - T_bot)·y):</strong> {T_centro_ss:.4f
           } °C</p>"
161         f"<p class=\"text-sm text-gray-700\"><strong>Desvio do
           regime permanente:</strong> "
162         f"{erro_centro:.4f} °C (aumentar n_steps para convergir)
           </p>"
163         "</div>"
164     )
165
166     # Mostrar resultado final
167     document.getElementById("plot-output").innerHTML = (
168         f"<div class='flex flex-col gap-3 w-full'>"
169         f"<img src='data:image/gif;base64,{gif_base64}' class='
           rounded shadow w-full'>"
170         f"{metrics_html}"
171         f"</div>"
172     )

```

B.4 Escoamento Potencial 2D

Listagem B.4: Escoamento potencial 2D — `simulation.py`

```

1 import numpy as np

```

```

2 import matplotlib.pyplot as plt
3 import matplotlib.tri as tri
4 from scipy.sparse.linalg import spsolve
5 from pyscript import when, document
6 import asyncio
7 from common import generate_mesh_generic, assemble_matrices,
    plot_to_base64
8
9 # =====
10 # 1. Geometrias
11 # =====
12 def get_geometry(geo_type, params):
13     cx, cy = params["cx"], params["cy"]
14
15     if geo_type == "cylinder":
16         r = params["r"]
17         n_pts = 100
18         theta = np.linspace(0, 2*np.pi, n_pts, endpoint=False)
19         x_bound = cx + r * np.cos(theta)
20         y_bound = cy + r * np.sin(theta)
21
22         def mask_func(x, y):
23             return (x - cx)**2 + (y - cy)**2 > (r * 1.01)**2
24
25         return np.column_stack([x_bound, y_bound]), mask_func
26
27     elif geo_type == "airfoil" or geo_type == "naca4412":
28         # General NACA 4-Digit Series Generator
29         # M: Max camber (1st digit)
30         # P: Max camber pos (2nd digit)
31         # T: Max thickness (3rd, 4th digits)
32
33         if geo_type == "naca4412":
34             M = 0.04
35             P = 0.40
36             T = 0.12
37         else: # Default 0012
38             M = 0.00
39             P = 0.00

```

```

40         T = 0.12
41
42         chord = params["chord"]
43         angle = np.radians(params["angle"])
44
45         n_pts = 100
46         beta = np.linspace(0, np.pi, n_pts)
47         xc = 0.5 * (1 - np.cos(beta)) # Cosine spacing [0, 1]
48
49         # Thickness distribution (yt)
50         yt = 5 * T * (0.2969*np.sqrt(xc) - 0.1260*xc - 0.3516*xc
51             **2 + 0.2843*xc**3 - 0.1015*xc**4)
52
53         # Camber line (yc) and gradient (dyc_dx)
54         yc = np.zeros_like(xc)
55         dyc_dx = np.zeros_like(xc)
56
57         if M > 0:
58             # Forward of max camber (0 <= x <= P)
59             idx1 = xc <= P
60             yc[idx1] = (M / P**2) * (2*P*xc[idx1] - xc[idx1]**2)
61             dyc_dx[idx1] = (2*M / P**2) * (P - xc[idx1])
62
63             # Aft of max camber (P < x <= 1)
64             idx2 = xc > P
65             yc[idx2] = (M / (1-P)**2) * ((1-2*P) + 2*P*xc[idx2]
66                 - xc[idx2]**2)
67             dyc_dx[idx2] = (2*M / (1-P)**2) * (P - xc[idx2])
68
69         theta_c = np.arctan(dyc_dx)
70
71         # Upper and Lower surface coordinates
72         x_upper = xc - yt * np.sin(theta_c)
73         y_upper = yc + yt * np.cos(theta_c)
74         x_lower = xc + yt * np.sin(theta_c)
75         y_lower = yc - yt * np.cos(theta_c)
76
77         # Scale by chord
78         x_upper *= chord

```

```

77         y_upper *= chord
78         x_lower *= chord
79         y_lower *= chord
80
81         # Combine (trailing edge to leading edge to trailing
            edge)
82         x_local = np.concatenate([x_upper[::-1], x_lower[1:]])
83         y_local = np.concatenate([y_upper[::-1], y_lower[1:]])
84
85         # Center at cx, cy (approximate center of chord)
86         x_local -= chord / 2
87
88         # Rotate and Translate
89         c, s = np.cos(angle), np.sin(angle)
90         x_rot = c * x_local - s * y_local + cx
91         y_rot = s * x_local + c * y_local + cy
92
93         from matplotlib.path import Path
94         poly_path = Path(np.column_stack([x_rot, y_rot]))
95
96         def mask_func(x, y):
97             pts = np.column_stack([x, y])
98             return ~poly_path.contains_points(pts, radius=-1e-3)
99
100        return np.column_stack([x_rot, y_rot]), mask_func
101
102    return None, None
103
104    # =====
105    # 2. Solver Potencial
106    # =====
107    async def solve_potential(params, plot_div):
108        # 1. Geometria e Malha
109        boundary_pts, mask_func = get_geometry(params["geo_type"],
110                                                params)
111
112        X, Y, triang = generate_mesh_generic(
113            params["L"], params["H"], boundary_pts, mask_func,
114            params["cx"], params["cy"], params["nx"], params["ny"]

```

```

114     )
115     n_nodes = len(X)
116
117     # 2. Matrizes
118     K, M = assemble_matrices(X, Y, triang)
119
120     # 3. Fronteiras
121     eps = 1e-5
122     inlet_nodes = np.where(X < eps)[0]
123     outlet_nodes = np.where(X > params["L"] - eps)[0]
124     wall_bottom = np.where(Y < eps)[0]
125     wall_top = np.where(Y > params["H"] - eps)[0]
126
127     # Obstacle nodes: são os últimos N pontos em X, Y (pois
128     concatenamos)
129     n_bound = len(boundary_pts)
130     obstacle_nodes = np.arange(n_nodes - n_bound, n_nodes)
131
132     # 4. Condições de Contorno
133     psi = np.zeros(n_nodes)
134
135     # Normalizado para  $\Psi(H) = v_{inf} * H$ 
136     def get_psi_inlet(y_val):
137         return params["v_inf"] * y_val
138
139     psi[inlet_nodes] = get_psi_inlet(Y[inlet_nodes])
140     psi[wall_bottom] = 0.0
141     psi[wall_top] = params["v_inf"] * params["H"]
142
143     # Obstacle: Psi constante + Gamma
144     # Qual valor base? Psi no centro do obstáculo (aproximado)
145     psi_base = get_psi_inlet(params["cy"])
146     # Hardcode Gamma = 0.0 (Pure Potential Flow)
147     params["Gamma"] = 0.0
148     psi[obstacle_nodes] = psi_base + params["Gamma"]
149
150     # Dirichlet
151     dirichlet_psi = np.concatenate([inlet_nodes, wall_bottom,
152                                     wall_top, obstacle_nodes])

```



```

151     dirichlet_psi = np.unique(dirichlet_psi)
152     free_psi = np.setdiff1d(np.arange(n_nodes), dirichlet_psi)
153
154     # Solver
155     K_free = K[free_psi, :][:, free_psi]
156     b_free = -K[free_psi, :][:, dirichlet_psi] @ psi[
        dirichlet_psi]
157     psi[free_psi] = spsolve(K_free, b_free)
158
159     # --- CÁLCULO AERODINÂMICO ---
160     # 1. Velocidade nos nós
161     # Interpolador linear para gradientes (velocidade) nos
        elementos
162     tci = tri.LinearTriInterpolator(triang, psi)
163     (dpsi_dx, dpsi_dy) = tci.gradient(X, Y)
164     u_nodes = dpsi_dy
165     v_nodes = -dpsi_dx
166
167     # Velocidade de referência (Inlet)
168     U_inf = params["v_inf"]
169     V_sq = u_nodes**2 + v_nodes**2
170     Cp = 1.0 - V_sq / (U_inf**2)
171
172     # 2. Extrair dados na superfície do obstáculo
173     # obstacle_nodes já temos. Precisamos ordenar para plotar e
        integrar.
174     # Ordenar pelo ângulo em relação ao centro (cx, cy) funciona
        para todos (convexos)
175     obs_idx = obstacle_nodes
176     x_obs = X[obs_idx]
177     y_obs = Y[obs_idx]
178     cp_obs = Cp[obs_idx]
179
180     angles = np.arctan2(y_obs - params["cy"], x_obs - params["cx
        "])
181     sorted_indices = np.argsort(angles)
182
183     x_sorted = x_obs[sorted_indices]
184     y_sorted = y_obs[sorted_indices]

```

```

185     cp_sorted = cp_obs[sorted_indices]
186     angles_sorted = angles[sorted_indices]
187
188     # 3. Integração Numérica para CL e CD
189     # F = Integral(-Cp * n * ds)
190     # Aproximação por segmentos lineares entre pontos ordenados
191     CL_num = 0.0
192     CD_num = 0.0
193
194     # Fechar o loop (último conecta ao primeiro)
195     x_loop = np.append(x_sorted, x_sorted[0])
196     y_loop = np.append(y_sorted, y_sorted[0])
197     cp_loop = np.append(cp_sorted, cp_sorted[0])
198
199     for i in range(len(x_sorted)):
200         dx = x_loop[i+1] - x_loop[i]
201         dy = y_loop[i+1] - y_loop[i]
202         ds = np.sqrt(dx*dx + dy*dy)
203
204         # Normal (rotacionar tangente 90 graus para fora)
205         # Tangente: (dx, dy). Normal: (dy, -dx) (Verificar
206             sentido horário/anti-horário)
207         # Pontos ordenados por ângulo (-pi a pi) -> Sentido Anti
208             -Horário
209         # Tangente aponta no sentido anti-horário.
210         # Normal para fora deve ser (dy, -dx).
211
212         nx = dy / ds
213         ny = -dx / ds
214
215         cp_avg = 0.5 * (cp_loop[i] + cp_loop[i+1])
216
217         # Força = -Cp * n * ds
218         CD_num += -cp_avg * nx * ds
219         CL_num += -cp_avg * ny * ds
220
221     # Normalizar pela corda/diâmetro?
222     # Cp já é adimensional. A integral dá Força/ (0.5 rho U^2).
223     # Para obter CL, dividimos pela corda (ou diâmetro).
224     if params["geo_type"] == "cylinder":

```

```

222         ref_len = 2 * params["r"]
223     elif params["geo_type"] == "square":
224         ref_len = params["side"] # Aproximado
225     elif params["geo_type"] == "airfoil" or params["geo_type"]
226         == "naca4412":
227         ref_len = params["chord"]
228     else:
229         ref_len = 1.0
230
231     CL_num /= ref_len
232     CD_num /= ref_len
233
234     # --- CÁLCULO DIMENSIONAL ---
235     # Lift (L) = 0.5 * rho * V^2 * CL * chord
236     # Circulation (Gamma) = L / (rho * V)
237
238     rho = params["rho"]
239     v_inf = params["v_inf"]
240     q_inf = 0.5 * rho * v_inf**2
241
242     # Dimensional Force (N) per unit span
243     Lift_dim = q_inf * CL_num * ref_len
244     Drag_dim = q_inf * CD_num * ref_len
245
246     # Dimensional Circulation (m^2/s)
247     # L = rho * V * Gamma => Gamma = L / (rho * V)
248     if abs(v_inf) > 1e-6:
249         Gamma_dim = Lift_dim / (rho * v_inf)
250     else:
251         Gamma_dim = 0.0
252
253     # --- VISUALIZAÇÃO ---
254     fig = plt.figure(figsize=(10, 12))
255     gs = fig.add_gridspec(3, 1, height_ratios=[1.5, 1.5, 1])
256
257     ax1 = fig.add_subplot(gs[0])
258     ax2 = fig.add_subplot(gs[1])
259     ax3 = fig.add_subplot(gs[2])

```

```

260     # 1. Função Corrente + Malha
261     # Smooth gradient with Gouraud shading and Jet colormap
262     contour = ax1.tripcolor(triang, psi, shading='gouraud', cmap
        = 'jet')
263     # Stronger mesh visibility
264     ax1.triplot(triang, color='k', alpha=0.2, linewidth=0.5)
265     ax1.set_title(f"Função Corrente ( $\psi$ ) - {params['
        geo_type'].capitalize()}")
266     ax1.set_aspect('equal')
267     ax1.set_xlabel("x [m]")
268     ax1.set_ylabel("y [m]")
269     fig.colorbar(contour, ax=ax1)
270
271     # 2. Linhas de Corrente
272     xi = np.linspace(0, params["L"], 100)
273     yi = np.linspace(0, params["H"], 100)
274     Xi, Yi = np.meshgrid(xi, yi)
275     (dpsi_dx_grid, dpsi_dy_grid) = tci.gradient(Xi, Yi)
276     u_grid = dpsi_dy_grid
277     v_grid = -dpsi_dx_grid
278     vel_mag_grid = np.sqrt(u_grid**2 + v_grid**2)
279     strm = ax2.streamplot(Xi, Yi, u_grid, v_grid, color=
        vel_mag_grid, cmap='jet', density=1.5, linewidth=1,
        arrowsize=1.5)
280     ax2.set_title("Linhas de Corrente")
281     ax2.set_aspect('equal')
282     ax2.set_xlim(0, params["L"])
283     ax2.set_ylim(0, params["H"])
284     ax2.set_xlabel("x [m]")
285     ax2.set_ylabel("y [m]")
286     fig.colorbar(strm.lines, ax=ax2, label='Velocidade')
287
288     # 3. Distribuição de Cp
289     if params["geo_type"] == "cylinder":
290         # Plotar vs Theta (graus)
291         theta_deg = np.degrees(angles_sorted)
292         ax3.plot(theta_deg, cp_sorted, 'b-', label='Numérico',
            linewidth=2)
293         ax3.set_xlabel(r' $\theta$  (graus)')

```

```

294         ax3.set_xlim(-180, 180)
295
296         # Analítico (Potencial sem circulação):  $C_p = 1 - 4\sin^2(\theta)$ 
297         theta_ana = np.linspace(-np.pi, np.pi, 200)
298         cp_ana = 1 - 4 * np.sin(theta_ana)**2
299         ax3.plot(np.degrees(theta_ana), cp_ana, 'r--', label='
        Analítico (Teórico)', alpha=0.7)
300
301     elif params["geo_type"] == "airfoil" or params["geo_type"]
        == "naca4412":
302         # Plotar vs x/c
303         # Separar extradorso (upper) e intradorso (lower)
304         # Upper:  $y > c_y$  (aprox, ou usar ângulo)
305         # Rotacionar de volta para achar x ao longo da corda?
306         # Simplificação: usar x_sorted
307         ax3.plot(x_sorted, cp_sorted, 'b.-', label='Cp Numérico'
        )
308         ax3.set_xlabel('x [m]')
309
310     else:
311         ax3.plot(angles_sorted, cp_sorted, 'b.-')
312         ax3.set_xlabel('Ângulo (rad)')
313
314     ax3.set_ylabel(r'$C_p$ (adimensional)')
315     ax3.set_title(r'Distribuição de Coeficiente de Pressão (
        $C_p$)')
316     ax3.grid(True, alpha=0.3)
317     ax3.invert_yaxis() # Convenção aerodinâmica:  $C_p$  negativo
        para cima
318     ax3.legend()
319
320     plt.tight_layout()
321
322     img_base64 = plot_to_base64(fig)
323     plot_div.innerHTML = f''
324

```

```

325     # Validacao: Cp em angulos canonicos vs Cp teorico 1 - 4 sin
        ^2(theta)
326     # Apenas faz sentido para o cilindro (geometria com Cp
        teorico conhecido).
327     cp_validation_rows = ""
328     if params["geo_type"] == "cylinder":
329         canonical_degs = [0, 90, 180, -90]
330         rows_html = []
331         for td in canonical_degs:
332             tr = td * np.pi / 180.0
333             diff = (angles_sorted - tr + np.pi) % (2 * np.pi) -
                np.pi
334             j = int(np.argmin(np.abs(diff)))
335             cp_n = float(cp_sorted[j])
336             cp_t = 1.0 - 4.0 * np.sin(tr) ** 2
337             theta_j = float(np.degrees(angles_sorted[j]))
338             rows_html.append(
339                 f"<p><span class=\"font-semibold\">θ = {td}°:</
                    span> "
340                 f"Cp MEF = {cp_n:.3f} | teórico = {cp_t:.3f} "
341                 f"(Δ = {cp_n - cp_t:+.3f})</p>"
342             )
343         cp_validation_rows = (
344             "<div class=\"bg-blue-50 p-3 rounded shadow-sm\">"
345             "<p class=\"font-bold text-indigo-700 mb-2\">Validaça
                ão: Cp na superfície do cilindro "
346             "(teoria: Cp = 1 - 4 sen2θ)</p>"
347             "<div class=\"grid grid-cols-1 gap-1 text-sm\">"
348             + "".join(rows_html)
349             + "</div></div>"
350         )
351
352     # Exibir Resultados Numéricos (Apenas Numérico)
353     res_html = f"""
354     <div class="grid grid-cols-1 gap-4 text-sm">
355         <div class="bg-blue-50 p-3 rounded text-center shadow-sm
            ">
356             <p class="font-bold text-indigo-700 mb-2">Resultados
                Aerodinâmicos</p>

```

```

357         <div class="grid grid-cols-1 gap-1">
358             <p><span class="font-semibold">Sustentação (L)
359                 :</span> {Lift_dim:.4f} N/m</p>
360             <p><span class="font-semibold">Arrasto (D):</
361                 span> {Drag_dim:.4f} N/m</p>
362             <p><span class="font-semibold">Circulação (&
363                 Gamma;):</span> {Gamma_dim:.4f} m2/s</p>
364         </div>
365     </div>
366     {cp_validation_rows}
367 </div>
368 """
369 document.getElementById("lift-results").innerHTML = res_html
370
371 # =====
372 # 3. Interface Handler
373 # =====
374
375 @when("click", "#run-btn")
376 async def run_handler(event=None):
377     plot_div = document.getElementById("plot-output")
378     plot_div.innerHTML = "Calculando..."
379     await asyncio.sleep(0.1)
380
381     try:
382         params = {
383             "L": float(document.getElementById("L").value),
384             "H": float(document.getElementById("H").value),
385             "cx": float(document.getElementById("cx").value),
386             "cy": float(document.getElementById("cy").value),
387             "nx": int(document.getElementById("nx").value),
388             "ny": int(document.getElementById("ny").value),
389             # "Gamma": float(document.getElementById("Gamma").
390                 value), # Removed
391             "geo_type": document.getElementById("geo_type").
392                 value,
393             "rho": float(document.getElementById("rho").value),
394             "v_inf": float(document.getElementById("v_inf").
395                 value)
396         }

```

```

390
391     if params["geo_type"] == "cylinder":
392         params["r"] = float(document.getElementById("r").
393                               value)
394     elif params["geo_type"] == "airfoil" or params["geo_type"] == "naca4412":
395         params["chord"] = float(document.getElementById("
396                               chord").value)
397         params["angle"] = float(document.getElementById("
398                               angle_airfoil").value)
399
400     await solve_potential(params, plot_div)
401
402 except Exception as e:
403     plot_div.innerHTML = f"Erro: {e}"
404     print(e)

```

B.5 Navier-Stokes 2D (Vorticidade-Corrente)

Listagem B.5: Navier-Stokes 2D — simulation.py

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  import matplotlib.tri as tri
4  from scipy.sparse import lil_matrix
5  from scipy.sparse.linalg import spsolve
6  from pyscript import when, document
7  import asyncio
8  from common import generate_mesh_generic, assemble_matrices,
9    plot_to_base64
10 import geometry
11
12 # Global control flag
13 STOP_SIMULATION = False
14
15 @when("click", "#stop-btn")
16 def stop_simulation_handler(event=None):
17     global STOP_SIMULATION

```



```

17     STOP_SIMULATION = True
18     document.getElementById("stop-btn").classList.add("hidden")
19     document.getElementById("stop-btn").classList.remove("flex")
20     document.getElementById("run-btn").classList.remove("hidden"
21         )
22     document.getElementById("run-btn").classList.add("flex")
23
24     # =====
25     # NS Helpers
26     # =====
27     def assemble_convection(X, Y, triang, u_el, v_el):
28         n_nodes = len(X)
29         C = lil_matrix((n_nodes, n_nodes))
30         elements = triang.triangles
31         mask = triang.mask if triang.mask is not None else np.zeros(
32             len(elements), dtype=bool)
33
34         for i, el in enumerate(elements):
35             if mask[i]: continue
36             x = X[el]
37             y = Y[el]
38             area = 0.5 * np.abs(x[0]*(y[1]-y[2]) + x[1]*(y[2]-y[0])
39                 + x[2]*(y[0]-y[1]))
40             b = np.array([y[1] - y[2], y[2] - y[0], y[0] - y[1]])
41             c = np.array([x[2] - x[1], x[0] - x[2], x[1] - x[0]])
42             ue = u_el[i]
43             ve = v_el[i]
44             K_dot_grad = (ue * b + ve * c) / (2 * area)
45             ce = np.outer(np.ones(3) * (area / 3.0), K_dot_grad)
46             for row in range(3):
47                 for col in range(3):
48                     C[el[row], el[col]] += ce[row, col]
49
50         return C.tocsr()
51
52     def get_boundary_normals(X, Y, boundary_nodes, triang):
53         adj = [set() for _ in range(len(X))]
54         mask = triang.mask if triang.mask is not None else np.zeros(
55             len(triang.triangles), dtype=bool)
56
57         for i, el in enumerate(triang.triangles):

```

```

52         if mask[i]: continue
53         for j in range(3):
54             n1, n2 = el[j], el[(j+1)%3]
55             adj[n1].add(n2)
56             adj[n2].add(n1)
57
58     boundary_set = set(boundary_nodes)
59     valid_neighbors = np.zeros(len(boundary_nodes), dtype=int)
60     h_sq = np.zeros(len(boundary_nodes))
61
62     for i, node in enumerate(boundary_nodes):
63         node_neighbors = list(adj[node])
64         best_nb = -1
65         min_dist = 1e9
66         for nb in node_neighbors:
67             if nb not in boundary_set:
68                 d = (X[node]-X[nb])**2 + (Y[node]-Y[nb])**2
69                 if d < min_dist:
70                     min_dist = d
71                     best_nb = nb
72         if best_nb == -1:
73             for nb in node_neighbors:
74                 d = (X[node]-X[nb])**2 + (Y[node]-Y[nb])**2
75                 if d < min_dist:
76                     min_dist = d
77                     best_nb = nb
78         valid_neighbors[i] = best_nb
79         h_sq[i] = min_dist
80     return valid_neighbors, h_sq
81
82     # =====
83     # Solver NS
84     # =====
85     async def solve_navier_stokes(params, plot_div):
86         # 1. Malha
87         cx, cy = params["cx"], params["cy"]
88
89         # Helper for inlet profile
90         def get_psi_inlet(y_val):

```

```

91         # Default: u=1, psi=y
92         return y_val
93
94     scenario = params.get("scenario", "channel")
95
96     if scenario == "cavity":
97         # Force no geometry/obstruction for cavity
98         boundary_pts = np.empty((0, 2))
99         def mask_func(x, y): return np.ones_like(x, dtype=bool)
100     else:
101         boundary_pts, mask_func = geometry.get_geometry(params["
            geo_type"], params)
102
103     X, Y, triang = generate_mesh_generic(
104         params["L"], params["H"], boundary_pts, mask_func,
105         cx, cy, params["nx"], params["ny"]
106     )
107     n_nodes = len(X)
108
109     # 2. Matrices
110     K, M = assemble_matrices(X, Y, triang)
111
112     # 3. Fronteiras
113     eps = 1e-5
114     inlet_nodes = np.where(X < eps)[0]
115     wall_bottom = np.where(Y < eps)[0]
116     wall_top = np.where(Y > params["H"] - eps)[0]
117
118     # Obstacle nodes (last N points)
119     n_bound = len(boundary_pts)
120     obstacle_nodes = np.arange(n_nodes - n_bound, n_nodes)
121
122     solid_walls = np.concatenate([wall_bottom, wall_top,
        obstacle_nodes])
123     solid_walls = np.unique(solid_walls)
124
125     wall_neighbors, wall_h_sq = get_boundary_normals(X, Y,
        solid_walls, triang)
126

```

```

127     # 4. Inicialização
128     # 4. Inicialização e Condições de Contorno
129     psi = np.zeros(n_nodes)
130     omega = np.zeros(n_nodes)
131
132     scenario = params.get("scenario", "channel")
133
134     if scenario == "cavity":
135         # Lid-Driven Cavity
136         # Walls: Bottom, Left, Right, Top
137         # Psi = 0 on all walls (closed box)
138         # Omega BC drives the flow.
139
140         # Identify walls
141         eps = 1e-5
142         wall_bottom = np.where(Y < eps)[0]
143         wall_top = np.where(Y > params["H"] - eps)[0]
144         wall_left = np.where(X < eps)[0]
145         wall_right = np.where(X > params["L"] - eps)[0]
146
147         # All boundary nodes
148         all_walls = np.concatenate([wall_bottom, wall_top,
149                                     wall_left, wall_right])
150         all_walls = np.unique(all_walls)
151
152         # Psi BC: 0 everywhere on boundary
153         psi[all_walls] = 0.0
154
155         dirichlet_psi = all_walls
156
157         # Omega BC nodes (same as Psi for now, updated in loop)
158         dirichlet_omega = all_walls
159
160         # For Thom's formula, we need neighbors for ALL walls
161         wall_neighbors, wall_h_sq = get_boundary_normals(X, Y,
162                                                         all_walls, triang)
163
164         # Store wall types for specific BC application
165         # Top wall moves: u = 1 -> dPsi/dy = 1

```

```

164         # Thom's formula with moving wall:
165         #  $\omega_w = -2(\psi_{nb} - \psi_w - h u_w) / h^2$ 
166         # Here  $u_w = 1$  for top, 0 for others.
167
168         # We need to map each wall node to its velocity ( $u_{tan}$ )
169         wall_u_tan = np.zeros(len(all_walls))
170
171         # Find indices of top wall nodes within all_walls
172         # This is  $O(N^2)$  worst case but  $N_{bound}$  is small.
173         # Better: Create a map.
174
175         # Let's just iterate
176         for i, node_idx in enumerate(all_walls):
177             if Y[node_idx] > params["H"] - eps:
178                 wall_u_tan[i] = 1.0 # Moving lid
179             else:
180                 wall_u_tan[i] = 0.0
181
182         solid_walls = all_walls # For consistency with loop
183
184     else:
185         # Channel Flow (Original Logic)
186         psi[inlet_nodes] = get_psi_inlet(Y[inlet_nodes])
187         psi[wall_bottom] = 0.0
188
189         # Top Wall BC
190         psi[wall_top] = params["H"]
191
192         # Obstacle BC
193         if params.get("geo_type") == "step":
194             psi[obstacle_nodes] = 0.0
195         else:
196             # Cylinder/Rectangle/etc
197             psi[obstacle_nodes] = get_psi_inlet(cy)
198
199         dirichlet_psi = np.concatenate([inlet_nodes, wall_bottom
200                                         , wall_top, obstacle_nodes])
201         dirichlet_psi = np.unique(dirichlet_psi)

```

```

202         solid_walls = np.concatenate([wall_bottom, wall_top,
203                                         obstacle_nodes])
204
205         solid_walls = np.unique(solid_walls)
206
207         dirichlet_omega = np.concatenate([inlet_nodes,
208                                           solid_walls])
209         dirichlet_omega = np.unique(dirichlet_omega)
210
211         wall_neighbors, wall_h_sq = get_boundary_normals(X, Y,
212                                                         solid_walls, triang)
213
214         # Stationary walls
215         wall_u_tan = np.zeros(len(solid_walls))
216
217         free_psi = np.setdiff1d(np.arange(n_nodes), dirichlet_psi)
218         free_omega = np.setdiff1d(np.arange(n_nodes),
219                                   dirichlet_omega)
220
221         K_free_psi = K[free_psi, :][:, free_psi]
222
223         dt = params["dt"]
224         Re = params["Re"]
225         steps_per_frame = params["steps_per_frame"]
226         total_steps = params["total_steps"]
227
228         current_step = 0
229
230         while current_step < total_steps and not STOP_SIMULATION:
231             for _ in range(steps_per_frame):
232                 # A. Poisson Psi
233                 rhs_psi = M @ omega
234                 b_psi = rhs_psi[free_psi] - K[free_psi, :][:,
235                                                         dirichlet_psi] @ psi[dirichlet_psi]
236                 psi[free_psi] = spsolve(K_free_psi, b_psi)
237
238                 # B. Thom's Formula (Generalized for moving walls)
239                 # Canonical form: omega_w = -2 (psi_nb - psi_w)/h^2
240                 # - 2 u_tan/h

```

```

234         # Derivation (top lid, n inward = -y): Taylor at the
           wall with step -h gives
235         #   psi_nb = psi_w - h (dpsi/dy)_w + (h^2/2) (d2psi/
           dy2)_w
236         # Since u = dpsi/dy, (dpsi/dy)_w = u_tan. With psi
           constant along the wall,
237         # d2psi/dx2 = 0 there, so d2psi/dy2 = -omega_w.
           Solving:
238         #   omega_w = -2 (psi_nb - psi_w + h u_tan) / h^2
239         # Validated against Ghia et al. (1982) at Re=100.
240         psi_nb = psi[wall_neighbors]
241         psi_w = psi[solid_walls]
242         h_vals = np.sqrt(wall_h_sq)
243         omega[solid_walls] = -2.0 * (psi_nb - psi_w + h_vals
           * wall_u_tan) / wall_h_sq
244
245         # C. Velocities
246         elements = triang.triangles
247         mask = triang.mask if triang.mask is not None else
           np.zeros(len(elements), dtype=bool)
248         el_nodes = elements[~mask]
249         x_el = X[el_nodes]
250         y_el = Y[el_nodes]
251         p_el = psi[el_nodes]
252
253         b_coeff = np.column_stack([y_el[:,1]-y_el[:,2], y_el
          [:,2]-y_el[:,0], y_el[:,0]-y_el[:,1]])
254         c_coeff = np.column_stack([x_el[:,2]-x_el[:,1], x_el
          [:,0]-x_el[:,2], x_el[:,1]-x_el[:,0]])
255         area_el = 0.5 * np.abs(x_el[:,0]*(y_el[:,1]-y_el
          [:,2]) + x_el[:,1]*(y_el[:,2]-y_el[:,0]) + x_el
          [:,2]*(y_el[:,0]-y_el[:,1]))
256
257         u_vals = np.sum(p_el * c_coeff, axis=1) / (2 *
           area_el)
258         v_vals = -np.sum(p_el * b_coeff, axis=1) / (2 *
           area_el)
259
260         u_el_full = np.zeros(len(elements))

```

```

261         v_el_full = np.zeros(len(elements))
262         u_el_full[~mask] = u_vals
263         v_el_full[~mask] = v_vals
264
265         # D. Convection
266         C = assemble_convection(X, Y, triang, u_el_full,
267                                 v_el_full)
268
269         # E. Vorticity Transport
270         A = M + (dt / Re) * K
271         rhs = M @ omega - dt * (C @ omega)
272
273         A_free = A[free_omega, :][:, free_omega]
274         b_omega = rhs[free_omega] - A[free_omega, :][:,
275                                     dirichlet_omega] @ omega[dirichlet_omega]
276         omega[free_omega] = spsolve(A_free, b_omega)
277
278         current_step += 1
279
280         # Plot
281         tci = tri.LinearTriInterpolator(triang, psi)
282         (dpsi_dx, dpsi_dy) = tci.gradient(X, Y)
283         u = dpsi_dy
284         v = -dpsi_dx
285
286         fig = plt.figure(figsize=(10, 12))
287         gs = fig.add_gridspec(3, 1, height_ratios=[1, 1, 1])
288
289         ax1 = fig.add_subplot(gs[0])
290         ax2 = fig.add_subplot(gs[1])
291         ax3 = fig.add_subplot(gs[2])
292
293         # Smooth gradient for Psi using tripcolor with Gouraud
294         shading
295
296         # User requested "Blue to Red" (Jet)
297         contour_psi = ax1.tripcolor(triang, psi, shading='
298                                     gouraud', cmap='jet')
299         # Removed discrete contour lines for cleaner look

```



```

295     ax1.triplot(triang, color='k', alpha=0.2, linewidth=0.5)
        # Visible mesh
296     ax1.set_title(f"Navier-Stokes (Step {current_step}) - $
        \\psi$")
297     ax1.set_aspect('equal')
298     ax1.set_xlabel("x [m]")
299     ax1.set_ylabel("y [m]")
300     fig.colorbar(contour_psi, ax=ax1)
301
302     # Smooth gradient for Omega
303     contour_omega = ax2.tripcolor(triang, omega, shading='
        gouraud', cmap='jet')
304     ax2.triplot(triang, color='k', alpha=0.2, linewidth=0.5)
305     ax2.set_title(f"Navier-Stokes (Step {current_step}) - $
        \\omega$")
306     ax2.set_aspect('equal')
307     ax2.set_xlabel("x [m]")
308     ax2.set_ylabel("y [m]")
309     fig.colorbar(contour_omega, ax=ax2)
310
311     xi = np.linspace(0, params["L"], 100)
312     yi = np.linspace(0, params["H"], 100)
313     Xi, Yi = np.meshgrid(xi, yi)
314     (dpsi_dx_grid, dpsi_dy_grid) = tci.gradient(Xi, Yi)
315     u_grid = dpsi_dy_grid
316     v_grid = -dpsi_dx_grid
317     vel_mag_grid = np.sqrt(u_grid**2 + v_grid**2)
318
319     # Streamlines with 'jet'
320     strm = ax3.streamplot(Xi, Yi, u_grid, v_grid, color=
        vel_mag_grid, cmap='jet', density=1.5, linewidth=1,
        arrowsize=1.5)
321     ax3.set_title("Linhas de Corrente")
322     ax3.set_aspect('equal')
323     ax3.set_xlim(0, params["L"])
324     ax3.set_ylim(0, params["H"])
325     ax3.set_xlabel("x [m]")
326     ax3.set_ylabel("y [m]")
327     fig.colorbar(strm.lines, ax=ax3)

```

```

328
329
330
331 plt.tight_layout()
332
333 img_base64 = plot_to_base64(fig)
334
335 # Metricas de validacao (cavidade quadrada): psi_min,
    centro do vortice,
336 # e comparacao com Ghia 1982 para Re=100 quando
    aplicavel.
337 psi_min = float(np.min(psi))
338 i_vort = int(np.argmin(psi))
339 x_vort = float(X[i_vort])
340 y_vort = float(Y[i_vort])
341
342 scenario = params.get("scenario", "")
343 is_square = abs(params["L"] - params["H"]) < 1e-9
344 show_ghia = (scenario == "cavity" and is_square
345              and abs(params["Re"] - 100.0) < 1e-6)
346
347 if show_ghia:
348     ghia_psi_min = -0.1034
349     ghia_center = (0.6172, 0.7344)
350     ghia_row = (
351         "<p class=\"text-sm text-gray-700\"><strong>
352             Referência Ghia et al. (1982) "
353             "Re=100:</strong> "
354             f"ψ_min ≈ {ghia_psi_min}, centro em ({
355                 ghia_center[0]}, {ghia_center[1]})</p>"
356     )
357 else:
358     ghia_row = ""
359
360 metrics_html = (
361     "<div class=\"bg-blue-50 border-l-4 border-blue-400"
362     "p-3 rounded w-full\">"
363     "<h4 class=\"font-semibold text-blue-800 mb-1\">
364         Validação</h4>"

```

```

361         f"<p class=\"text-sm text-gray-700\"><strong>Passo
            :</strong> "
362         f"{current_step} / {total_steps}, Re = {params['Re
            ']:.1f}, "
363         f"malha {params['nx']}]×{params['ny']}]</p>"
364         f"<p class=\"text-sm text-gray-700\"><strong> $\psi_{min}$ 
            :</strong> "
365         f"{psi_min:+.5f} em ({x_vort:.3f}, {y_vort:.3f})</p>
            "
366         f"{ghia_row}"
367         "</div>"
368     )
369
370     plot_div.innerHTML = (
371         "<div class='flex flex-col gap-3 w-full'>"
372         f"<img src='data:image/png;base64,{img_base64}'
            class='max-w-full h-auto rounded shadow-lg' />"
373         f"{metrics_html}"
374         "</div>"
375     )
376     await asyncio.sleep(0.01)
377
378     if not STOP_SIMULATION:
379         plot_div.innerHTML += '<p class="text-center text-green
            -600 font-bold mt-2">Simulação Concluída!</p>'
380     else:
381         plot_div.innerHTML += '<p class="text-center text-red
            -600 font-bold mt-2">Simulação Parada pelo Usuário.</
            p>'
382
383     @when("click", "#run-btn")
384     async def run_handler(event=None):
385         global STOP_SIMULATION
386         STOP_SIMULATION = False
387
388         plot_div = document.getElementById("plot-output")
389         plot_div.innerHTML = "Iniciando..."
390         await asyncio.sleep(0.1)
391

```

```

392     # Show stop button
393     document.getElementById("run-btn").classList.add("hidden")
394     document.getElementById("run-btn").classList.remove("flex")
395     document.getElementById("stop-btn").classList.remove("hidden
        ")
396     document.getElementById("stop-btn").classList.add("flex")
397
398     try:
399         params = {
400             "L": float(document.getElementById("L").value),
401             "H": float(document.getElementById("H").value),
402             "cx": float(document.getElementById("cx").value),
403             "cy": float(document.getElementById("cy").value),
404             "scenario": document.getElementById("scenario").
                value,
405             "geo_type": document.getElementById("geo_type").
                value,
406             "r": float(document.getElementById("r").value),
407             "w": float(document.getElementById("w").value),
408             "h": float(document.getElementById("h").value),
409             "step_h": float(document.getElementById("step_h").
                value),
410             "step_l": float(document.getElementById("step_l").
                value),
411             "nx": int(document.getElementById("nx").value),
412             "ny": int(document.getElementById("ny").value),
413             "Re": float(document.getElementById("Re").value),
414             "dt": float(document.getElementById("dt").value),
415             "steps_per_frame": int(document.getElementById("
                steps_per_frame").value),
416             "total_steps": int(document.getElementById("
                total_steps").value)
417         }
418
419         await solve_navier_stokes(params, plot_div)
420
421     except Exception as e:
422         plot_div.innerHTML = f"Erro: {e}"
423         print(e)

```

```
424     finally:
425         document.getElementById("stop-btn").classList.add("
            hidden")
426         document.getElementById("stop-btn").classList.remove("
            flex")
427         document.getElementById("run-btn").classList.remove("
            hidden")
428         document.getElementById("run-btn").classList.add("flex")
```

Apêndice C

Scripts de Benchmark Computacional

Este apêndice reúne as listagens dos *scripts* utilizados para gerar os tempos reportados na Seção 5.2.4 (Tabela 5.2 e Figura 5.2). Os *solvers* numéricos em `benchmark/core/` são cópias diretas das implementações dos módulos `simulation.py` apresentadas nos Apêndices A e B, com a única alteração da remoção das chamadas ao DOM (`document.getElementById`) e à camada de visualização (`matplotlib`, `imageio`). Os algoritmos numéricos — montagem das matrizes elementares e globais, imposição das condições de contorno, resolução do sistema linear e avanço temporal — são preservados *linha a linha*, de modo a garantir que o tempo medido corresponde de fato ao núcleo numérico executado pelo usuário no navegador. Os mesmos *scripts* são invocados em ambos os ambientes do *benchmark* (CPython local e Pyodide no navegador), assegurando a comparabilidade dos resultados.

C.1 Solver de Condução Permanente 1D Adaptado

Listagem C.1: `benchmark/core/cond_1d.py` — cópia adaptada do solver MEF de condução permanente 1D

```
1  """
```

```

2  Cópia adaptada do solver MEF de Condução Permanente 1D do
    MinervaMesh,
3  para uso nos benchmarks. O algoritmo numérico é idêntico ao de
4  ‘‘simulations/mef/conducao-permanente-barra1d-mef/simulation.py
    ‘‘ --- apenas
5  removidos os imports JS/DOM e o pós-processamento (Matplotlib,
    imageio,
6  escrita no DOM). Os parâmetros entram como argumentos de função.
7  """
8
9  import time
10 import numpy as np
11
12
13 def analytical_solution(x, Q, k, L, T0, TL):
14     C1 = (TL - T0 + (Q * L * L) / (2 * k)) / L
15     C2 = T0
16     return -(Q / (2 * k)) * x * x + C1 * x + C2
17
18
19 def solve_cond_1d(L=1.0, nel=10, T0=0.0, TL=0.0, k=1.0, Q=1.0):
20     """
21     Resolve a condução permanente 1D pelo MEF (elementos
        lineares).
22
23     Retorna um dict com:
24         x      --- coordenadas nodais
25         T      --- temperaturas nodais
26         T_ana  --- solução analítica nos mesmos nós
27         t_assembly --- tempo de montagem da matriz global (s)
28         t_solve  --- tempo de np.linalg.solve (s)
29         t_total  --- soma dos dois acima
30         nn      --- número de nós (= nel + 1)
31     """
32     nn = nel + 1
33     x = np.linspace(0.0, L, nn)
34     h = L / nel
35
36     # ---- Montagem ----

```

```

37     t0 = time.perf_counter()
38     K = np.zeros((nn, nn))
39     b = np.zeros(nn)
40     ke = (k / h) * np.array([[1.0, -1.0], [-1.0, 1.0]])
41     be = (Q * h / 2.0) * np.array([1.0, 1.0])
42     for e in range(nel):
43         n1, n2 = e, e + 1
44         K[n1:n2 + 1, n1:n2 + 1] += ke
45         b[n1:n2 + 1] += be
46
47     # Dirichlet
48     K[0, :] = 0.0
49     K[0, 0] = 1.0
50     b[0] = T0
51     K[-1, :] = 0.0
52     K[-1, -1] = 1.0
53     b[-1] = TL
54     t_assembly = time.perf_counter() - t0
55
56     # ---- Solve ----
57     t0 = time.perf_counter()
58     T = np.linalg.solve(K, b)
59     t_solve = time.perf_counter() - t0
60
61     T_ana = analytical_solution(x, Q, k, L, T0, TL)
62
63     return {
64         "x": x,
65         "T": T,
66         "T_ana": T_ana,
67         "t_assembly": t_assembly,
68         "t_solve": t_solve,
69         "t_total": t_assembly + t_solve,
70         "nn": nn,
71     }

```

C.2 Solver de Transcalor 2D Adaptado

Listagem C.2: benchmark/core/transcalor_2d.py — cópia adaptada do solver MEF de transferência de calor 2D

```
1  """
2  Cópia adaptada do solver MEF de Transferência de Calor 2D do
3  MinervaMesh,
4  para uso nos benchmarks. O algoritmo numérico é idêntico ao de
5  ‘simulations/mef/transcalor/simulation.py’ --- apenas
6  removidos os imports
7  JS/DOM e a geração de frames (Matplotlib, imageio). A malha e os
8  parâmetros
9  entram como argumentos de função em vez de globais de módulo.
10 """
11
12 import time
13 import numpy as np
14 import matplotlib.tri as tri
15 from scipy.sparse import lil_matrix
16 from scipy.sparse.linalg import spsolve
17
18 def element_matrices(coords, k, rho, cv):
19     x0, x1, x2 = coords
20     area = 0.5 * abs(
21         (x1[0] - x0[0]) * (x2[1] - x0[1]) - (x2[0] - x0[0]) * (
22             x1[1] - x0[1])
23     )
24     b = np.array([x1[1] - x2[1], x2[1] - x0[1], x0[1] - x1[1]])
25     c = np.array([x2[0] - x1[0], x0[0] - x2[0], x1[0] - x0[0]])
26     ke = (k / (4 * area)) * (np.outer(b, b) + np.outer(c, c))
27     me = (rho * cv * area / 12.0) * (np.ones((3, 3)) + np.eye(3))
28     return ke, me
29
30 def solve_transcalor_2d(
31     nx=40, ny=40, dt=0.01, n_steps=50,
32     Tbottom=0.0, Ttop=100.0, k=1.0, rho=1.0, cv=1.0,
```

```

32     """
33     Resolve condução de calor 2D transiente em placa unitária
        pelo MEF
34     (triângulos lineares + Crank-Nicolson). Mesma lógica que o m
        ódulo
35     ‘‘transcalor’’ da plataforma, sem geração de frames.
36
37     Retorna um dict com:
38         u          --- temperaturas nodais finais
39         X, Y       --- coordenadas dos nós
40         n_nodes    --- número de nós
41         n_elements --- número de elementos
42         t_mesh     --- tempo de geração de malha
43         t_assembly --- tempo de montagem (M, K) + aplicação de
            CCs em A
44         t_loop     --- tempo total do loop temporal (n_steps ×
            spsolve)
45         t_total    --- soma de assembly + loop (núcleo numérico)
46     """
47     # ---- Malha ----
48     t0 = time.perf_counter()
49     x_lin = np.linspace(0.0, 1.0, nx)
50     y_lin = np.linspace(0.0, 1.0, ny)
51     Xg, Yg = np.meshgrid(x_lin, y_lin)
52     X = Xg.flatten()
53     Y = Yg.flatten()
54     points = np.column_stack((X, Y))
55     triang = tri.Triangulation(X, Y)
56     elements = triang.triangles
57     n_nodes = len(points)
58     t_mesh = time.perf_counter() - t0
59
60     # ---- Montagem ----
61     t0 = time.perf_counter()
62     M = lil_matrix((n_nodes, n_nodes))
63     K = lil_matrix((n_nodes, n_nodes))
64     for el in elements:
65         coords = points[el]
66         ke, me = element_matrices(coords, k, rho, cv)

```

```

67         for i in range(3):
68             for j in range(3):
69                 K[el[i], el[j]] += ke[i, j]
70                 M[el[i], el[j]] += me[i, j]
71     M = M.tocsr()
72     K = K.tocsr()
73
74     u = np.zeros(n_nodes)
75     for i in range(n_nodes):
76         xi, yi = X[i], Y[i]
77         if np.isclose(xi, 0) or np.isclose(xi, 1):
78             u[i] = Tbottom + (Ttop - Tbottom) * yi
79         elif np.isclose(yi, 0):
80             u[i] = Tbottom
81         elif np.isclose(yi, 1):
82             u[i] = Ttop
83
84     u_init = u.copy()
85     boundary_nodes = [
86         i for i in range(n_nodes)
87         if np.isclose(X[i], 0) or np.isclose(X[i], 1)
88         or np.isclose(Y[i], 0) or np.isclose(Y[i], 1)
89     ]
90
91     A = M + dt / 2 * K
92     B = M - dt / 2 * K
93     A = A.tolil()
94     for i in boundary_nodes:
95         A[i, :] = 0
96         A[i, i] = 1
97     A = A.tocsr()
98     t_assembly = time.perf_counter() - t0
99
100     # ---- Loop temporal ----
101     t0 = time.perf_counter()
102     for _ in range(n_steps):
103         b = B @ u
104         b[boundary_nodes] = u_init[boundary_nodes]
105         u = spsolve(A, b)

```

```

106     t_loop = time.perf_counter() - t0
107
108     return {
109         "u": u,
110         "X": X,
111         "Y": Y,
112         "n_nodes": n_nodes,
113         "n_elements": len(elements),
114         "t_mesh": t_mesh,
115         "t_assembly": t_assembly,
116         "t_loop": t_loop,
117         "t_total": t_assembly + t_loop,
118     }

```

C.3 Biblioteca Auxiliar de Cronometragem

Listagem C.3: `benchmark/bench_lib.py` — *context manager* para medição de fases, função de rodadas repetidas com mediana e MAD, e escrita CSV

```

1  """
2  Utilidades de benchmarking compartilhadas entre o runner local (
    CPython) e o
3  runner no navegador (PyScript). Define um *context manager*
    simples para
4  medir fases, função para rodadas repetidas com estatísticas
    robustas
5  (mediana e MAD), e escrita de CSV em formato comum.
6  """
7
8  import csv
9  import statistics
10 import time
11
12
13 class Phase:
14     """Mede o tempo de uma fase do código.
15

```

```

16     Uso:
17         with Phase('solve') as p:
18             ...
19             print(p.dt)
20     """
21
22     def __init__(self, name=""):
23         self.name = name
24         self.dt = 0.0
25
26     def __enter__(self):
27         self._t0 = time.perf_counter()
28         return self
29
30     def __exit__(self, exc_type, exc_val, exc_tb):
31         self.dt = time.perf_counter() - self._t0
32         return False
33
34
35     def run_repeated(fn, n_runs=5, warmup=1):
36         """Executa ``fn`` ``warmup`` vezes (descartadas) e depois ``
37             n_runs`` vezes,
38             coletando o dict de tempos retornado.
39
40             ``fn`` deve retornar um dict com chaves do tipo ``t_assembly
41            ``,
42             ``t_solve``, ``t_total`` etc.---qualquer chave começando com
43             ``t_`` é
44             tratada como métrica de tempo. As demais chaves do último
45             run são
46             preservadas no resultado (e.g., ``n_nodes``, ``n_elements``)
47             .
48
49             Retorna um dict ``{"<chave>_median": ..., "<chave>_mad":
50                 ..., ...
51                 "_n_runs": n_runs}``` agregando as métricas de tempo, e
52                 propaga as
53                 chaves não-temporais do último run.
54         """

```

```

48     for _ in range(warmup):
49         fn()
50
51     samples = [fn() for _ in range(n_runs)]
52     out = {"_n_runs": n_runs}
53     last = samples[-1]
54
55     time_keys = [k for k in last.keys() if isinstance(last[k], (
56         int, float)) and k.startswith("t_")]
57     for k in time_keys:
58         vals = [s[k] for s in samples]
59         med = statistics.median(vals)
60         mad = statistics.median([abs(v - med) for v in vals])
61         out[f"{k}_median"] = med
62         out[f"{k}_mad"] = mad
63         out[f"{k}_min"] = min(vals)
64         out[f"{k}_max"] = max(vals)
65
66     for k, v in last.items():
67         if k.startswith("t_"):
68             continue
69         if isinstance(v, (int, float, str)):
70             out[k] = v
71
72     return out
73
74 def write_csv(rows, path, fieldnames=None):
75     """Escreve uma lista de dicts em CSV. Se 'fieldnames' não
76     for dado,
77     usa a união ordenada das chaves de todos os dicts."""
78     if not rows:
79         return
80     if fieldnames is None:
81         seen = []
82         for r in rows:
83             for k in r.keys():
84                 if k not in seen:
85                     seen.append(k)

```

```

85         fieldnames = seen
86     with open(path, "w", newline="") as f:
87         w = csv.DictWriter(f, fieldnames=fieldnames)
88         w.writeheader()
89         for r in rows:
90             w.writerow({k: r.get(k, "") for k in fieldnames})

```

C.4 Runner Local (CPython)

Listagem C.4: `benchmark/run_benchmark_local.py` — executa a varredura de tamanhos em CPython nativo e grava `results_local.csv`

```

1  """
2  Benchmark local (CPython nativo) dos solvers do MinervaMesh.
3
4  Roda os mesmos solvers que o navegador (importados de ‘‘
    benchmark.core‘‘)
5  em uma varredura de tamanhos, mede tempos com mediana e MAD de 5
    rodadas
6  após 1 warm-up, e salva ‘‘benchmark/results/results_local.csv‘‘.
7
8  Uso:
9      python3 benchmark/run_benchmark_local.py
10 """
11
12 import os
13 import platform
14 import sys
15 import time
16
17 import numpy as np
18
19 HERE = os.path.dirname(os.path.abspath(__file__))
20 if HERE not in sys.path:
21     sys.path.insert(0, HERE)
22
23 from bench_lib import run_repeated, write_csv # noqa: E402

```

```

24 from core.cond_1d import solve_cond_1d # noqa: E402
25 from core.transcalor_2d import solve_transcalor_2d # noqa: E402
26
27
28 COND_1D_NEL = [100, 1000, 10000]
29 TRANSCALOR_NX = [20, 40, 80]
30 TRANSCALOR_N_STEPS = 50
31
32
33 def report_environment():
34     print("=" * 60)
35     print("Ambiente de execução")
36     print("=" * 60)
37     print(f"  Plataforma : {platform.platform()}")
38     print(f"  Processador: {platform.processor()}")
39     print(f"  Python      : {sys.version.split()[0]}")
40     print(f"  NumPy       : {np.__version__}")
41     print("  NumPy backend (np.show_config()):")
42     np.show_config()
43     print("=" * 60)
44
45
46 def bench_cond_1d():
47     rows = []
48     for nel in COND_1D_NEL:
49         print(f"[cond_1d] nel={nel} ...", flush=True)
50         agg = run_repeated(
51             lambda: solve_cond_1d(L=1.0, nel=nel, T0=0.0, TL
52                                   =0.0, k=1.0, Q=1.0),
53             n_runs=5, warmup=1,
54         )
55         agg.update({"case": "cond_1d", "param": f"nel={nel}", "
56                   size_n": nel + 1})
57         rows.append(agg)
58         print(f"  median t_total = {agg['t_total_median
59               ']*1000:.3f} ms (MAD {agg['t_total_mad']*1000:.3f} ms
60               )")
61     return rows

```



```

59
60 def bench_transcalor_2d():
61     rows = []
62     for nx in TRANSCALOR_NX:
63         print(f"[transcalor_2d] nx=ny={nx}, n_steps={
64             TRANSCALOR_N_STEPS} ...", flush=True)
65         # Reduzir n_runs para 3 quando nx=80 para não inflar o
66         wall-clock
67         n_runs = 3 if nx >= 80 else 5
68         agg = run_repeated(
69             lambda nx=nx: solve_transcalor_2d(
70                 nx=nx, ny=nx, dt=0.01, n_steps=
71                 TRANSCALOR_N_STEPS,
72             ),
73             n_runs=n_runs, warmup=1,
74         )
75         agg.update({
76             "case": "transcalor_2d",
77             "param": f"nx=ny={nx}, n_steps={TRANSCALOR_N_STEPS}"
78             ,
79             "size_n": nx * nx,
80         })
81         rows.append(agg)
82         print(f"    median t_total = {agg['t_total_median']:.3f} s
83             (MAD {agg['t_total_mad']:.3f} s)")
84     return rows
85
86 def main():
87     report_environment()
88     t_start = time.time()
89
90     rows = []
91     rows.extend(bench_cond_1d())
92     rows.extend(bench_transcalor_2d())
93
94     out_path = os.path.join(HERE, "results", "results_local.csv"
95         )
96     fieldnames = [

```

```

92         "case", "param", "size_n", "_n_runs",
93         "n_nodes", "n_elements", "nn",
94         "t_total_median", "t_total_mad", "t_total_min", "
           t_total_max",
95         "t_assembly_median", "t_assembly_mad",
96         "t_solve_median", "t_solve_mad",
97         "t_loop_median", "t_loop_mad",
98         "t_mesh_median", "t_mesh_mad",
99     ]
100     write_csv(rows, out_path, fieldnames=fieldnames)
101     print(f"\nCSV salvo em {out_path}")
102     print(f"Wall-clock total: {time.time() - t_start:.1f} s")
103
104
105 if __name__ == "__main__":
106     main()

```

C.5 Runner no Navegador (PyScript)

Listagem C.5: `benchmark/run_benchmark_browser.py` — mesmo *benchmark* executado sob Pyodide; o usuário inicia a execução por um botão na página `run_benchmark_browser.html` e obtém `results_browser.csv` via *download* no navegador

```

1  """
2  Benchmark no navegador (PyScript / Pyodide / WebAssembly).
3
4  Mesmo conjunto de varreduras do runner local: cond_1d (nel in
   {100, 1000, 10000})
5  e transcalor_2d (nx=ny in {20, 40, 80}, n_steps=50). Mediana e
   MAD de 5 runs
6  após 1 warm-up. Resultado é exposto na página e oferecido como
   download de CSV.
7
8  Carregado via ‘run_benchmark_browser.html’. Os arquivos ‘
   bench_lib.py’,

```

```

9  'core/__init__.py', 'core/cond_1d.py', 'core/transcalor_2d.
    py' são
10 fornecidos pelo bloco '[[fetch]]' em 'config.toml'.
11 """
12
13 import io
14 import platform
15 import sys
16 import time
17
18 from js import Blob, URL, document, navigator
19 from pyscript import when
20
21 from bench_lib import run_repeated, write_csv
22 from core.cond_1d import solve_cond_1d
23 from core.transcalor_2d import solve_transcalor_2d
24
25
26 COND_1D_NEL = [100, 1000, 10000]
27 TRANSCALOR_NX = [20, 40, 80]
28 TRANSCALOR_N_STEPS = 50
29
30
31 def env_html():
32     return (
33         "<ul class='text-sm leading-relaxed'"
34         f"<li><b>Python:</b> {sys.version.split()[0]}</li>"
35         f"<li><b>Plataforma (Pyodide):</b> {platform.platform()}</li>"
36         f"<li><b>navigator.userAgent:</b> <code class='text-xs'"
37         f">{navigator.userAgent}</code></li>"
38         "</ul>"
39     )
40
41 def append_log(msg):
42     log_el = document.getElementById("bench-log")
43     log_el.innerHTML = log_el.innerHTML + f"<div>{msg}</div>"
44

```

```

45
46 def _bench_cond_1d():
47     rows = []
48     for nel in COND_1D_NEL:
49         append_log(f"[cond_1d] nel={nel} ...")
50         agg = run_repeated(
51             lambda nel=nel: solve_cond_1d(L=1.0, nel=nel, T0
52                 =0.0, TL=0.0, k=1.0, Q=1.0),
53             n_runs=5, warmup=1,
54         )
55         agg.update({"case": "cond_1d", "param": f"nel={nel}", "
56             size_n": nel + 1})
57         rows.append(agg)
58         append_log(
59             f" median t_total = {agg['t_total_median']*1000:.3f
60                 } ms "
61             f"(MAD {agg['t_total_mad']*1000:.3f} ms)"
62         )
63     return rows
64
65 def _bench_transcalor_2d():
66     rows = []
67     for nx in TRANSCALOR_NX:
68         append_log(f"[transcalor_2d] nx=ny={nx}, n_steps={
69             TRANSCALOR_N_STEPS} ...")
70         n_runs = 3 if nx >= 80 else 5
71         agg = run_repeated(
72             lambda nx=nx: solve_transcalor_2d(
73                 nx=nx, ny=nx, dt=0.01, n_steps=
74                     TRANSCALOR_N_STEPS,
75                 ),
76             n_runs=n_runs, warmup=1,
77         )
78         agg.update({
79             "case": "transcalor_2d",
80             "param": f"nx=ny={nx}, n_steps={TRANSCALOR_N_STEPS}"
81             ,
82             "size_n": nx * nx,

```

```

78         })
79         rows.append(agg)
80         append_log(
81             f"    median t_total = {agg['t_total_median']:.3f} s "
82             f"(MAD {agg['t_total_mad']:.3f} s)"
83         )
84     return rows
85
86
87 def offer_download(csv_text, filename):
88     blob = Blob.new([csv_text], {"type": "text/csv"})
89     url = URL.createObjectURL(blob)
90     a = document.createElement("a")
91     a.href = url
92     a.download = filename
93     a.textContent = f"Baixar {filename}"
94     a.className = "inline-block mt-3 px-4 py-2 bg-blue-600 text-
        white rounded shadow hover:bg-blue-700"
95     container = document.getElementById("bench-download")
96     container.innerHTML = ""
97     container.appendChild(a)
98
99
100 @when("click", "#bench-run")
101 def on_run(event=None):
102     document.getElementById("bench-log").innerHTML = ""
103     document.getElementById("bench-download").innerHTML = ""
104     document.getElementById("bench-run").disabled = True
105     document.getElementById("bench-run").textContent = "Rodando
        ..."
106
107     document.getElementById("bench-env").innerHTML = env_html()
108
109     t0 = time.time()
110     rows = []
111     rows.extend(_bench_cond_1d())
112     rows.extend(_bench_transcator_2d())
113     elapsed = time.time() - t0
114     append_log(f"<b>Wall-clock total:</b> {elapsed:.1f} s")

```

```

115
116     fieldnames = [
117         "case", "param", "size_n", "_n_runs",
118         "n_nodes", "n_elements", "nn",
119         "t_total_median", "t_total_mad", "t_total_min", "
            t_total_max",
120         "t_assembly_median", "t_assembly_mad",
121         "t_solve_median", "t_solve_mad",
122         "t_loop_median", "t_loop_mad",
123         "t_mesh_median", "t_mesh_mad",
124     ]
125     buf = io.StringIO()
126
127     # write_csv escreve em arquivo; faço aqui em memória usando
        csv.DictWriter direto
128     import csv as _csv
129     seen_keys = []
130     for r in rows:
131         for k in r.keys():
132             if k not in seen_keys:
133                 seen_keys.append(k)
134     use_fields = [f for f in fieldnames if f in seen_keys]
135     w = _csv.DictWriter(buf, fieldnames=use_fields)
136     w.writeheader()
137     for r in rows:
138         w.writerow({k: r.get(k, "") for k in use_fields})
139
140     offer_download(buf.getvalue(), "results_browser.csv")
141
142     document.getElementById("bench-run").disabled = False
143     document.getElementById("bench-run").textContent = "Rodar
        benchmark novamente"
144
145
146     # Confirma carregamento bem-sucedido
147     document.getElementById("bench-status").textContent = "Pyodide
        pronto. Clique em 'Rodar benchmark'."

```